

University Gate Pass Management System (UGPMS)

Comprehensive Enterprise-Grade Software Requirements Specification (SRS) & System Design Document (SDD)

Document Metadata

- **Project Name:** University Gate Pass Management System (UGPMS)
 - **Document Type:** Consolidated Software Requirements Specification (SRS), System Design Document (SDD), Solution Architecture & Technical Design Document
 - **Target Audience:** Software Architects, Developers, DevOps Engineers, Security Teams, Project Managers, University Administrators
 - **Version:** 1.0.0
 - **Release Date:** May 2026
 - **Classification:** Internal / Enterprise-Grade Technical Blueprint
-

Table of Contents

- 1. [Section 1 & 2 — Executive Summary & Business Requirements](#)
 - 2. [Section 3 & 4 — Stakeholder Analysis & RBAC Matrix](#)
 - 3. [Section 5 — Functional Requirements](#)
 - 4. [Section 6 — End-to-End Workflows](#)
 - 5. [Section 7 — React Frontend Architecture](#)
 - 6. [Section 8 — Node.js Backend Architecture](#)
 - 7. [Section 9 — System Architecture](#)
 - 8. [Section 10 & 11 — Database Design & Prisma Schema](#)
 - 9. [Section 12 — REST API Documentation](#)
 - 10. [Section 13 — Security Architecture](#)
 - 11. [Section 14 — Real-Time System Design](#)
 - 12. [Section 15 — Deployment, Docker & CI/CD](#)
 - 13. [Section 16 — Monitoring, Scalability & DR](#)
 - 14. [Section 17 — Future Enhancements](#)
-

Section 1 & 2 — Executive Summary & Business Requirements

1.1 Project Introduction

The **University Gate Pass Management System (UGPMS)** is an enterprise-grade, full-stack web application designed to digitize, automate, and secure every aspect of visitor access control across a university campus.

Built on a modern Node.js + React architecture, the system replaces paper-based registers with real-time, QR-verified digital passes, end-to-end audit trails, and role-based access control.

1.2 Business Problem

Universities and research institutions face persistent physical security challenges:

Problem	Impact
Paper-based visitor registers	No search, no analytics, easy to forge
Manual identity verification	Long queues, human error, frustrated visitors
No digital audit trail	Compliance failures, unable to investigate incidents
Unauthorized campus entry	Safety risk to students and staff
Hostel guest management gaps	Overnight guests untracked, regulatory exposure
Vehicle entry uncontrolled	Parking abuse, stolen vehicle risk
Event crowd management	No pre-registration, bottleneck at gate
No real-time security alerts	Guards react slowly to suspicious activity

1.3 Objectives

1. Replace all paper registers with a centralized digital system.
2. Implement QR-code-based, cryptographically signed gate passes.
3. Enforce RBAC so only authorized persons can approve or reject passes.
4. Provide real-time entry/exit logs visible to security and administrators.
5. Enable hostel wardens to manage overnight guest approvals digitally.
6. Track vehicles entering and exiting campus.
7. Generate analytics dashboards and exportable audit reports.
8. Support emergency lockdown across all gates simultaneously.
9. Integrate email notifications for every stage of the pass lifecycle.
10. Achieve sub-3-second QR verification at security checkpoints.

1.4 Vision

A fully digitized, zero-paper, real-time campus access control ecosystem that enhances security, improves visitor experience, and provides administration with actionable intelligence.

1.5 Mission

To deliver an implementation-ready, scalable, and secure gate pass management platform that every university security team can operate without specialized IT knowledge, while giving administrators complete visibility over campus access events.

1.6 Key Benefits

Benefit	Description
Security	Cryptographically signed QR passes prevent forgery
Efficiency	Average gate clearance drops from 5 min to under 30 sec
Compliance	Full audit logs satisfy regulatory and accreditation requirements
Experience	Visitors receive pass by email; no form-filling at the gate
Analytics	Real-time dashboards expose peak hours, frequent visitors, anomalies
Scalability	Horizontal scaling supports 100,000 visits/day
Cost	Eliminates paper, printing, and manual record-keeping costs

1.7 Stakeholders

Stakeholder	Role
University Administration	Sponsor, policy owner
Security Department	Primary operator
Students	Request visitor/guest passes
Faculty	Request visitor passes, approve departmental visits
Administrative Staff	Request contractor and vendor passes
Hostel Wardens	Approve overnight guest stays
Security Guards	Scan QR codes at checkpoints
IT Department	Deployment and maintenance
Visitors	End recipients of gate passes
Regulatory Bodies	Audit compliance consumers

1.8 Scope

In Scope

- Student, Faculty, Staff, Parent, Hostel Guest, Contractor, Vehicle, and Event pass types
- Web application (React frontend + Node.js backend)
- QR generation and scanning
- Email notifications
- Real-time WebSocket alerts
- Analytics and reporting
- Docker-based deployment
- CI/CD pipeline

Out of Scope (Phase 1)

- Mobile native apps (iOS/Android)
- Face recognition / biometric integration
- RFID hardware integration
- Multi-campus federation

2.1 Current Challenges (Detailed)

2.1.1 Unauthorized Entry

Security guards manually verify identity documents. Forged IDs or social engineering easily bypass this control. There is no cross-check against a pre-approved visitor list.

2.1.2 Manual Registers

Visitor name, ID, purpose, and time are written in physical log books. These cannot be searched, exported, or audited in real time. Books are often lost, damaged, or illegible.

2.1.3 Slow Verification

Peak hours (morning 8–10 AM, afternoon 12–2 PM) see queues of 50–100 visitors at a single gate. Manual checks take 3–8 minutes per visitor.

2.1.4 Missing Audit Trails

When an incident occurs, investigators cannot reconstruct who was on campus, which gate they used, or when they exited. CCTV alone is insufficient.

2.1.5 Poor Visitor Tracking

Overstayed visitors are not flagged. There is no mechanism to alert security when a pass expires but the visitor has not exited.

2.2 Expected Outcomes

Outcome	Measurement
All passes digital	0 paper passes issued within 30 days of go-live
QR verification time	< 30 seconds per visitor at gate
Audit trail completeness	100% of entry/exit events logged
Unauthorized entry incidents	Reduction of > 80% within 90 days
Visitor satisfaction score	> 4/5 in post-visit survey
Report generation time	< 10 seconds for any date range report

2.3 KPIs

KPI	Target	Measurement Frequency
Daily active passes	Baseline + growth	Daily
Average QR scan time	< 5 seconds	Real-time
Pass approval turnaround	< 2 hours	Daily
System uptime	99.9%	Monthly
Failed login attempts blocked	100% of brute force	Real-time
Expired pass alerts sent	100% of expiring passes	Hourly
Audit log completeness	100%	Daily
Email delivery rate	> 98%	Daily

2.4 Success Metrics

- **Month 1:** All security gates equipped and staff trained; paper register retired.
- **Month 3:** 95% of all campus visits processed digitally; < 3 unauthorized entry incidents.
- **Month 6:** Analytics dashboard actively used by administration weekly; first compliance audit passed using digital logs.
- **Year 1:** System handling peak 10,000 passes/day without degradation; NPS score > 40 from visitors.

Section 3 & 4 — Stakeholder Analysis & RBAC Matrix

3.1 Stakeholder Profiles

3.1.1 Visitor

Attribute	Detail
Description	Any external person arriving at campus — guest, vendor, parent, contractor
Responsibilities	Provide accurate personal information; carry valid government ID; follow campus rules
Permissions	View own pass; scan self QR at kiosk
Restrictions	Cannot access internal system; pass is time-limited and gate-specific
Access Rights	Public pass verification page only

3.1.2 Student

Attribute	Detail
Description	Enrolled student requesting passes for personal visitors or hostel guests
Responsibilities	Vouch for visitors; ensure visitor compliance; cancel pass if visitor cannot attend

Attribute	Detail
Permissions	Create visitor/hostel guest pass; view own pass history; receive notifications
Restrictions	Cannot approve own pass; cannot create event passes; cannot view other students' passes
Access Rights	Student portal — pass request, history, profile

3.1.3 Faculty

Attribute	Detail
Description	Academic staff inviting researchers, collaborators, or personal visitors
Responsibilities	Verify visitor purpose is academic/professional; provide office location
Permissions	Create visitor passes; approve departmental visitor passes; view own department visitor log
Restrictions	Cannot modify security configurations; cannot access hostel module
Access Rights	Faculty portal — pass management, department reports

3.1.4 Staff

Attribute	Detail
Description	Administrative, maintenance, and operational staff
Responsibilities	Request contractor and vendor passes; manage vehicle entry for deliveries
Permissions	Create contractor/vendor passes; create vehicle entry passes; view own requests
Restrictions	Cannot approve passes; cannot access academic visitor logs
Access Rights	Staff portal — pass request, vehicle request

3.1.5 Security Guard

Attribute	Detail
Description	Personnel stationed at gates to verify entry/exit
Responsibilities	Scan QR codes; verify physical ID matches pass; log manual overrides
Permissions	Scan QR; view pass details; log entry/exit; flag suspicious activity; issue temporary pass
Restrictions	Cannot create or approve regular passes; cannot modify pass data
Access Rights	Security terminal app — scan, log, alert

3.1.6 Hostel Warden

Attribute	Detail
Description	Warden responsible for a specific hostel block
Responsibilities	Approve/reject overnight guest requests; monitor guest check-in/out; enforce hostel rules
Permissions	Approve/reject hostel guest passes; set hostel capacity limits; view hostel guest log
Restrictions	Limited to own hostel block; cannot manage other pass types
Access Rights	Hostel warden portal — guest approvals, occupancy dashboard

3.1.7 Security Administrator

Attribute	Detail
Description	Head of security or security operations manager
Responsibilities	Monitor all gate activity; issue lockdowns; configure security rules; investigate incidents
Permissions	View all passes; revoke any pass; trigger lockdown; access full audit log; export reports
Restrictions	Cannot modify system configuration or user roles
Access Rights	Security operations center dashboard

3.1.8 University Administrator

Attribute	Detail
Description	Senior administrative officer managing policy and operations
Responsibilities	Set campus-wide access policies; manage departments; review analytics; manage user accounts
Permissions	All above + manage faculty/staff accounts; configure approval workflows; set pass validity rules
Restrictions	Cannot modify database schemas or system infrastructure
Access Rights	Administration portal — full operational control

3.1.9 Super Administrator

Attribute	Detail
Description	IT system owner with full system access
Responsibilities	System configuration; role management; infrastructure monitoring; disaster recovery
Permissions	Unrestricted access to all modules, configurations, and audit logs
Restrictions	All actions are audit-logged; cannot delete audit logs
Access Rights	Full system — all portals, system config, monitoring

4.1 RBAC Permission Matrix

The following matrix defines permissions across all system modules for each role.

Legend:  Full Access |  Limited/Own Only |  No Access |  View Only

4.1.1 Pass Management

Permission	Visitor	Student	Faculty	Staff	Guard	Warden	Sec Admin	Uni Admin	Super Admin
View own pass									
View all passes			 dept			 hostel			
Create visitor pass									
Create hostel pass									
Create event pass									
Create vehicle pass									
Approve pass			 dept			 hostel			
Reject pass			 dept			 hostel			
Revoke active pass		 own	 own	 own		 hostel			
Delete pass									
Bulk generate passes									
Export passes			 dept			 hostel			

4.1.2 Entry & Exit

Permission	Visitor	Student	Faculty	Staff	Guard	Warden	Sec Admin	Uni Admin	Super Admin
Scan QR									

Permission	Visitor	Student	Faculty	Staff	Guard	Warden	Sec Admin	Uni Admin	Super Admin
Log entry	✗	✗	✗	✗	✓	✗	✓	✗	✓
Log exit	✗	✗	✗	✗	✓	✗	✓	✗	✓
View entry logs	✗	♦ own	♦ dept	✗	✓	♦ hostel	✓	✓	✓
Manual override entry	✗	✗	✗	✗	✓	✗	✓	✗	✓
Trigger lockdown	✗	✗	✗	✗	♦ gate	✗	✓	✓	✓

4.1.3 User Management

Permission	Visitor	Student	Faculty	Staff	Guard	Warden	Sec Admin	Uni Admin	Super Admin
View own profile	✓	✓	✓	✓	✓	✓	✓	✓	✓
Edit own profile	✓	✓	✓	✓	✓	✓	✓	✓	✓
Create user accounts	✗	✗	✗	✗	✗	✗	✗	✓	✓
Deactivate users	✗	✗	✗	✗	✗	✗	✗	✓	✓
Assign roles	✗	✗	✗	✗	✗	✗	✗	♦	✓
Manage roles/permissions	✗	✗	✗	✗	✗	✗	✗	✗	✓

4.1.4 Analytics & Reporting

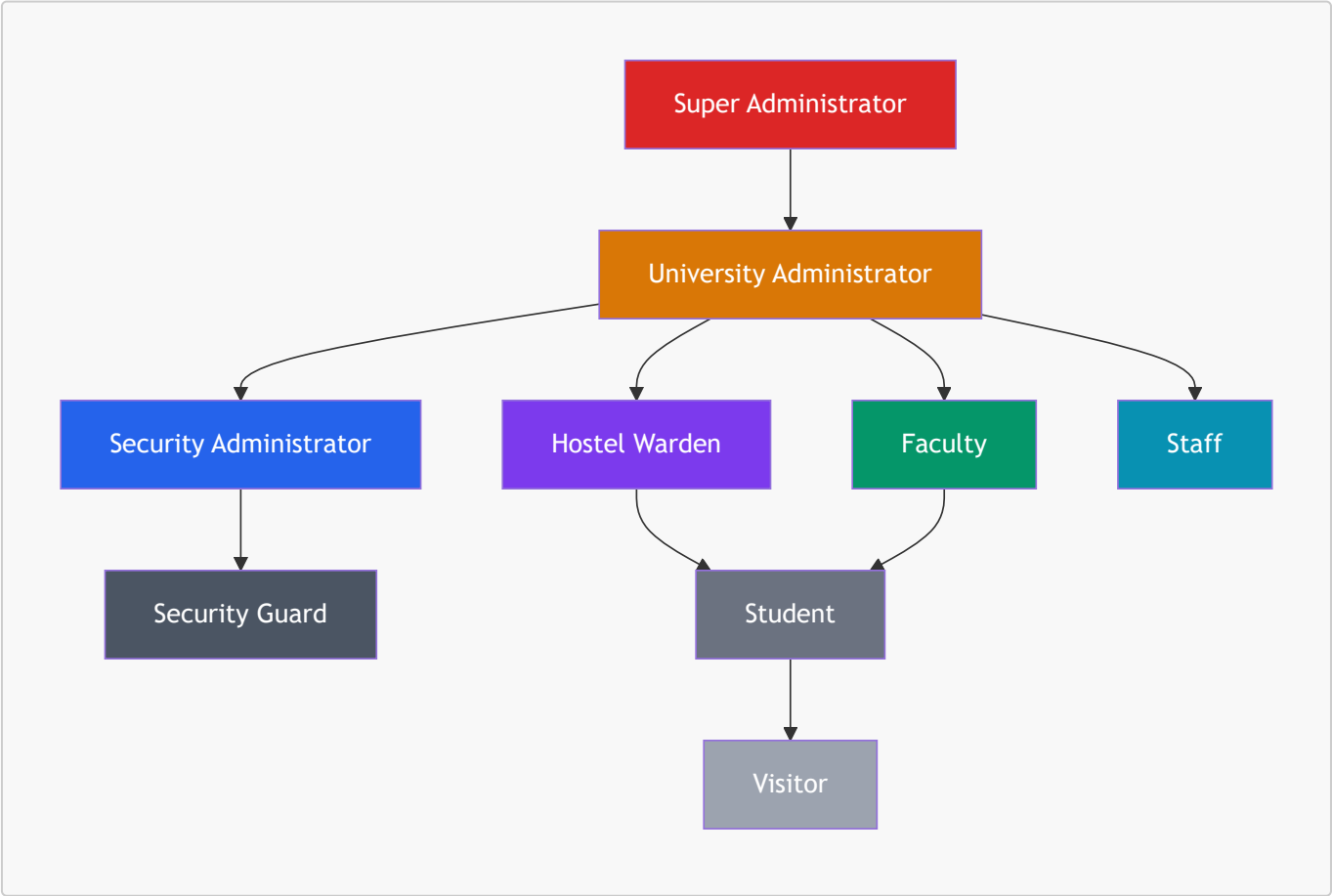
Permission	Visitor	Student	Faculty	Staff	Guard	Warden	Sec Admin	Uni Admin	Super Admin
View own stats	✓	✓	✓	✓	✗	✗	✓	✓	✓
View dept reports	✗	✗	✓	✗	✗	✗	✓	✓	✓
View campus-wide reports	✗	✗	✗	✗	✗	✗	✓	✓	✓
Export CSV/PDF	✗	✗	♦ dept	✗	✗	♦ hostel	✓	✓	✓

Permission	Visitor	Student	Faculty	Staff	Guard	Warden	Sec Admin	Uni Admin	Super Admin
View audit logs	✗	✗	✗	✗	✗	✗	✓	✓	✓
View security alerts	✗	✗	✗	✗	✓	✗	✓	✓	✓

4.1.5 System Configuration

Permission	Guard	Warden	Sec Admin	Uni Admin	Super Admin
Configure gates	✗	✗	✓	✓	✓
Configure pass rules	✗	◆ hostel	✓	✓	✓
Configure notifications	✗	✗	✓	✓	✓
Configure approval workflows	✗	✗	✗	✓	✓
System settings	✗	✗	✗	✗	✓
Backup / Restore	✗	✗	✗	✗	✓
View monitoring dashboard	✗	✗	✓	✓	✓

4.2 Role Hierarchy Diagram



Section 5 — Functional Requirements

5.1 Authentication Module

5.1.1 Features

Feature	Description
Registration	Self-registration for students/faculty with university email domain validation
Login	Email + password with optional MFA
Logout	Invalidate access token; rotate refresh token
Password Reset	Email OTP flow with 15-minute expiry
MFA	TOTP (Google Authenticator compatible) or email OTP
JWT Authentication	Short-lived access tokens (15 min) + long-lived refresh tokens (7 days)
Refresh Tokens	Rotate on use; stored hashed in Redis; revocable family
Account Lockout	5 failed attempts → 30-minute lockout → admin unlock
Session Management	List active sessions; revoke individual sessions

5.1.2 Business Rules

- **BR-AUTH-01:** Only university domain emails (@university.edu) may self-register as Student/Faculty.
- **BR-AUTH-02:** Security Guards and Wardens are created by administrators only.
- **BR-AUTH-03:** MFA is mandatory for Security Administrators and Super Administrators.
- **BR-AUTH-04:** Refresh tokens are single-use (rotation); if reuse is detected, entire family is revoked (token theft protection).
- **BR-AUTH-05:** All authentication events (login, logout, failure, password change) are written to the audit log.
- **BR-AUTH-06:** Access tokens are never stored in `localStorage`; stored in memory with refresh token in `httpOnly` cookie.

5.1.3 Validations

Field	Rule
Email	Valid format; domain whitelist check
Password	Min 8 chars; at least 1 uppercase, 1 number, 1 special char
OTP	6 digits; expires in 15 minutes; single use
TOTP	6 digits; 30-second window ±1 tolerance
University ID	Alphanumeric, 8–12 chars

5.1.4 Edge Cases

- User registers with email that already exists → return generic "check email" response (no enumeration).
- Password reset requested for non-existent account → same success response (no enumeration).
- MFA device lost → admin-assisted recovery with identity verification.
- Concurrent login from two devices → both allowed; sessions listed in profile.

5.2 Visitor Management Module

5.2.1 Features

Feature	Description
Visitor Registration	Collect name, phone, email, ID type, ID number, photo
ID Upload	Upload government ID image (JPEG/PNG/PDF, max 5 MB)
Visitor Verification	Admin/guard marks visitor as verified after physical ID check
Visitor Categories	General, Academic, Government, VIP, Contractor, Vendor, Parent
Blacklist	Add visitor to blacklist; automatically reject future pass requests
Repeat Visitor	System recognizes returning visitors by phone/email; auto-fills profile

5.2.2 Business Rules

- **BR-VIS-01:** A visitor record is created once per unique ID number. Subsequent pass requests reuse the record.
- **BR-VIS-02:** Blacklisted visitors cannot be issued passes; requester is notified.
- **BR-VIS-03:** Visitor ID documents are stored encrypted in MinIO; only security administrators can download originals.
- **BR-VIS-04:** Visitor data is retained for 2 years post-last-visit then auto-archived.
- **BR-VIS-05:** VIP visitors bypass normal approval; pass is auto-approved on creation.

5.2.3 Validations

Field	Rule
Name	2–100 chars; letters and spaces only
Phone	E.164 format
Email	Valid format (optional)
ID Type	Enum: AADHAAR, PASSPORT, DRIVING_LICENSE, VOTER_ID, OTHER
ID Number	Pattern match per ID type
ID Photo	JPEG/PNG/PDF; max 5 MB; virus-scanned before store

5.3 Pass Request Module

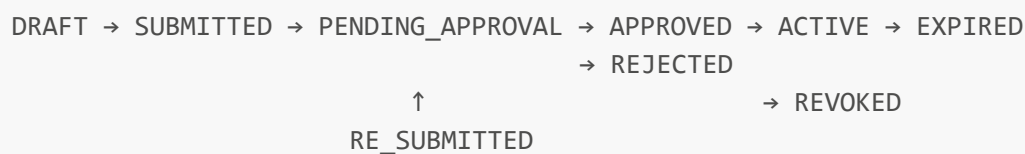
5.3.1 Features

Feature	Description
Pass Creation	Requester fills pass form specifying visitor, purpose, date/time, gate
Pass Types	VISITOR, HOSTEL_GUEST, VEHICLE, EVENT, CONTRACTOR, PARENT
Approval Workflow	Single or multi-level approval based on pass type and policy
Pass Rejection	Approver provides rejection reason; requester notified
Pass Expiry	Automatic expiry at configured end time; guard notified
Pass Revocation	Requester or approver can revoke an active pass; visitor notified
Re-Apply	Requester can clone a rejected/expired pass to re-submit

5.3.2 Business Rules

- **BR-PASS-01:** A student may have at most 3 active passes simultaneously.
- **BR-PASS-02:** A faculty may have at most 10 active passes simultaneously.
- **BR-PASS-03:** Pass validity cannot exceed 30 days from approval date.
- **BR-PASS-04:** Hostel guest passes require warden approval; other passes require faculty/staff supervisor approval.
- **BR-PASS-05:** Contractor passes require Security Administrator approval.
- **BR-PASS-06:** Emergency lockdown automatically suspends all active passes.
- **BR-PASS-07:** QR code is generated only after final approval.
- **BR-PASS-08:** Each pass has a unique, non-guessable token (UUIDv4 + HMAC signature).

5.3.3 Pass Status State Machine



5.4 QR Pass Module

5.4.1 Features

Feature	Description
QR Generation	Generate QR on approval using qrcode library
QR Payload	Encrypted JSON: passId, visitorId, validFrom, validTo, allowedGates, hmacSignature
QR Encryption	AES-256-CBC encrypt payload before encoding
Dynamic Validation	Each scan records timestamp; prevents replay across session

Feature	Description
Expiry Logic	QR invalid after <code>validTo</code> ; system checks server-side
One-Time Entry Mode	Optional: QR invalidated after first successful scan (event passes)

5.4.2 QR Payload Structure

```
{
  "v": 1,
  "pid": "uuid-of-pass",
  "vid": "uuid-of-visitor",
  "gates": ["MAIN", "NORTH"],
  "from": 1735000000,
  "to": 1735086400,
  "sig": "hmac-sha256-hex"
}
```

5.4.3 Validation Logic

1. Decrypt AES-256 payload using server key.
2. Verify HMAC signature against known secret.
3. Check `from` ≤ current time ≤ `to`.
4. Verify pass status is `ACTIVE` in database.
5. Verify scanning gate is in `gates` array.
6. If event/one-time: mark pass as `USED`; reject subsequent scans.
7. Log scan result (success or failure reason) to `EntryLogs`.

5.5 Entry & Exit Module

5.5.1 Features

Feature	Description
QR Scan	Camera-based or QR reader device scan at gate terminal
Entry Logging	Record visitor, gate, guard, timestamp, vehicle (if any)
Exit Logging	Match entry log; compute duration on campus
Re-Entry Validation	Multi-entry passes allow re-entry within validity window
Overstay Alert	Trigger alert if visitor has not exited 30 minutes before pass expiry
Manual Override	Guard can manually record entry/exit with reason and supervisor PIN

5.5.2 Business Rules

- **BR-ENTRY-01:** Only QR codes issued by this system are accepted. External QR codes are rejected.

- **BR-ENTRY-02:** A visitor cannot have two simultaneous active entry logs (must exit before re-entry, unless multi-entry pass).
 - **BR-ENTRY-03:** Exit can be logged without a corresponding QR scan (guard records visual confirmation).
 - **BR-ENTRY-04:** All manual overrides require supervisor approval code and are flagged in audit log.
 - **BR-ENTRY-05:** System sends real-time push notification to requester when visitor enters campus.
-

5.6 Hostel Guest Module

5.6.1 Features

Feature	Description
Guest Request	Student requests guest pass specifying room, duration, guest details
Warden Approval	Hostel warden reviews and approves/rejects with reason
Overnight Stay	Tracks check-in/check-out times; calculates nights stayed
Capacity Management	Warden sets max simultaneous guests per block
Curfew Enforcement	System rejects entry after configured curfew time unless warden override
Guest Log	Full log of all guests per hostel block, exportable

5.6.2 Business Rules

- **BR-HOSTEL-01:** Student can host at most 2 guests simultaneously.
 - **BR-HOSTEL-02:** Guest stays exceeding 3 nights require Security Administrator awareness notification.
 - **BR-HOSTEL-03:** Guests must exit before 10 PM unless overnight pass is explicitly approved.
 - **BR-HOSTEL-04:** Opposite-gender guest policies are configurable per hostel by warden.
-

5.7 Vehicle Module

5.7.1 Features

Feature	Description
Vehicle Registration	Register vehicle: number plate, type, owner, purpose
Vehicle Pass	Time-bound pass for non-campus vehicles
Entry Tracking	Guard scans/enters number plate; linked to visitor pass
ANPR-Ready	Number plate field indexed for future ANPR integration
Parking Assignment	Optional: assign parking zone on entry

5.7.2 Business Rules

- **BR-VEH-01:** Registered campus vehicles (faculty/staff) do not need a pass; they have permanent access records.
- **BR-VEH-02:** Visitor vehicles require an active vehicle pass linked to a visitor pass.

- **BR-VEH-03:** Commercial vehicles (delivery, contractor) require staff-authorized vehicle pass.

5.8 Event Management Module

5.8.1 Features

Feature	Description
Event Creation	Admin creates event with venue, date, capacity, allowed entry windows
Event Pass Request	Pre-registration form for external attendees
Bulk Pass Generation	Generate and email passes to all registered attendees in one click
Capacity Control	System rejects new registrations when capacity is reached
Check-in Tracking	Real-time check-in count visible on event dashboard

5.8.2 Business Rules

- **BR-EVENT-01:** Event passes are valid only for the event date $\pm$ 2 hours.
- **BR-EVENT-02:** Bulk generation initiates a background job; admin is notified on completion.
- **BR-EVENT-03:** Cancelled events auto-revoke all associated passes and notify registered attendees.

5.9 Notifications Module

5.9.1 Features

Channel	Triggers
Email	Pass created, approved, rejected, revoked, expired (warning), visitor entered
In-App	All above + real-time security alerts
Real-Time (Socket.io)	Entry/exit events, approval status changes, lockdown alerts

5.9.2 Email Templates

Template ID	Subject	Recipient
PASS_CREATED	Your visitor pass request is submitted	Requester
PASS_APPROVED	Your visitor pass is approved — QR attached	Requester + Visitor
PASS_REJECTED	Visitor pass rejected	Requester
PASS_EXPIRY_WARNING	Pass expires in 1 hour	Requester
VISITOR_ENTERED	Your visitor has arrived on campus	Requester
VISITOR_EXITED	Your visitor has left campus	Requester
PASS_REVOKED	Your pass has been revoked	Visitor + Requester

Template ID	Subject	Recipient
LOCKDOWN	Campus lockdown in effect	All active pass holders

5.10 Reporting Module

5.10.1 Features

Report	Filters	Export
Daily Visitor Report	Date, gate, pass type	CSV, PDF
Weekly Summary	Date range, department	CSV, PDF
Overstay Report	Date range	CSV
Security Incidents	Date range, severity	PDF
Audit Log Export	Date range, user, action	CSV
Hostel Guest Report	Block, date range	CSV
Vehicle Entry Report	Date range, vehicle type	CSV
Event Attendance	Event ID	CSV, PDF
Active Passes Dashboard	Live	UI only

5.10.2 Business Rules

- **BR-REPORT-01:** Reports older than 90 days are archived to cold storage; can be retrieved on request.
- **BR-REPORT-02:** All report exports are logged in the audit trail.
- **BR-REPORT-03:** PII in reports is masked for roles below Security Administrator (only last 4 digits of ID shown).

Section 6 — End-to-End Workflows

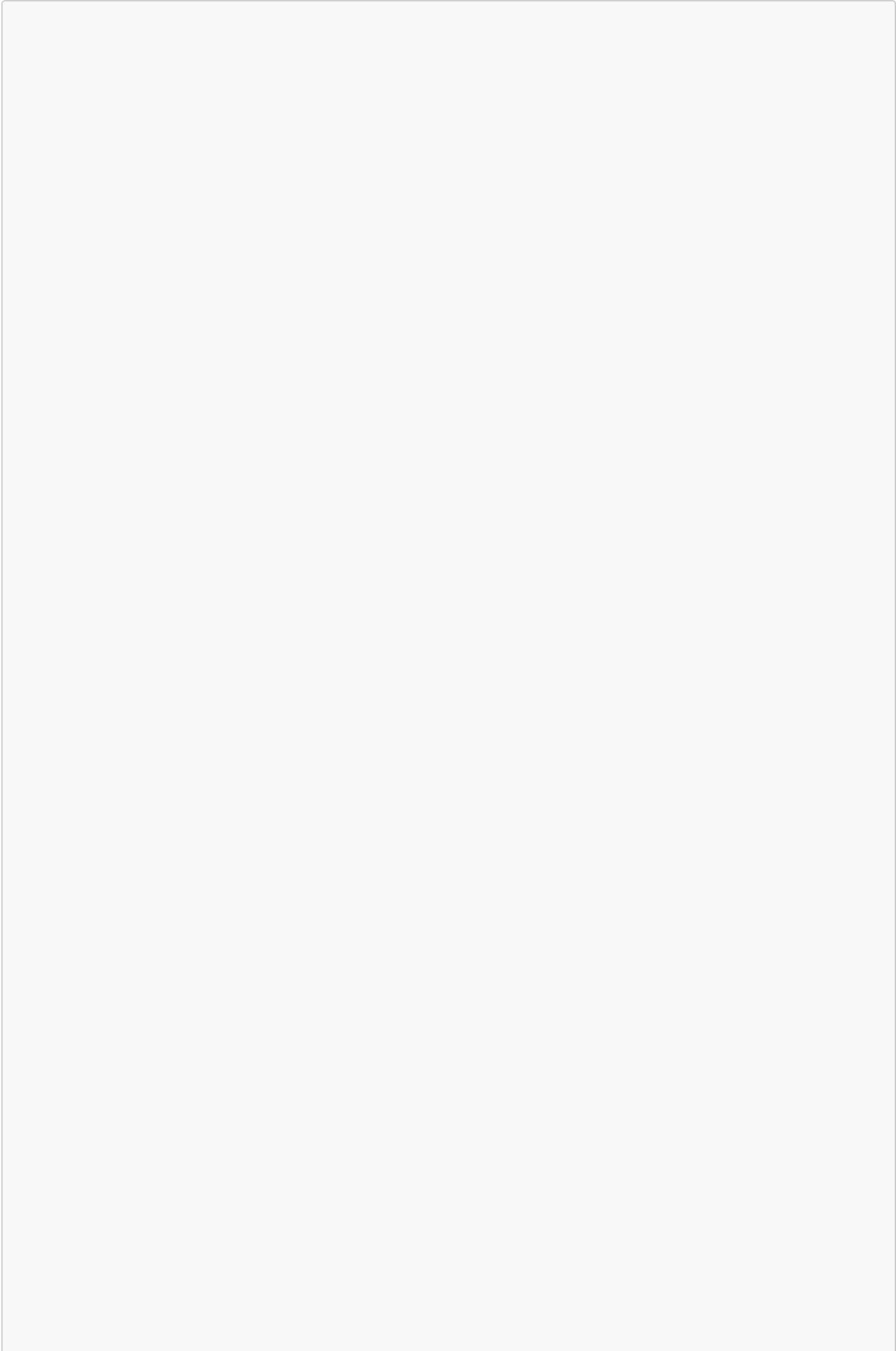
Workflow 1: Visitor Registration

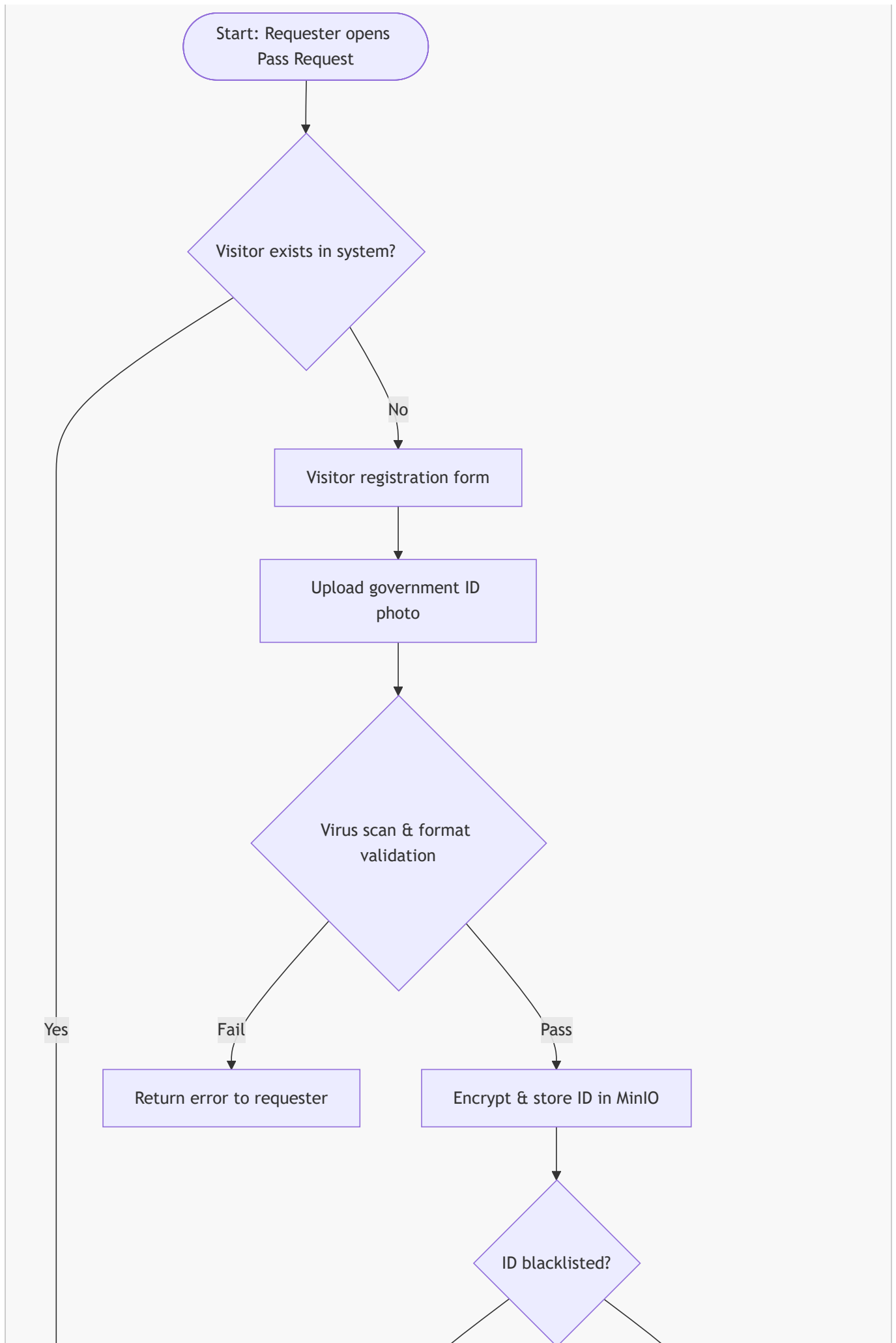
Trigger: A new visitor arrives or a requester creates a pass for someone not in the system.

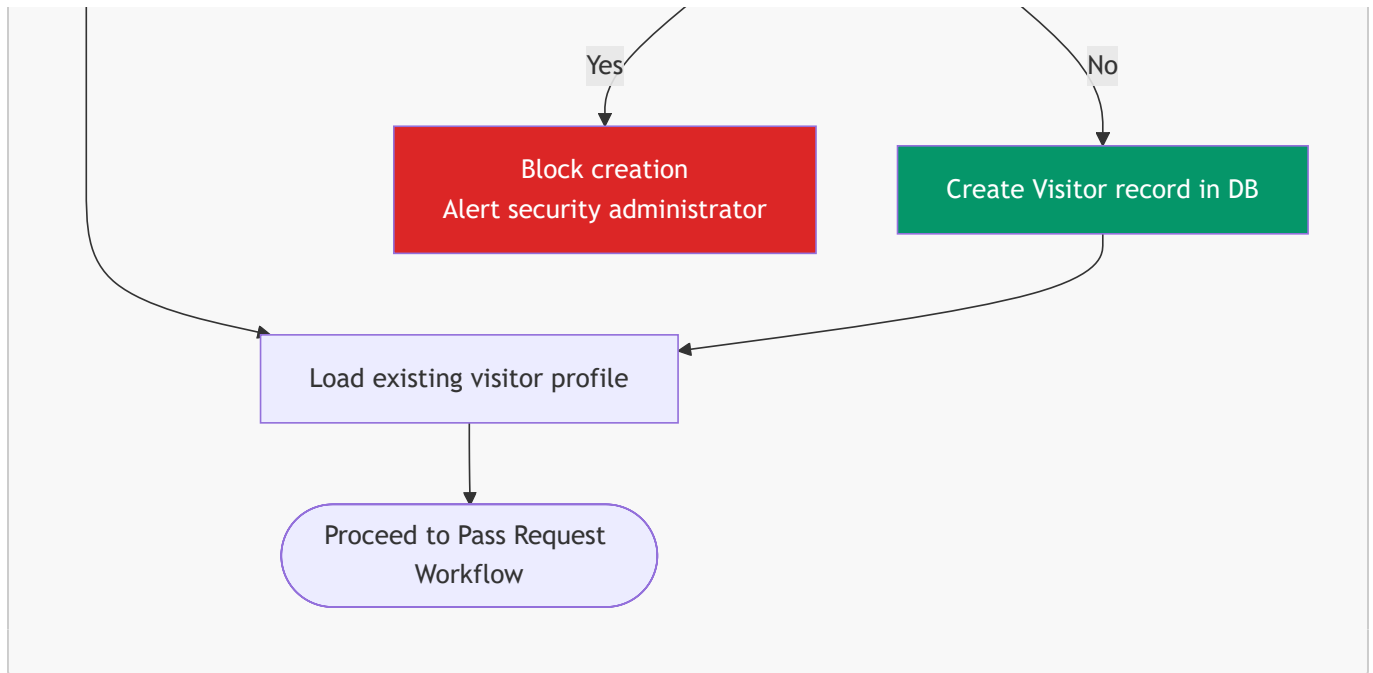
Inputs: Name, phone, email, ID type, ID number, ID photo.

Outputs: Visitor record created; visitor ID assigned.

Exception Handling: Duplicate ID → link to existing record; blacklisted ID → block and notify security.







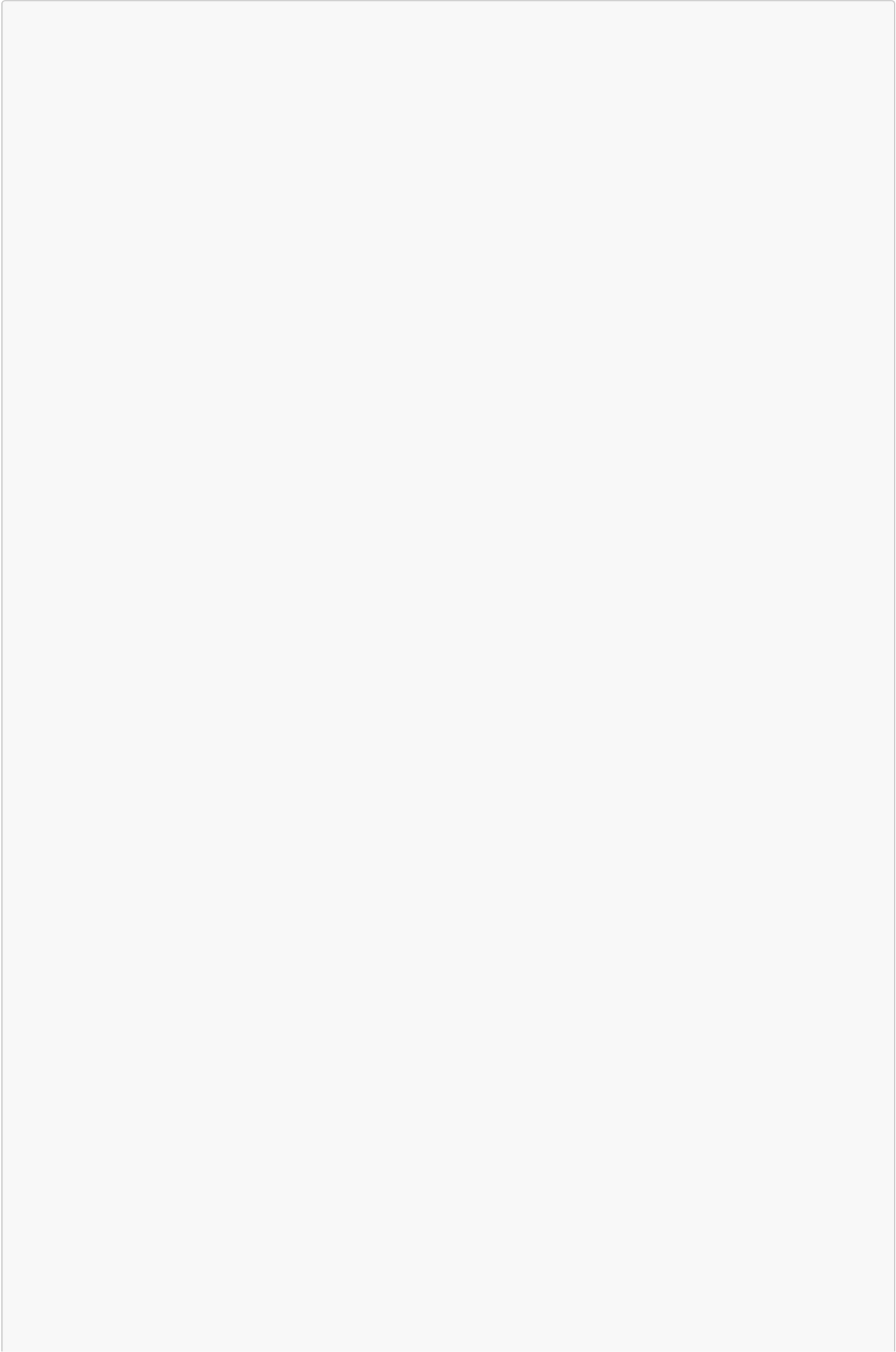
Workflow 2: Student Visitor Pass Workflow

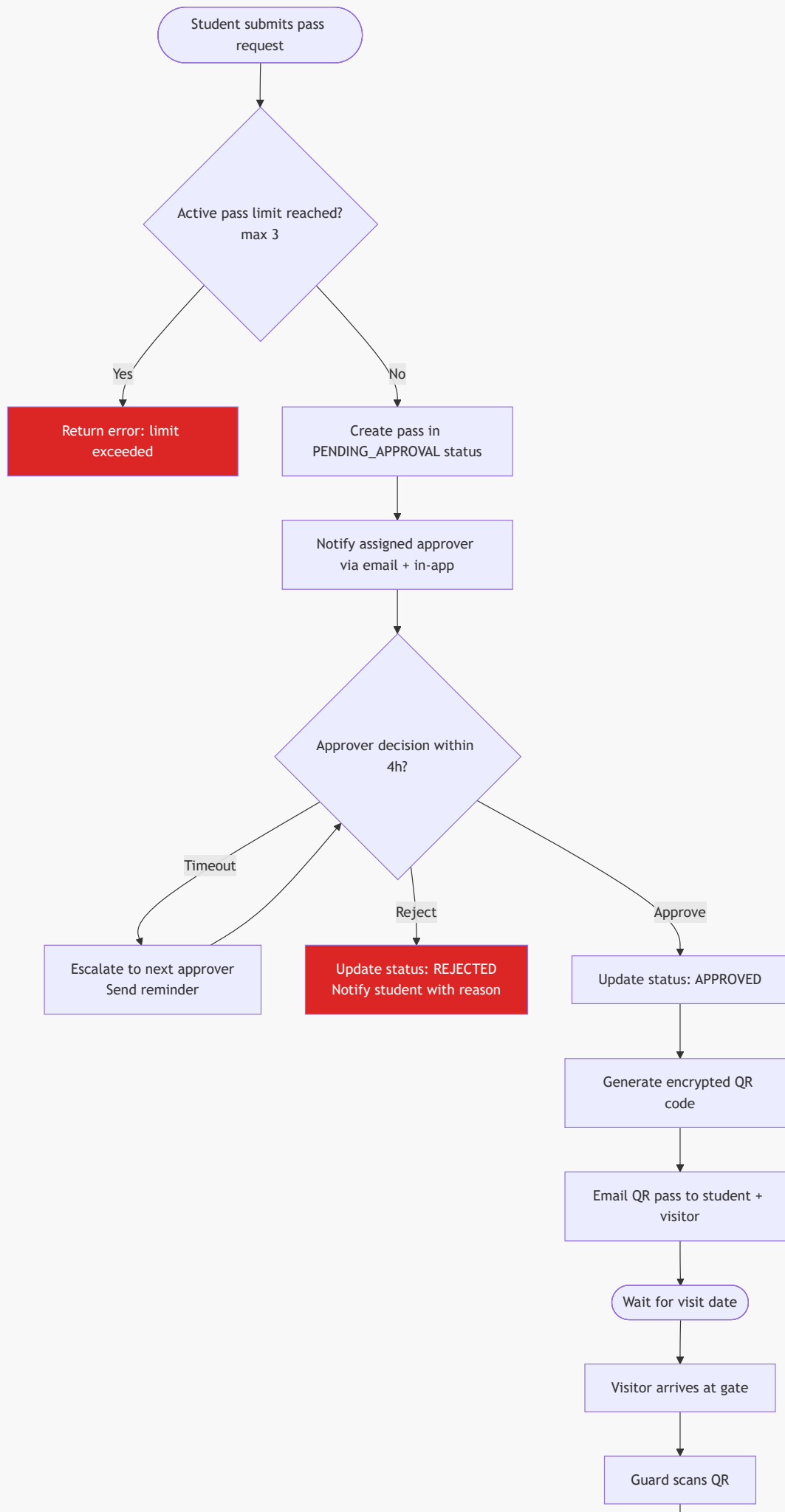
Trigger: Student logs in and clicks "Request Visitor Pass".

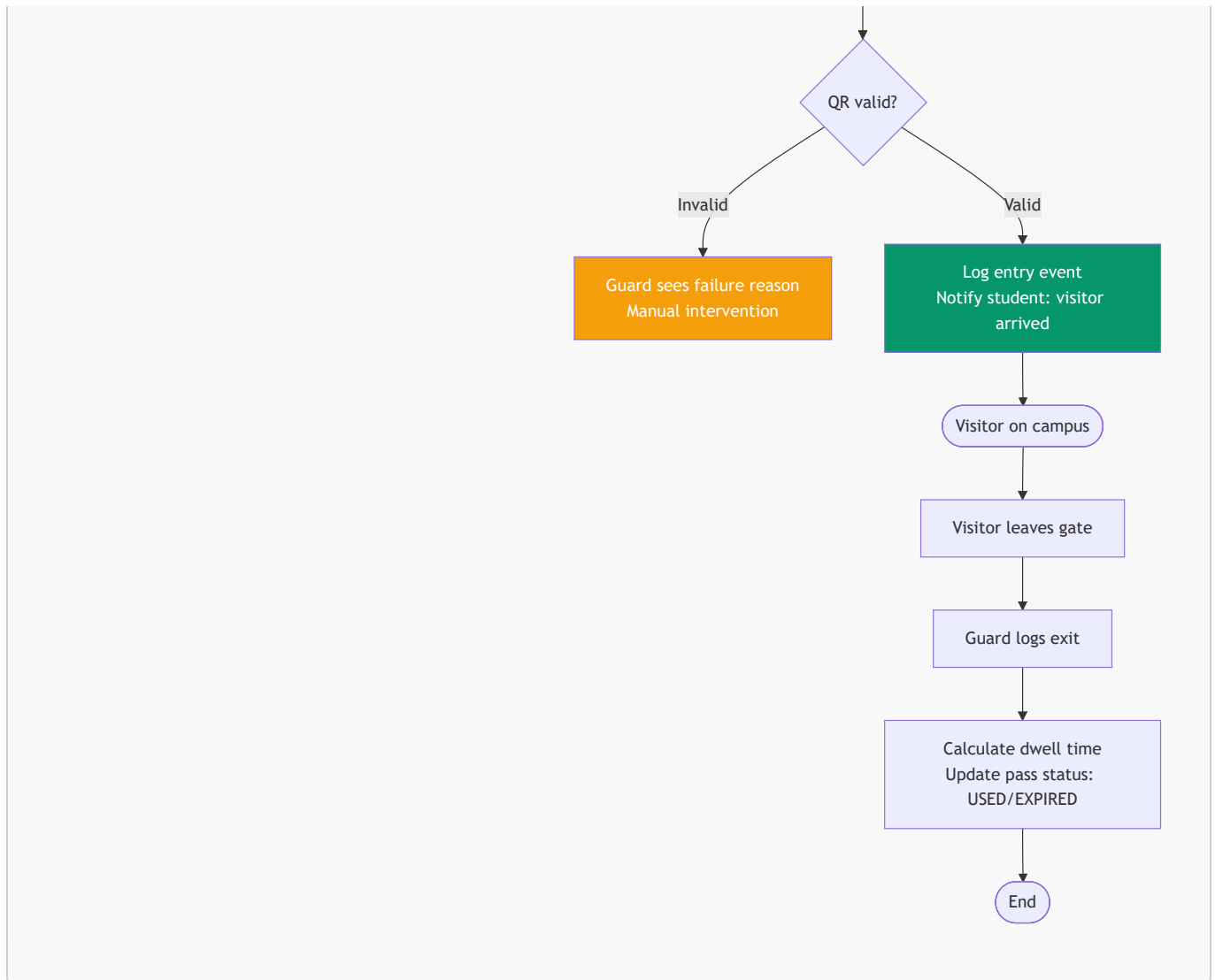
Inputs: Visitor details, visit date, purpose, gate preference.

Outputs: Approved pass + QR emailed to visitor; entry/exit logged.

Exception Handling: Supervisor unavailable → escalate to department head; approval timeout after 4h → notify.





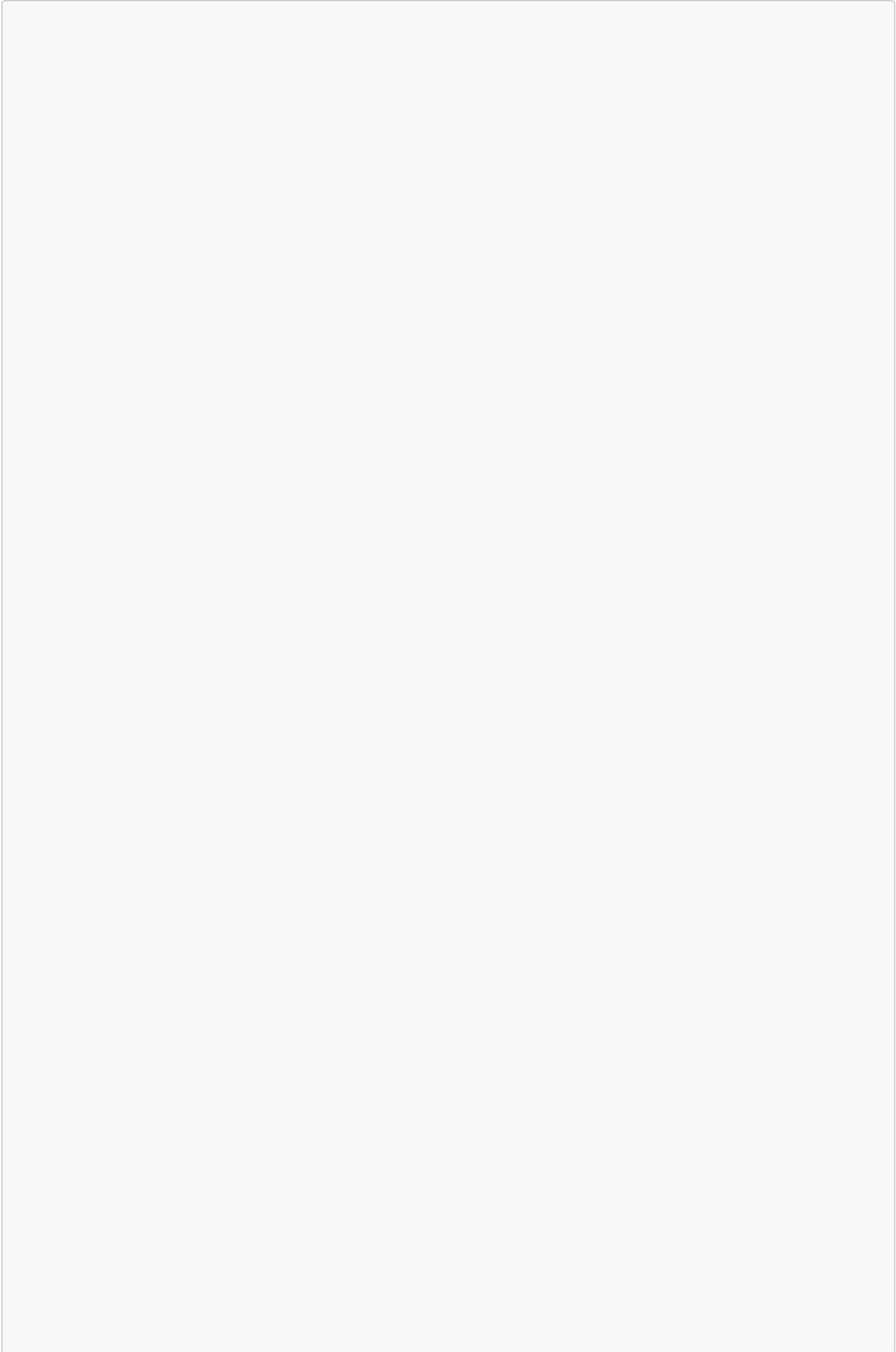


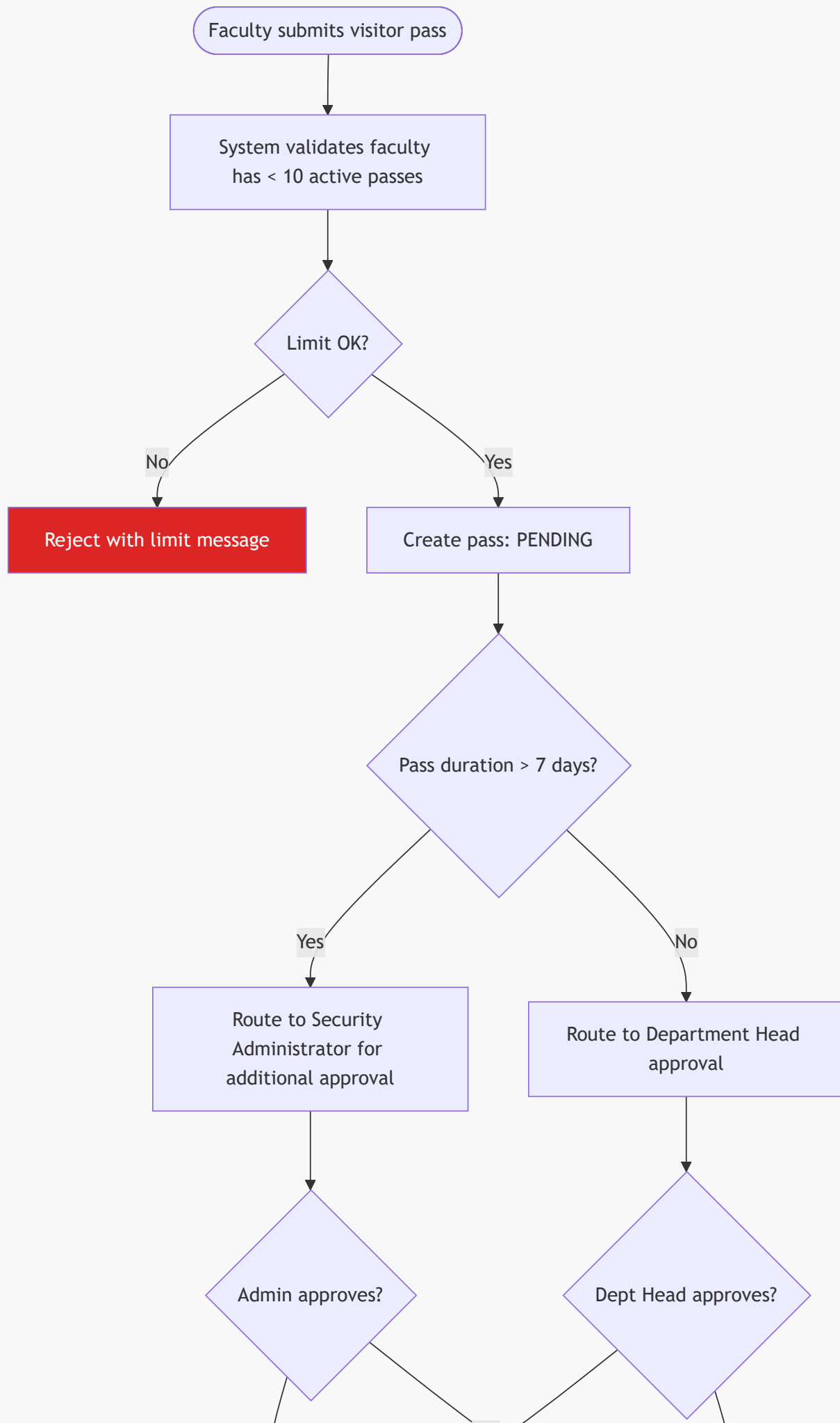
Workflow 3: Faculty Visitor Pass Workflow

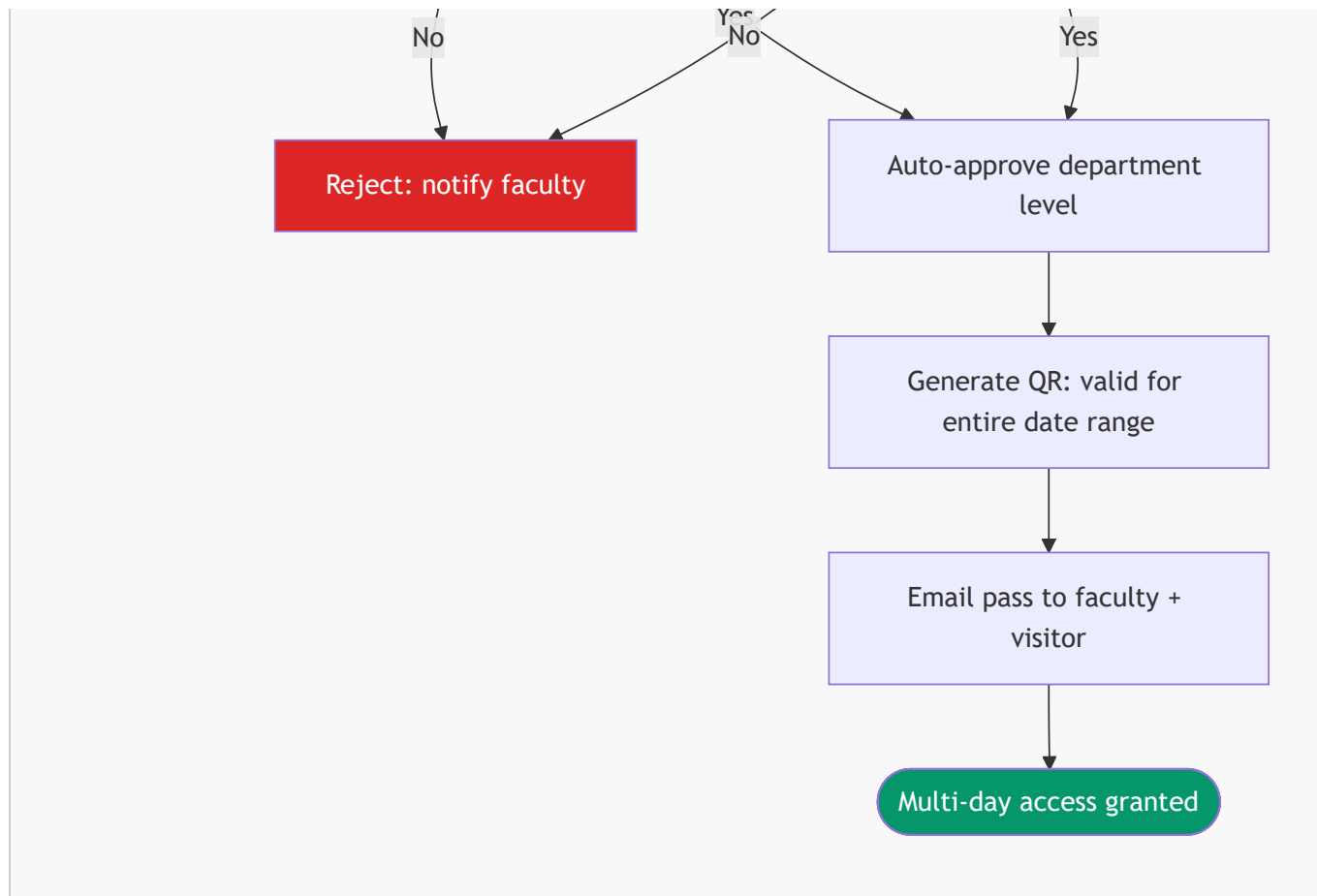
Trigger: Faculty member needs to invite an external researcher or collaborator.

Inputs: Visitor details, department, visit purpose, duration (up to 30 days), office location.

Outputs: Multi-day pass with date-range QR; departmental visit log updated.







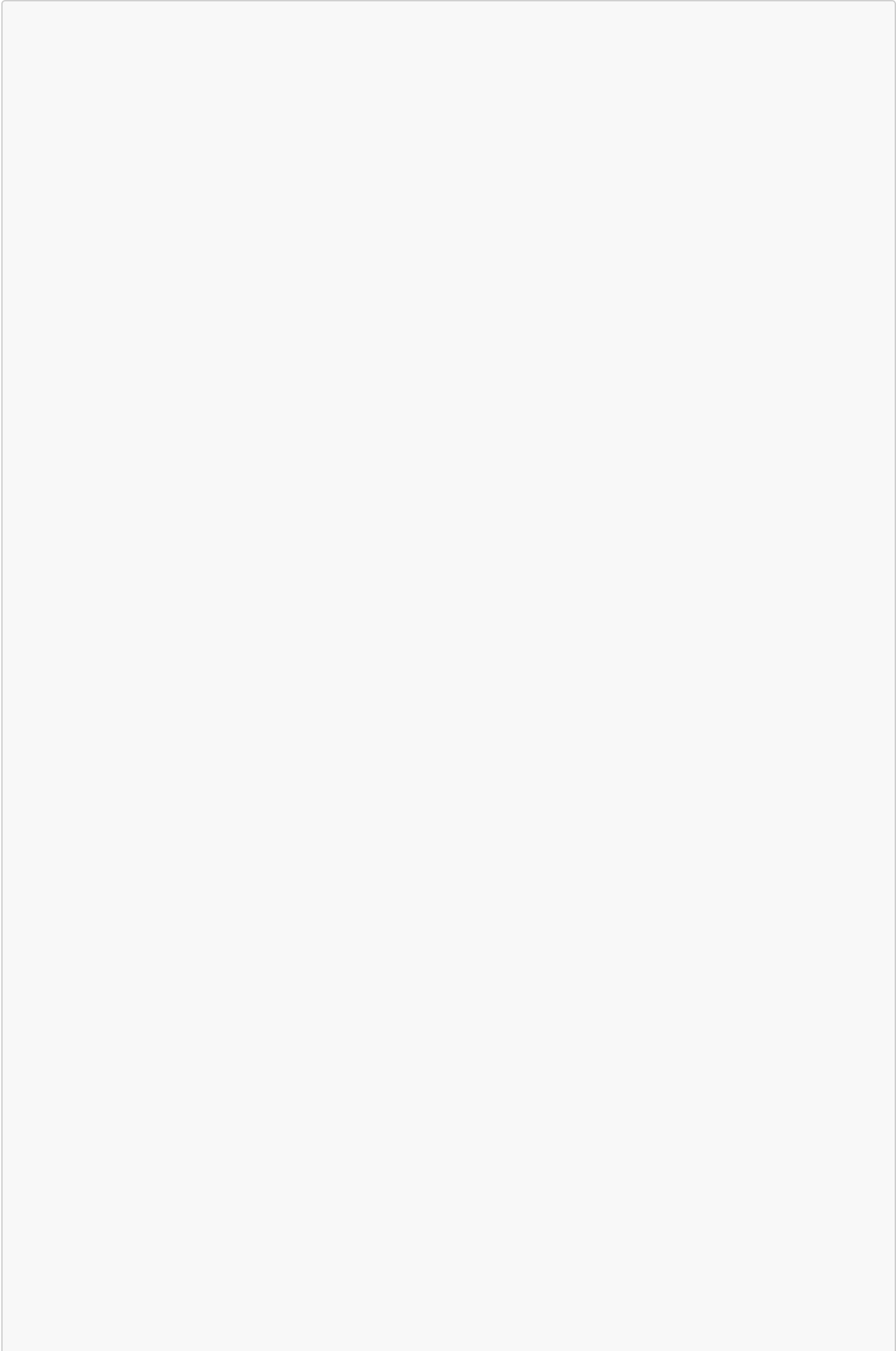
Workflow 4: Hostel Guest Pass Workflow

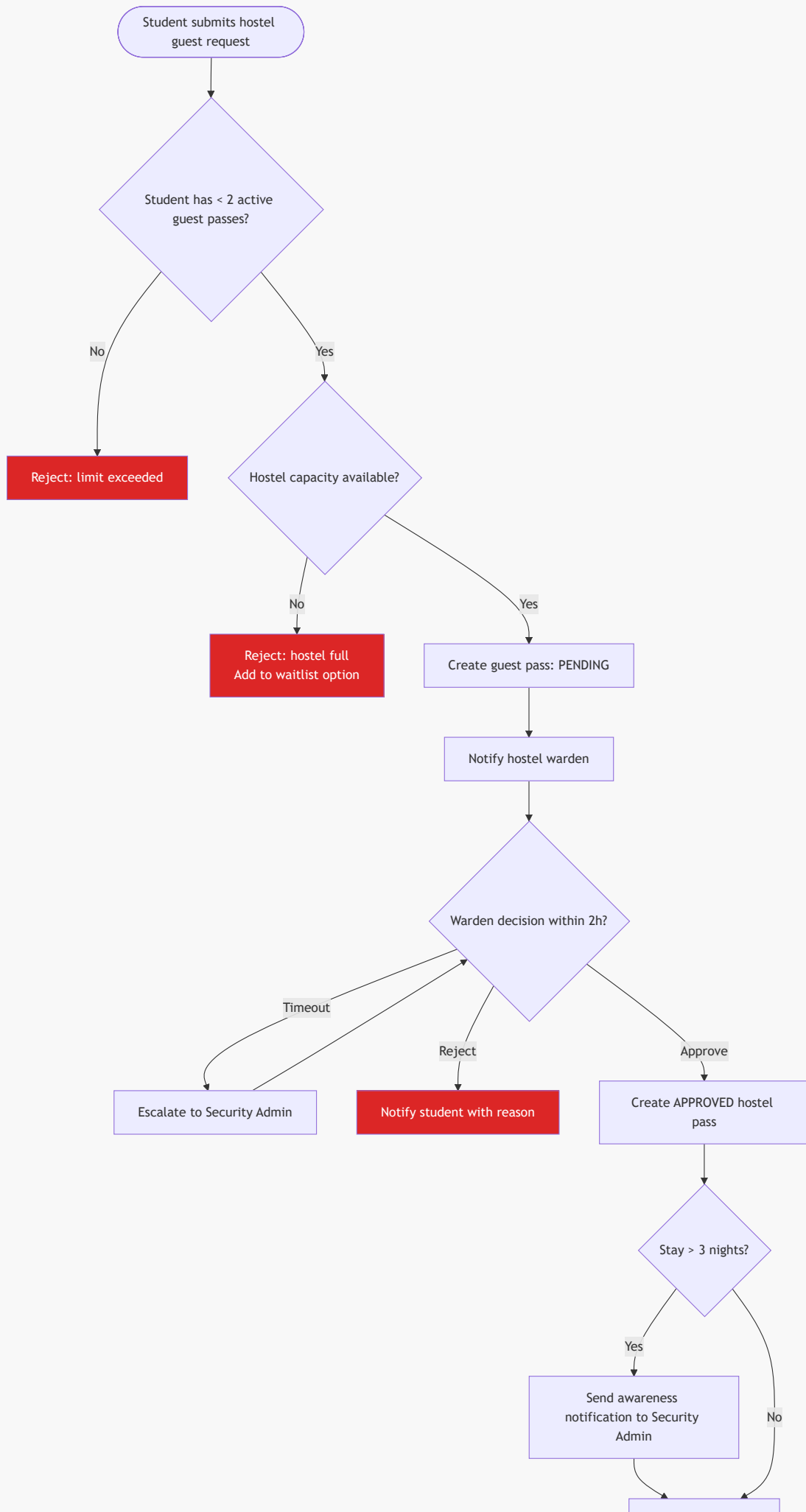
Trigger: Student requests overnight guest in hostel.

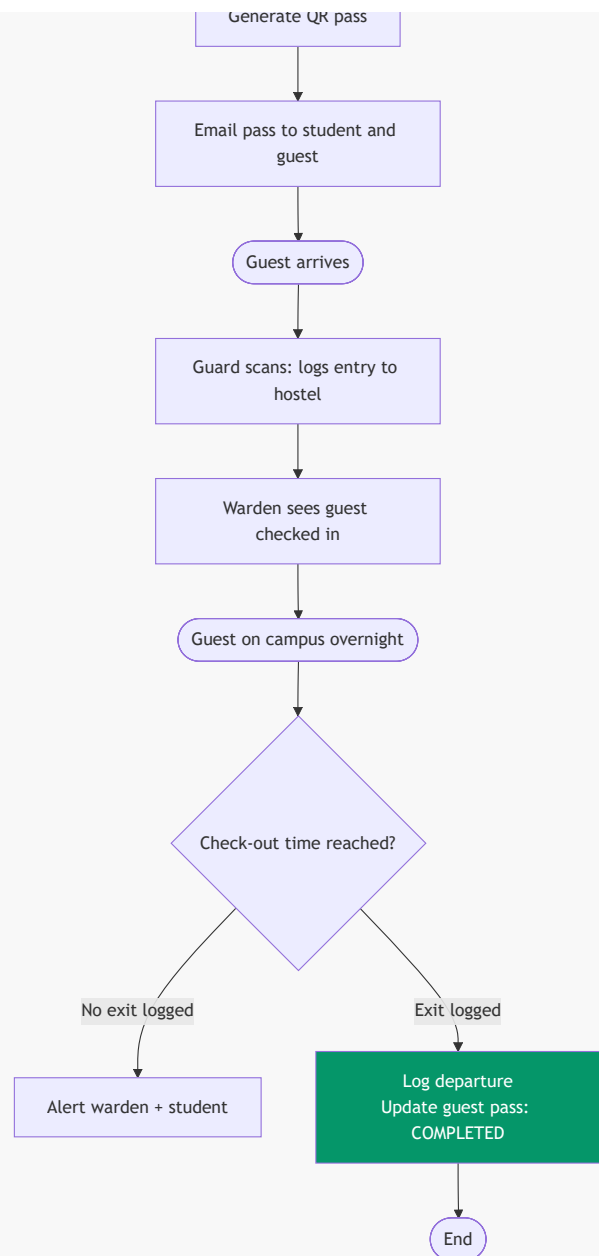
Inputs: Guest details, room number, check-in/out dates, relationship to guest.

Outputs: Approved hostel guest pass; warden notified of check-in/out.

Exception Handling: Capacity full → reject; curfew violation → reject unless warden override.





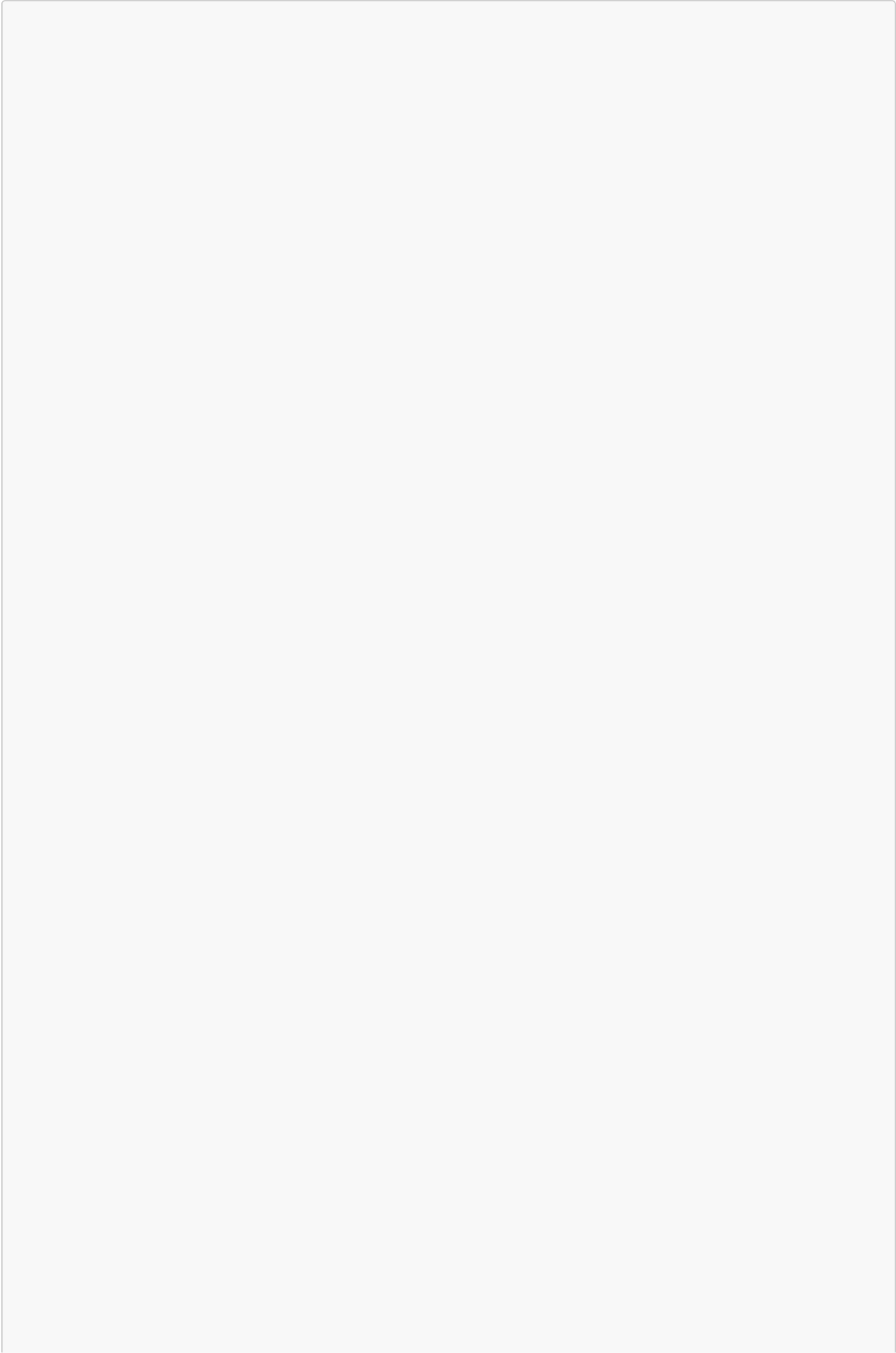


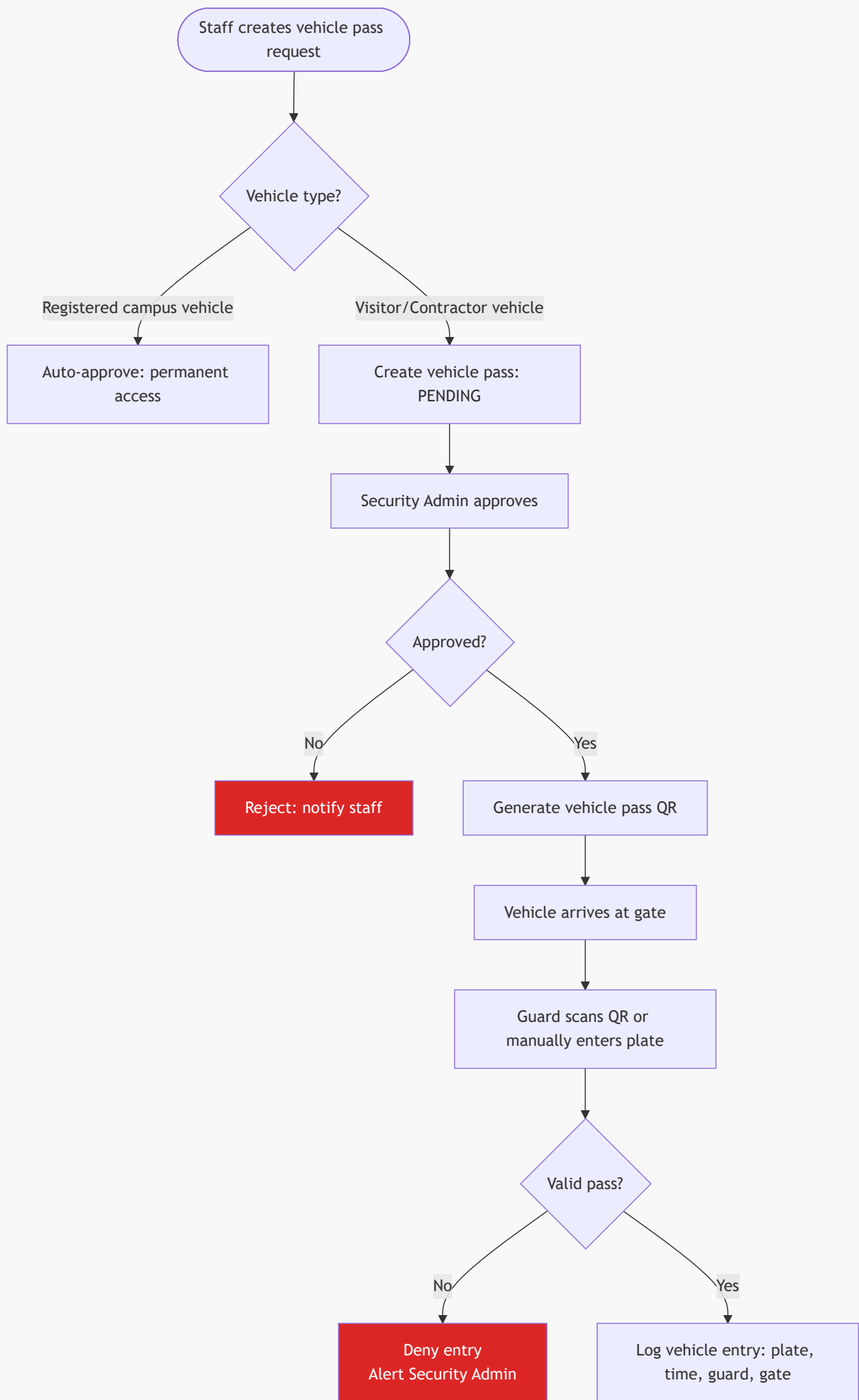
Workflow 5: Vehicle Entry Workflow

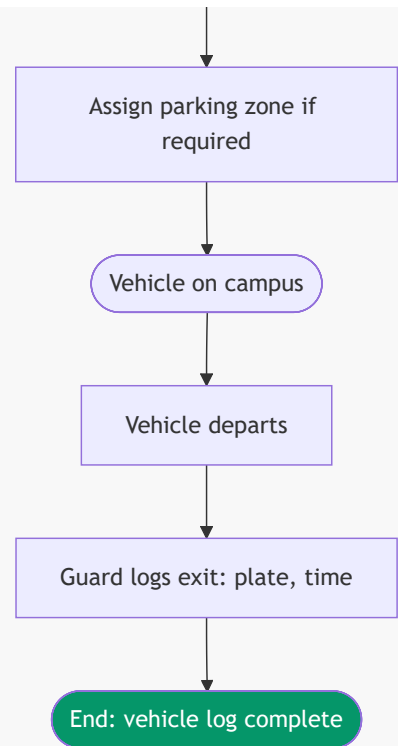
Trigger: Staff creates a vehicle entry pass for a delivery or contractor vehicle.

Inputs: Vehicle number, type, driver name, purpose, expected arrival window.

Outputs: Vehicle pass + QR; entry/exit logged.





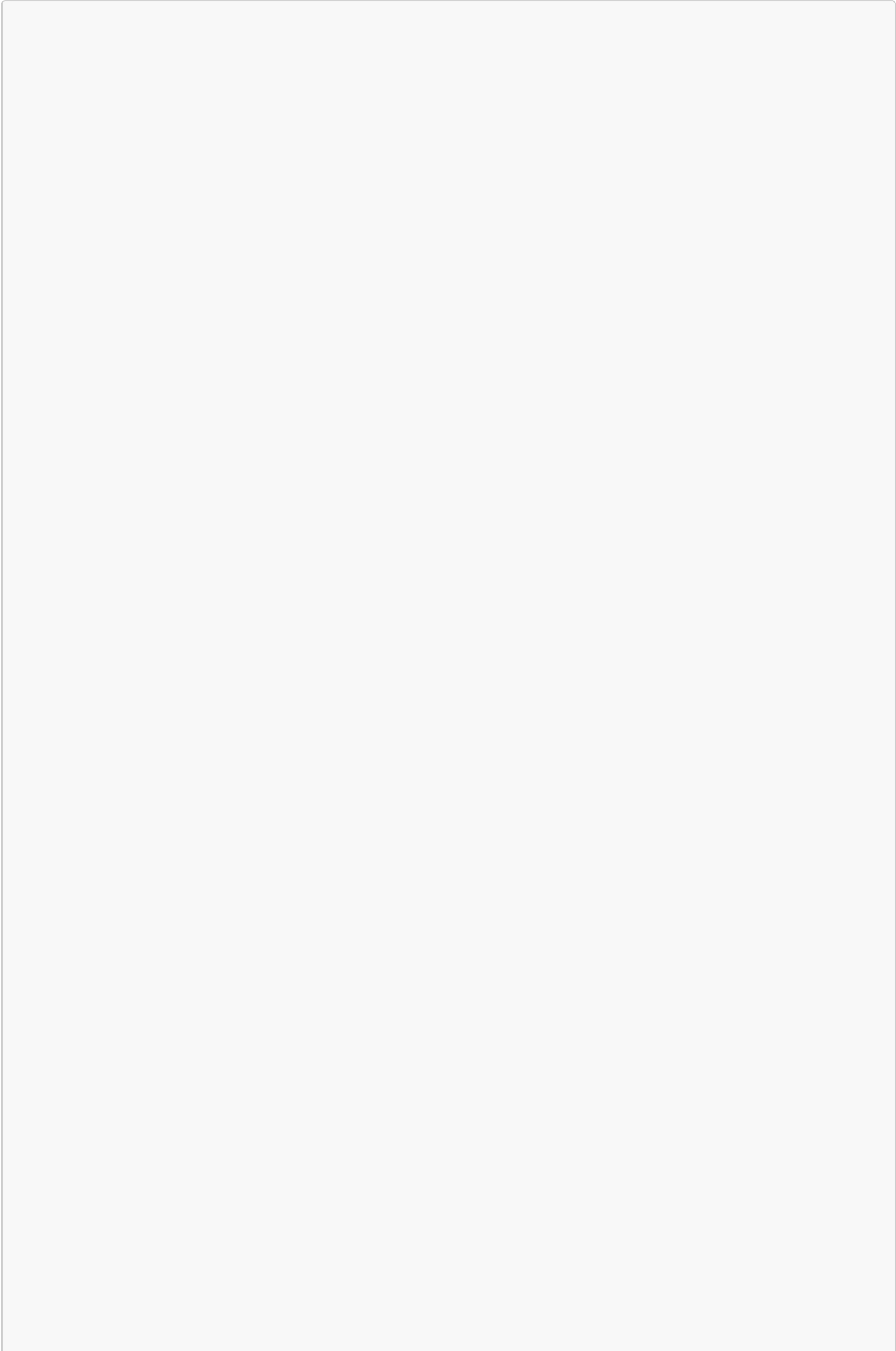


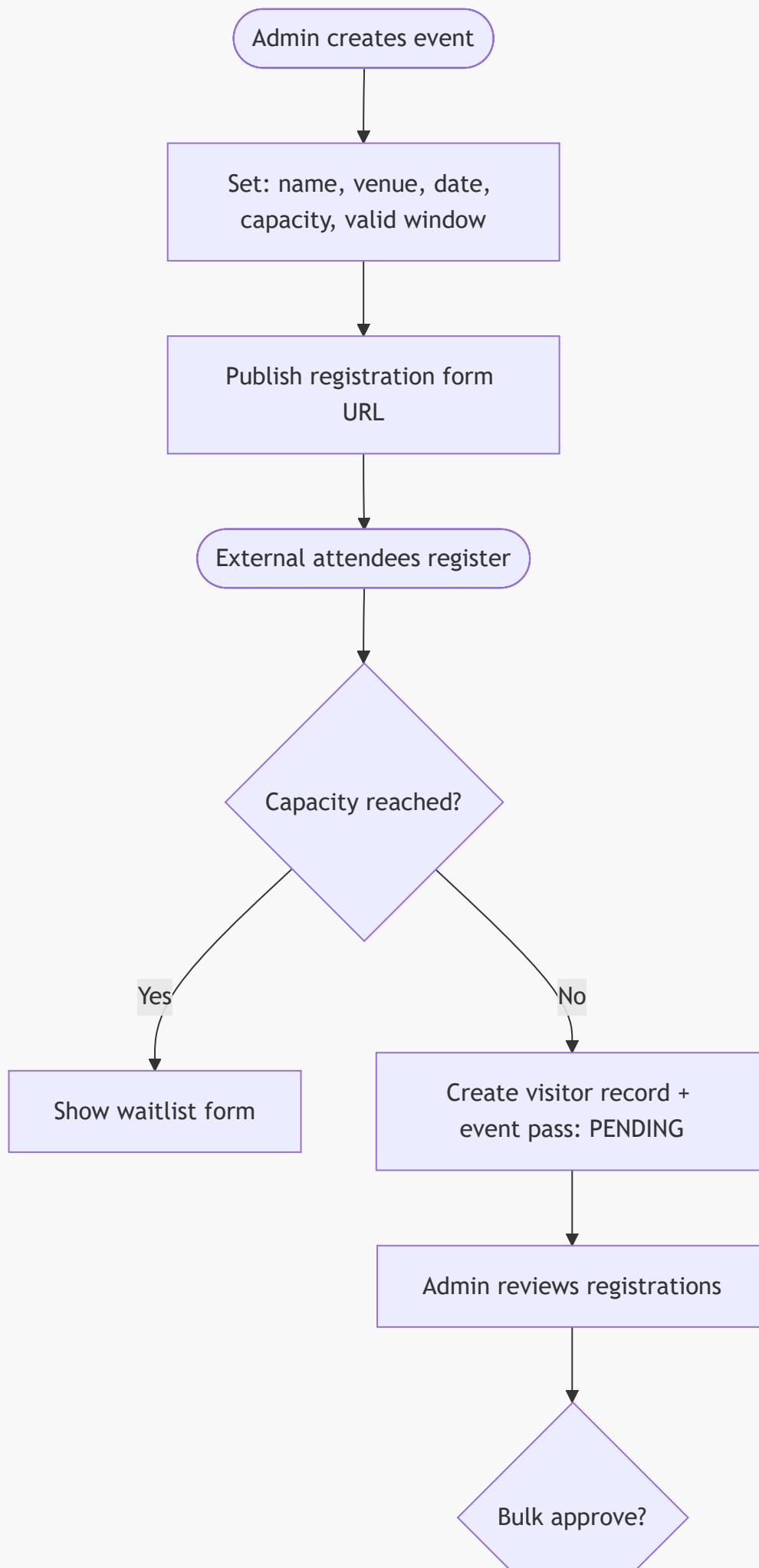
Workflow 6: Event Visitor Workflow

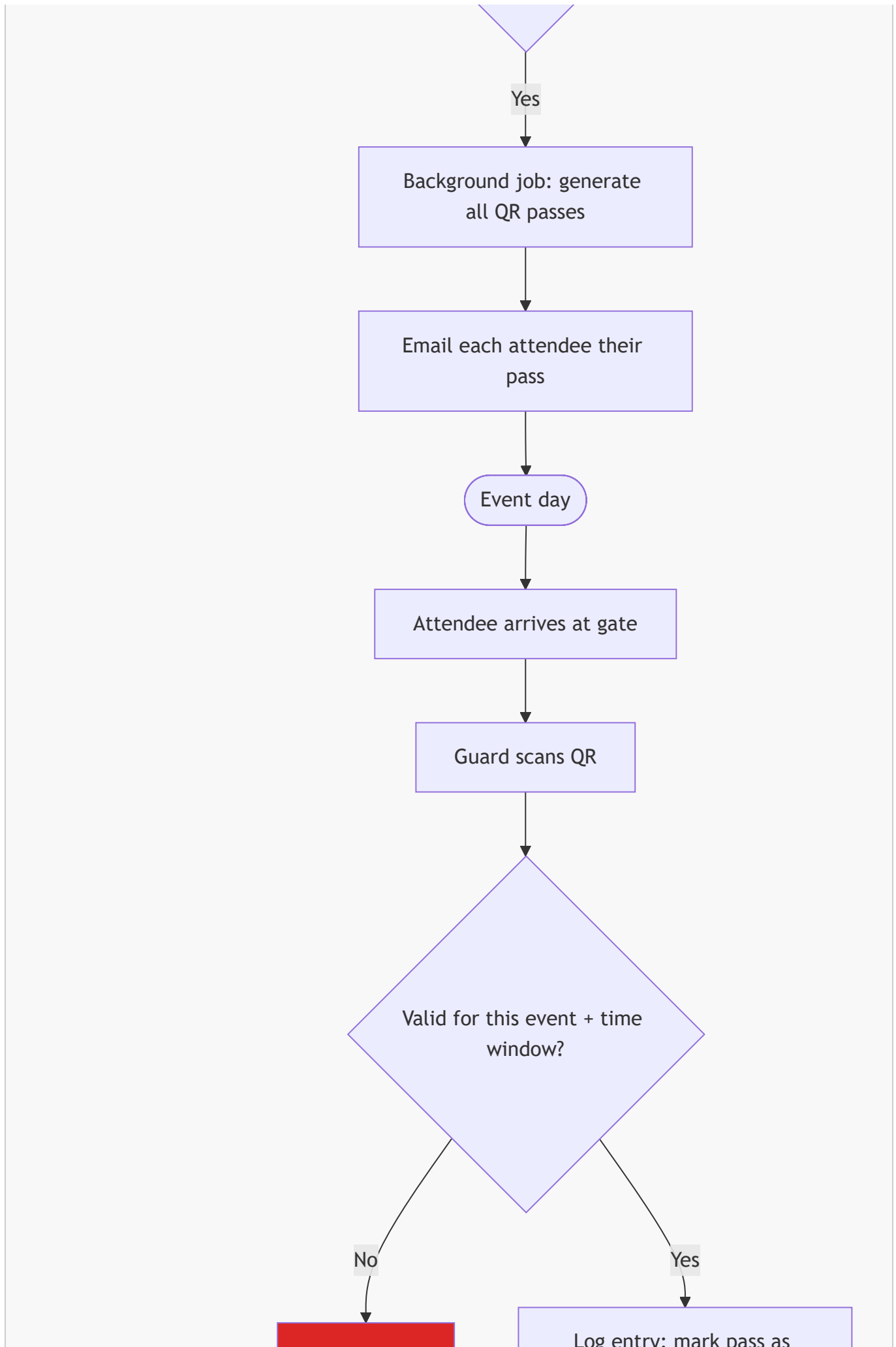
Trigger: Admin creates an event; attendees register via public form.

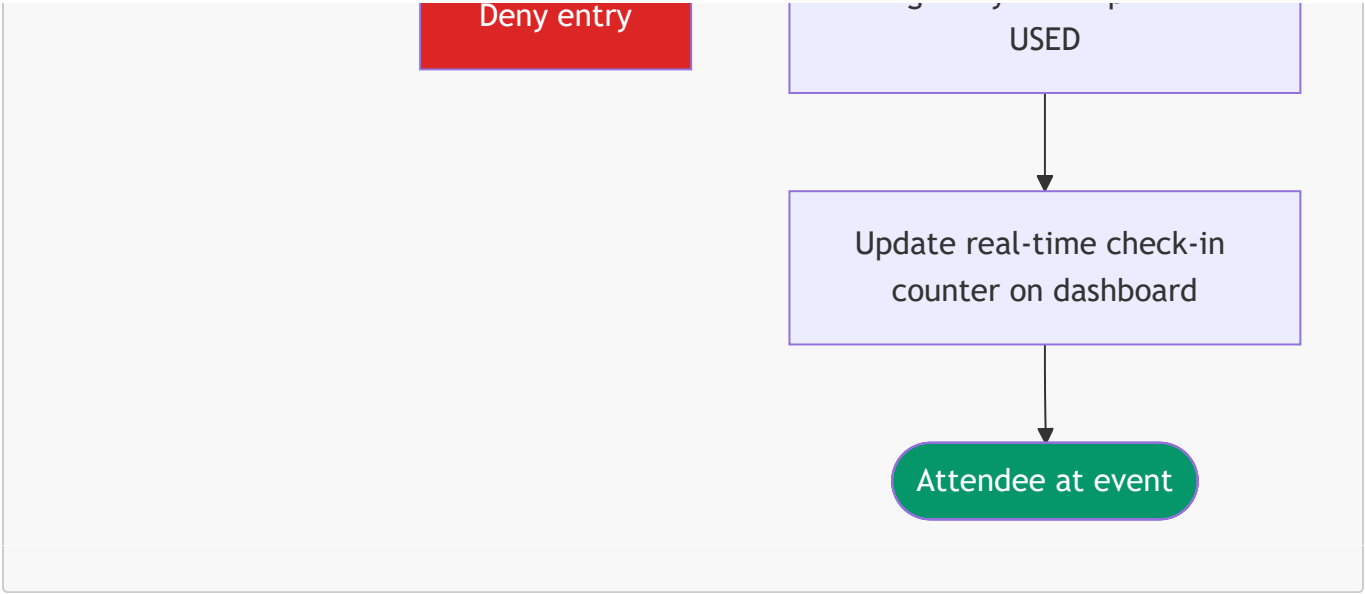
Inputs: Event details, attendee name/email/phone.

Outputs: Bulk QR passes emailed to all registered attendees.



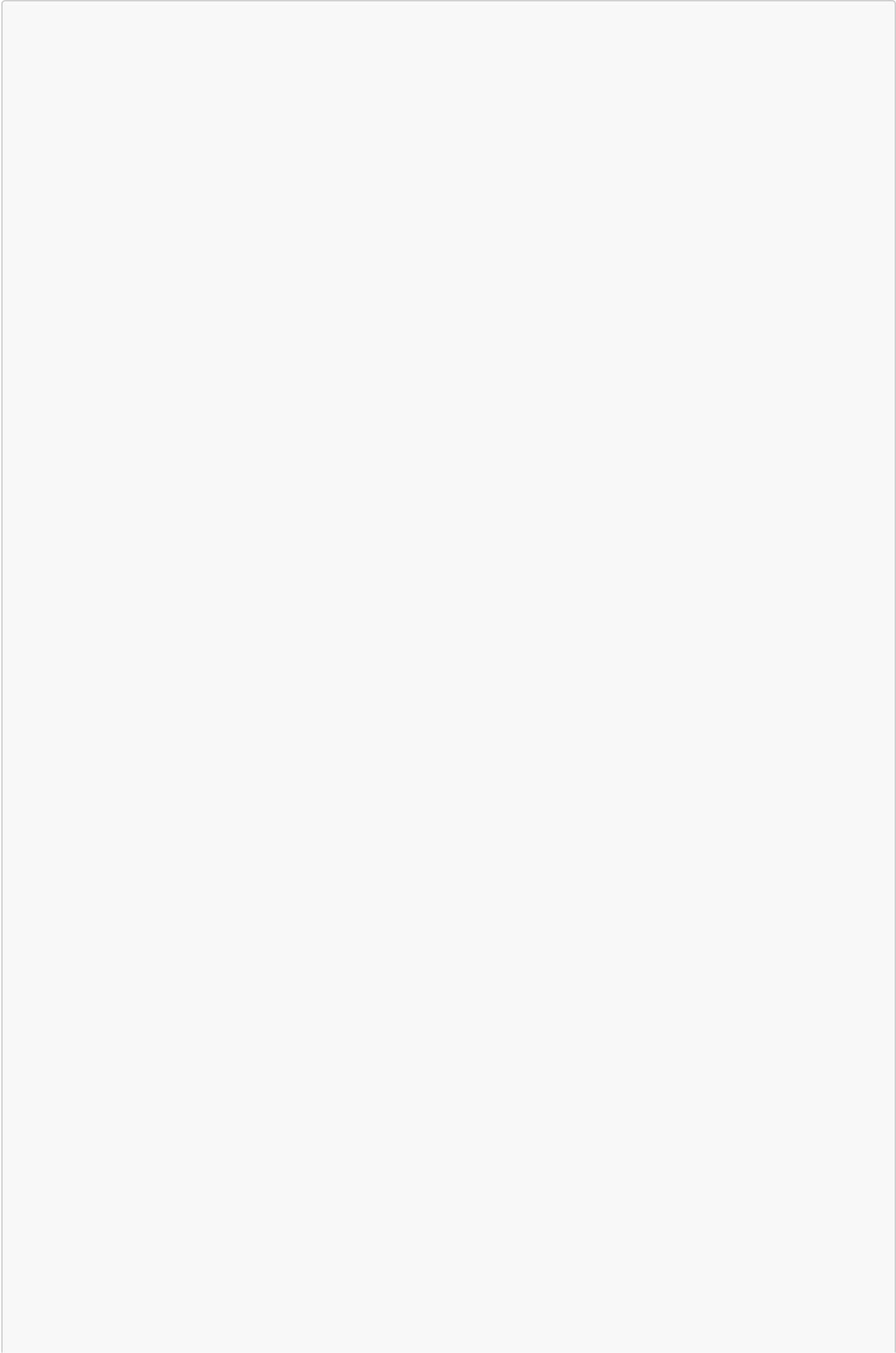


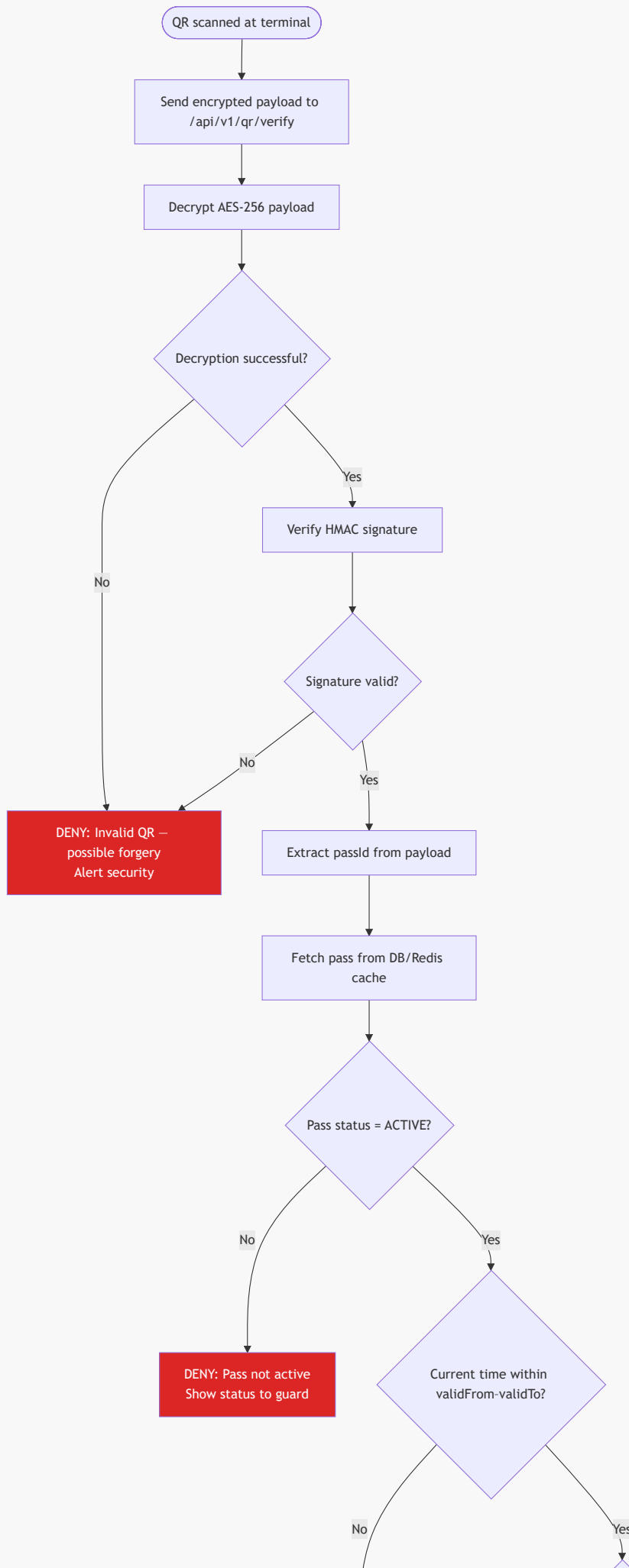


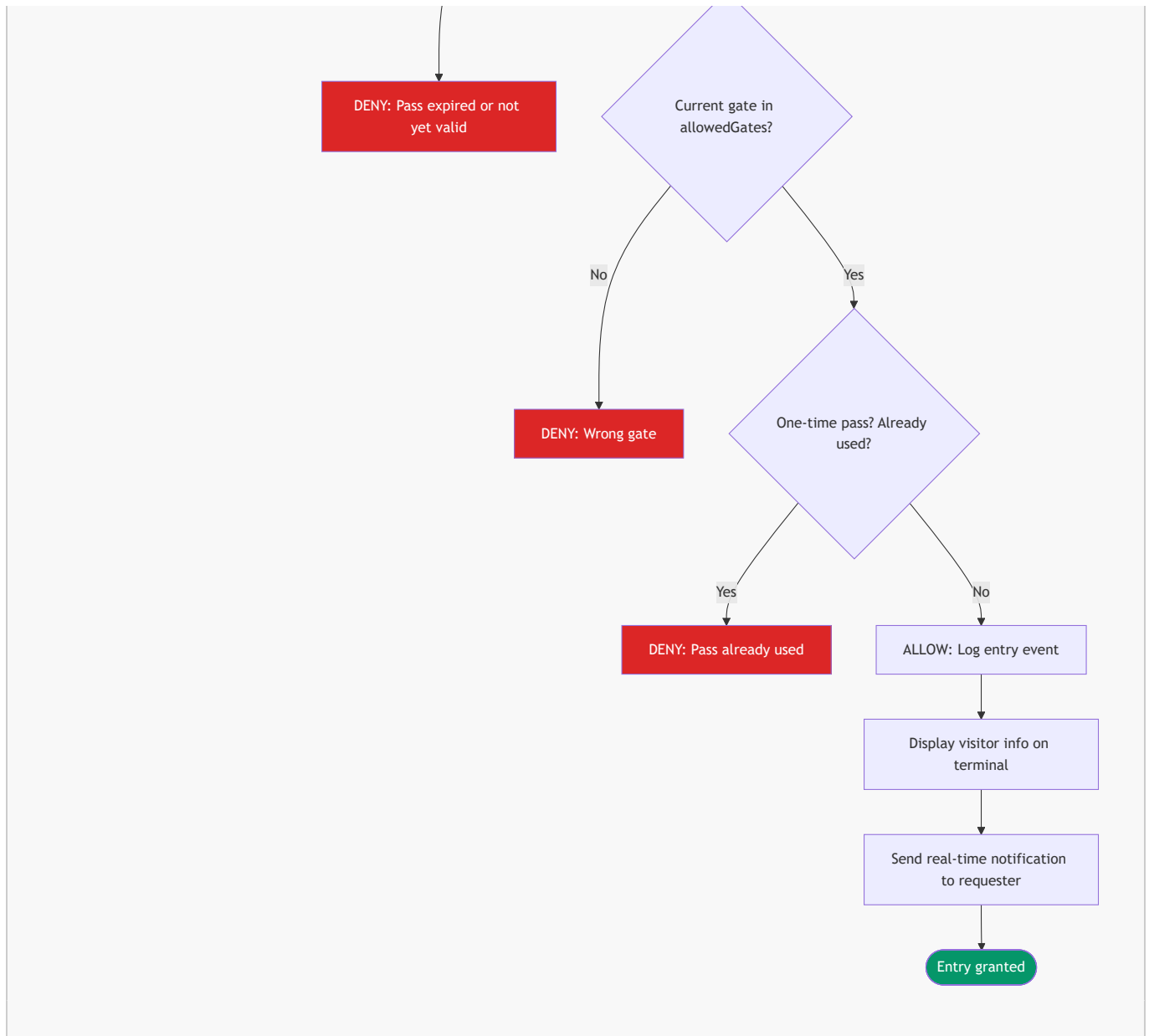


Workflow 7: QR Verification Workflow

- Trigger:** Guard scans QR code at gate terminal.
- Inputs:** Raw QR data from scan.
- Outputs:** Allow/Deny decision with reason displayed on terminal.

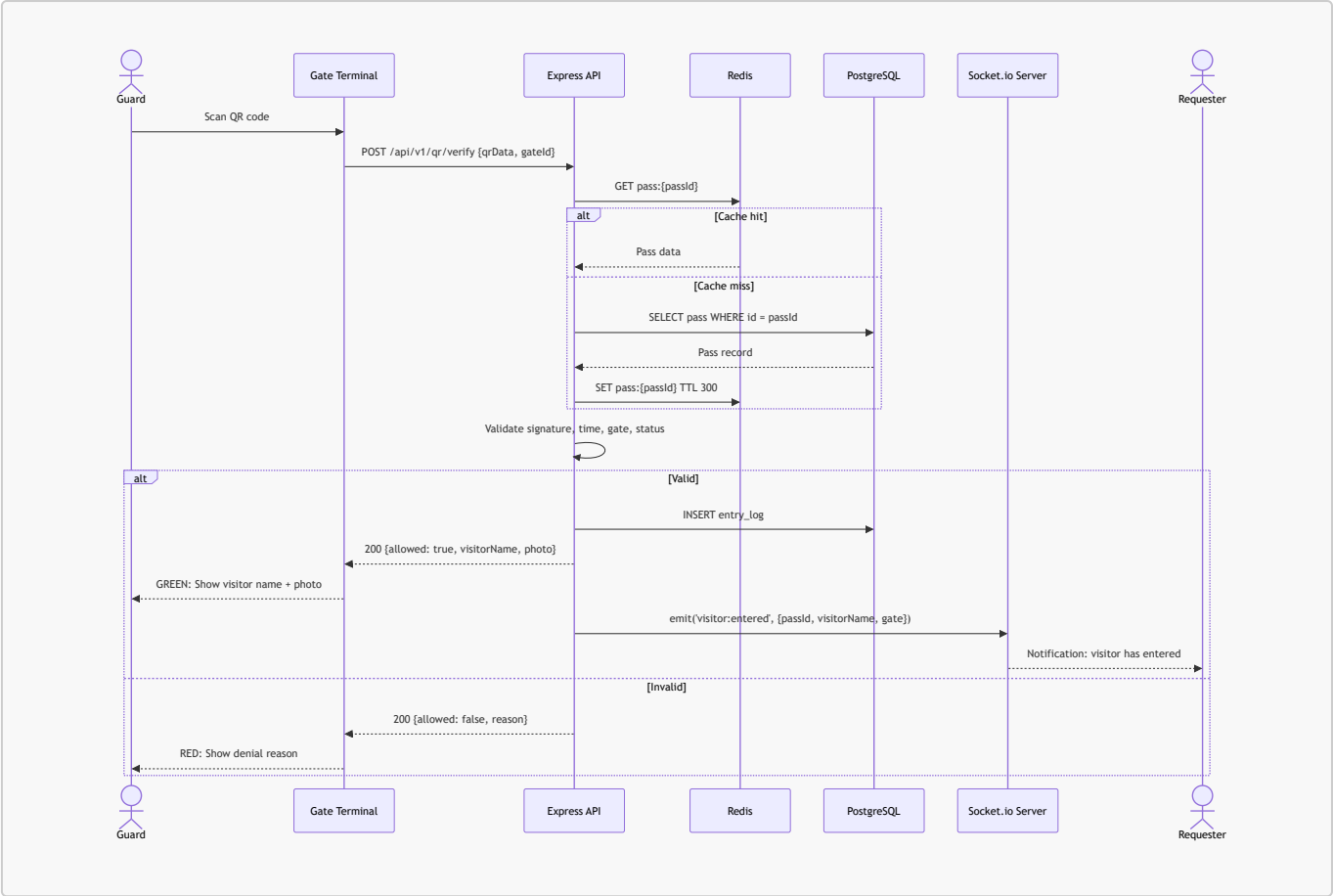






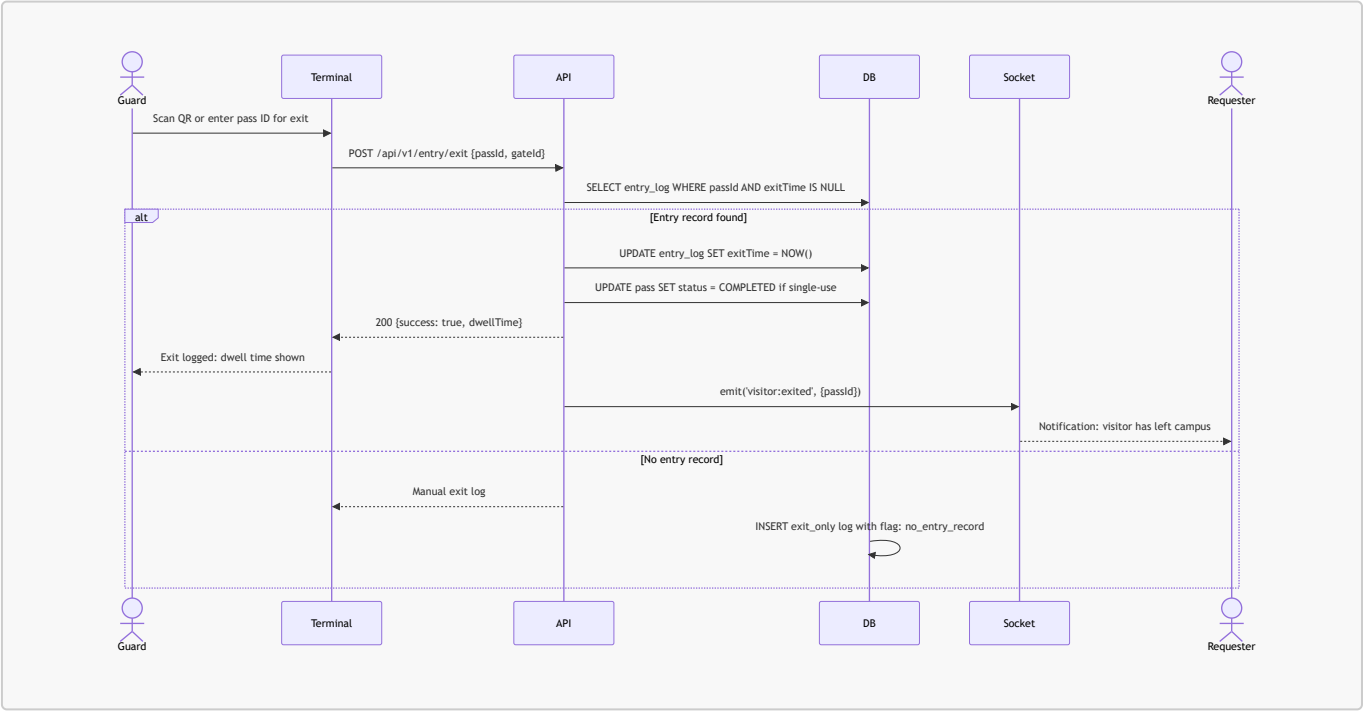
Workflow 8: Entry Workflow (Detailed)

Trigger: Visitor presents QR at gate.



Workflow 9: Exit Workflow

Trigger: Visitor exits campus; guard logs exit.



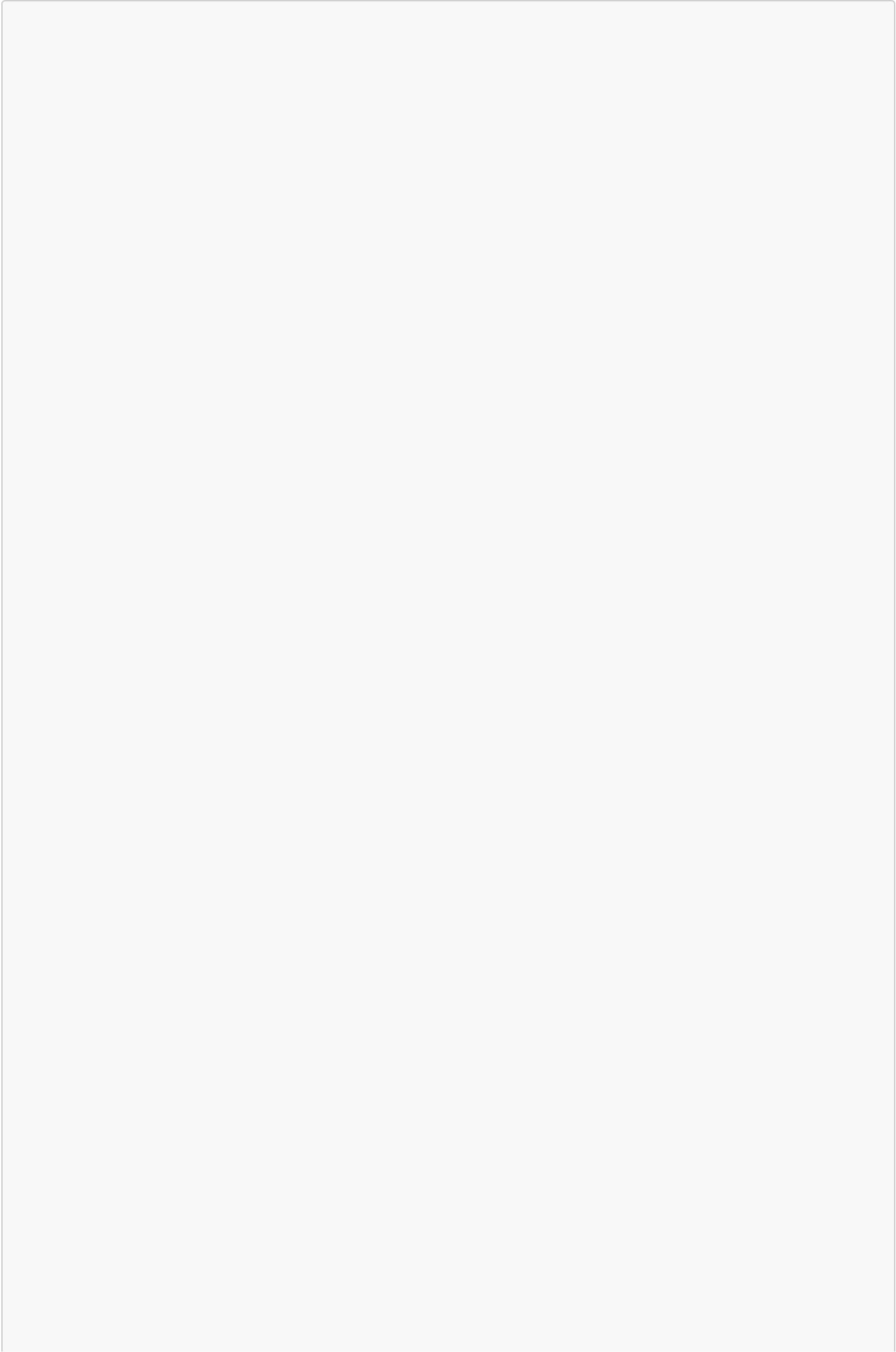
Workflow 10: Emergency Lockdown Workflow

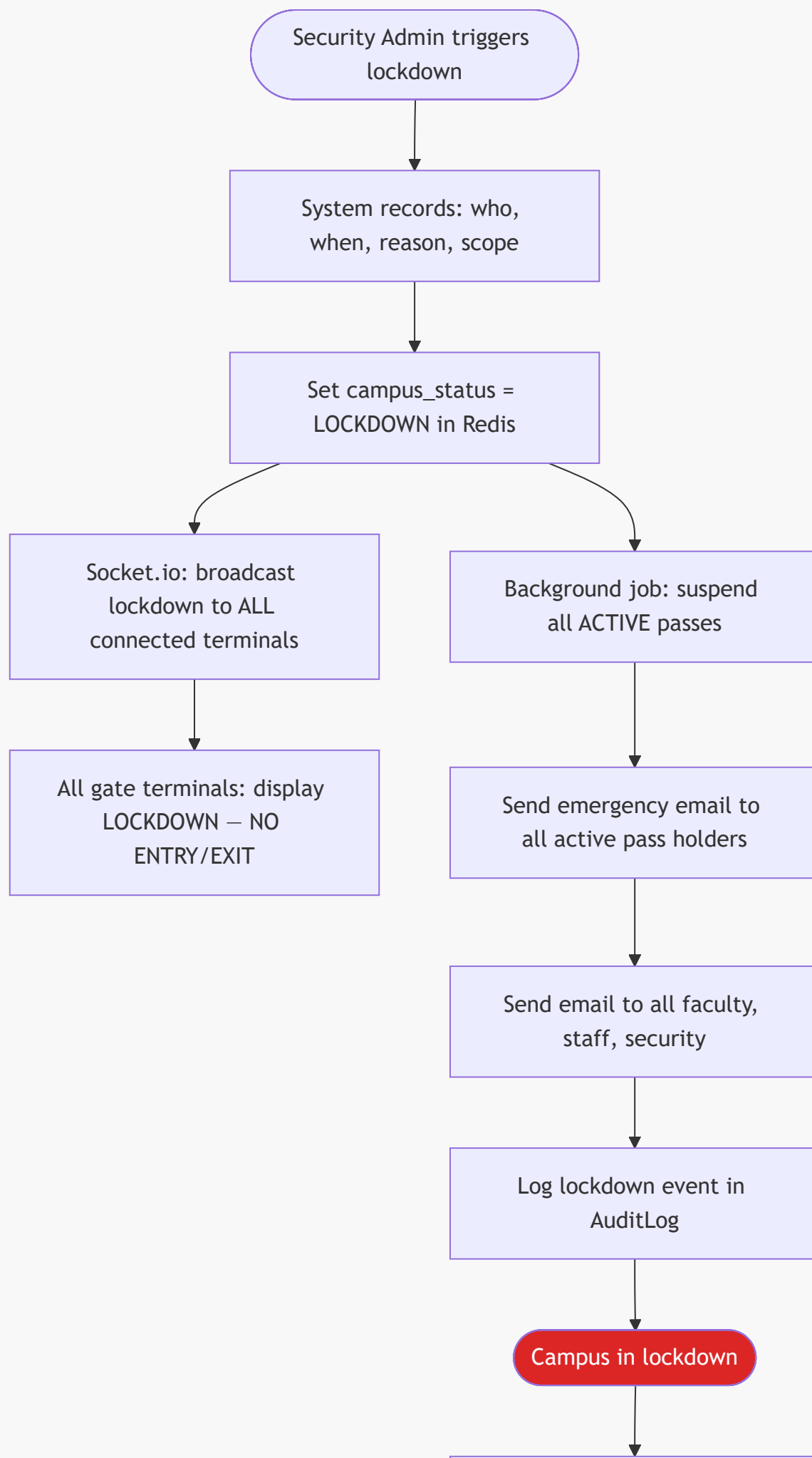
Trigger: Security Administrator or authorized guard initiates lockdown.

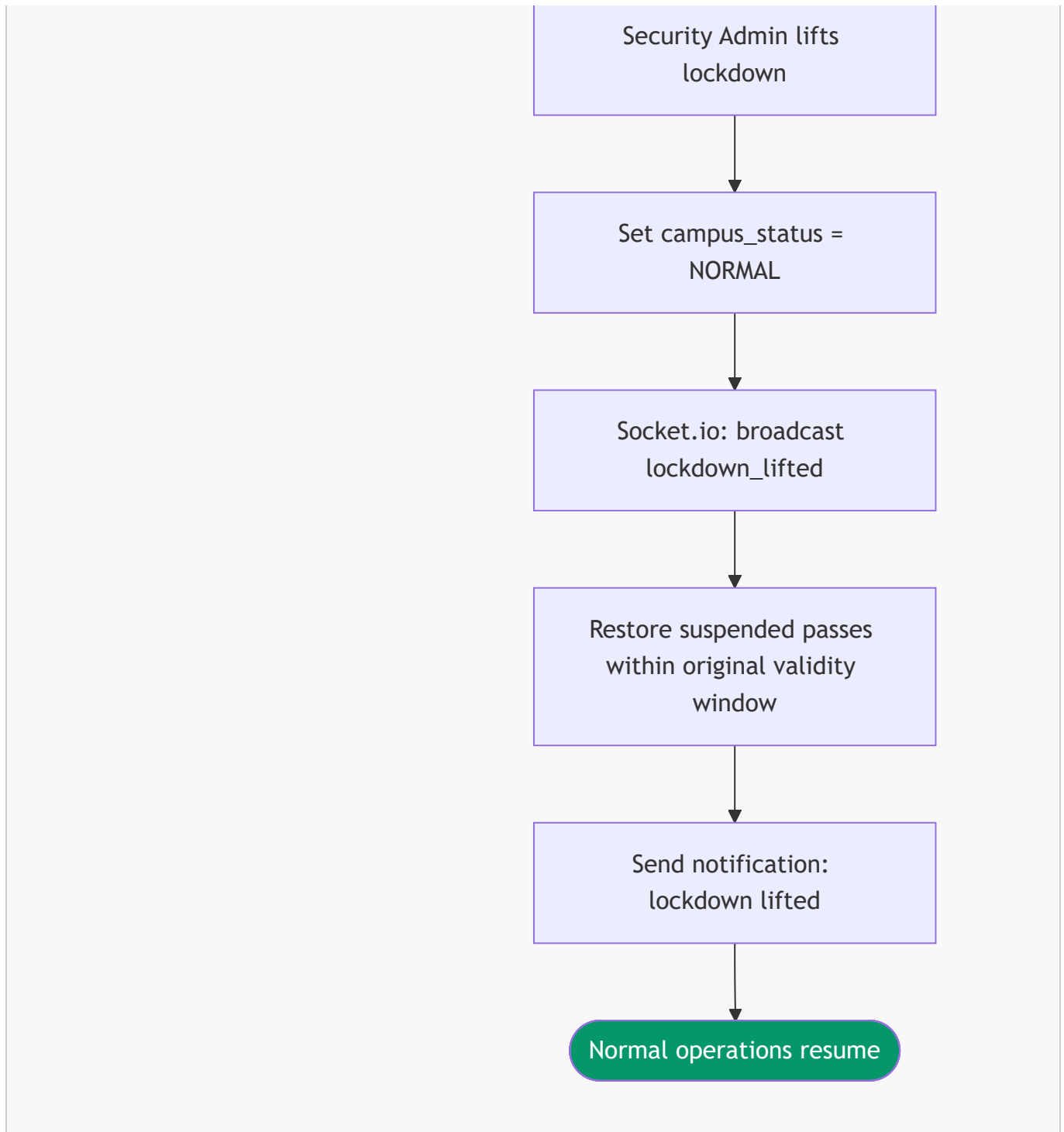
Inputs: Lockdown reason, affected gates (or all campus).

Outputs: All active passes suspended; all gates notified; all active sessions alerted.

Exception Handling: System failure during lockdown → gates fall back to manual physical lock.

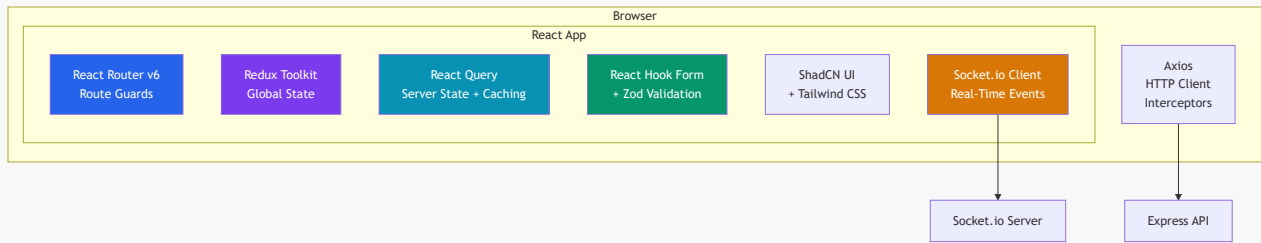






Section 7 — React Frontend Architecture

7.1 Architecture Overview



7.2 Folder Structure

```

src/
├── app/
│   ├── App.tsx                # Root component, providers setup
│   ├── providers.tsx          # QueryClient, Redux, Theme, Socket providers
│   └── store.ts               # Redux store configuration
├── pages/
│   ├── auth/
│   │   ├── LoginPage.tsx
│   │   ├── RegisterPage.tsx
│   │   └── ResetPasswordPage.tsx
│   ├── dashboard/
│   │   ├── StudentDashboard.tsx
│   │   ├── FacultyDashboard.tsx
│   │   ├── SecurityDashboard.tsx
│   │   └── AdminDashboard.tsx
│   ├── passes/
│   │   ├── PassListPage.tsx
│   │   ├── PassDetailPage.tsx
│   │   ├── CreatePassPage.tsx
│   │   └── PassHistoryPage.tsx
│   ├── visitors/
│   │   ├── VisitorListPage.tsx
│   │   └── VisitorDetailPage.tsx
│   ├── hostel/
│   │   ├── GuestRequestPage.tsx
│   │   └── WardenDashboard.tsx
│   ├── vehicles/
│   │   └── VehiclePassPage.tsx
│   ├── events/
│   │   ├── EventListPage.tsx
│   │   └── EventDetailPage.tsx
│   ├── security/
│   │   ├── QRScanPage.tsx
│   │   ├── EntryLogPage.tsx
│   │   └── LockdownPage.tsx
│   ├── analytics/
│   │   └── AnalyticsDashboard.tsx
│   └── admin/
  
```

```
├── UserManagementPage.tsx
├── SystemConfigPage.tsx

components/
├── common/
│   ├── Button.tsx
│   ├── DataTable.tsx
│   ├── StatusBadge.tsx
│   ├── ConfirmDialog.tsx
│   └── EmptyState.tsx
├── passes/
│   ├── PassCard.tsx
│   ├── PassQRModal.tsx
│   ├── PassStatusTimeline.tsx
│   └── ApprovalActions.tsx
├── visitors/
│   ├── VisitorForm.tsx
│   └── IDUploader.tsx
├── security/
│   ├── QRScanner.tsx
│   ├── EntryLogRow.tsx
│   └── AlertBanner.tsx
├── analytics/
│   ├── VisitorChart.tsx
│   ├── PeakHourChart.tsx
│   └── StatsCard.tsx
└── notifications/
    ├── NotificationBell.tsx
    └── NotificationPanel.tsx

layouts/
├── AuthLayout.tsx           # Centered card for login/register
├── DashboardLayout.tsx     # Sidebar + topbar layout
└── SecurityLayout.tsx      # Full-screen gate terminal layout

hooks/
├── useAuth.ts              # Auth state + token refresh
├── useSocket.ts            # Socket.io connection management
├── usePermissions.ts       # RBAC permission checks
├── usePasses.ts           # Pass CRUD react-query hooks
├── useNotifications.ts    # Notification state
└── useDebounce.ts

services/
├── api.ts                  # Axios instance with interceptors
├── auth.service.ts
├── pass.service.ts
├── visitor.service.ts
├── entry.service.ts
├── event.service.ts
└── analytics.service.ts

redux/
├── slices/
```

```

├── authSlice.ts          # User, tokens, isAuthenticated
├── uiSlice.ts            # Sidebar state, modals, toasts
├── notificationSlice.ts  # Real-time notification queue
├── lockdownSlice.ts      # Lockdown status
├── selectors/
│   ├── authSelectors.ts
│   └── permissionSelectors.ts
├── routes/
│   ├── index.tsx         # Root router setup
│   ├── ProtectedRoute.tsx # Auth + role guard
│   ├── GuestRoute.tsx    # Redirect if already logged in
│   └── routeConfig.ts     # Route definitions + required roles
├── utils/
│   ├── formatDate.ts
│   ├── passStatusColor.ts
│   ├── roleHelpers.ts
│   └── errorHandler.ts
├── schemas/
│   ├── auth.schema.ts    # Zod schemas for auth forms
│   ├── pass.schema.ts    # Pass creation form schema
│   ├── visitor.schema.ts
│   └── vehicle.schema.ts
├── types/
│   ├── auth.types.ts
│   ├── pass.types.ts
│   ├── visitor.types.ts
│   ├── entry.types.ts
│   └── api.types.ts
├── assets/
│   ├── icons/
│   └── images/

```

7.3 State Management Strategy

Redux Toolkit — Global Client State

Store only persistent, cross-cutting state:

Slice	What It Stores
<code>authSlice</code>	Authenticated user profile, role, access token (memory only)
<code>uiSlice</code>	Sidebar open/close, active modal, toast queue
<code>notificationSlice</code>	Unread notification count, recent alerts from socket
<code>lockdownSlice</code>	<code>isLockdown</code> : <code>boolean</code> , lockdown details

React Query — Server State

All API data is server state. React Query handles:

- Caching (staleTime: 5 min for passes, 30 sec for entry logs)
- Background refetch
- Optimistic updates (approve/reject pass)
- Infinite scroll (entry log history)
- Mutation + cache invalidation

```
// hooks/usePasses.ts
export const usePasses = (filters: PassFilters) =>
  useQuery({
    queryKey: ['passes', filters],
    queryFn: () => passService.getAll(filters),
    staleTime: 1000 * 60 * 5,
  });

export const useApprovePass = () =>
  useMutation({
    mutationFn: (passId: string) => passService.approve(passId),
    onSuccess: (_, passId) => {
      queryClient.invalidateQueries({ queryKey: ['passes'] });
      queryClient.invalidateQueries({ queryKey: ['pass', passId] });
    },
  });
```

7.4 API Layer

```
// services/api.ts
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true, // Send httpOnly refresh token cookie
});

// Request interceptor: attach access token from Redux memory
api.interceptors.request.use((config) => {
  const token = store.getState().auth.accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Response interceptor: silent token refresh on 401
api.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401 && !error.config._retry) {
      error.config._retry = true;
      const { accessToken } = await authService.refreshToken();
    }
  }
);
```



```

        store.dispatch(setAccessToken(accessToken));
        return api(error.config);
    }
    return Promise.reject(error);
}
);

```

7.5 Routing & Protected Routes

```

// routes/ProtectedRoute.tsx
const ProtectedRoute = ({ roles }: { roles: UserRole[] }) => {
    const { user, isAuthenticated } = useAuth();
    const location = useLocation();

    if (!isAuthenticated)
        return <Navigate to="/login" state={{ from: location }} replace />;

    if (roles.length > 0 && !roles.includes(user.role))
        return <Navigate to="/unauthorized" replace />;

    return <Outlet />;
};

```

```

// routes/routeConfig.ts
export const routes = [
    { path: '/dashboard', element: <DashboardLayout />, roles: [ALL_ROLES] },
    { path: '/passes/create', element: <CreatePassPage />, roles: ['STUDENT',
'FACULTY', 'STAFF'] },
    { path: '/security/scan', element: <QRScanPage />, roles: ['GUARD',
'SECURITY_ADMIN'] },
    { path: '/admin/users', element: <UserManagementPage />, roles:
['UNIVERSITY_ADMIN', 'SUPER_ADMIN'] },
];

```

7.6 Form Management with React Hook Form + Zod

```

// schemas/pass.schema.ts
export const createPassSchema = z.object({
    visitorId: z.string().uuid(),
    passType: z.enum(['VISITOR', 'HOSTEL_GUEST', 'VEHICLE', 'EVENT', 'CONTRACTOR']),
    purpose: z.string().min(10).max(500),
    validFrom: z.date().min(new Date()),
    validTo: z.date(),
    allowedGates: z.array(z.string()).min(1),
    notes: z.string().max(1000).optional(),
});

```

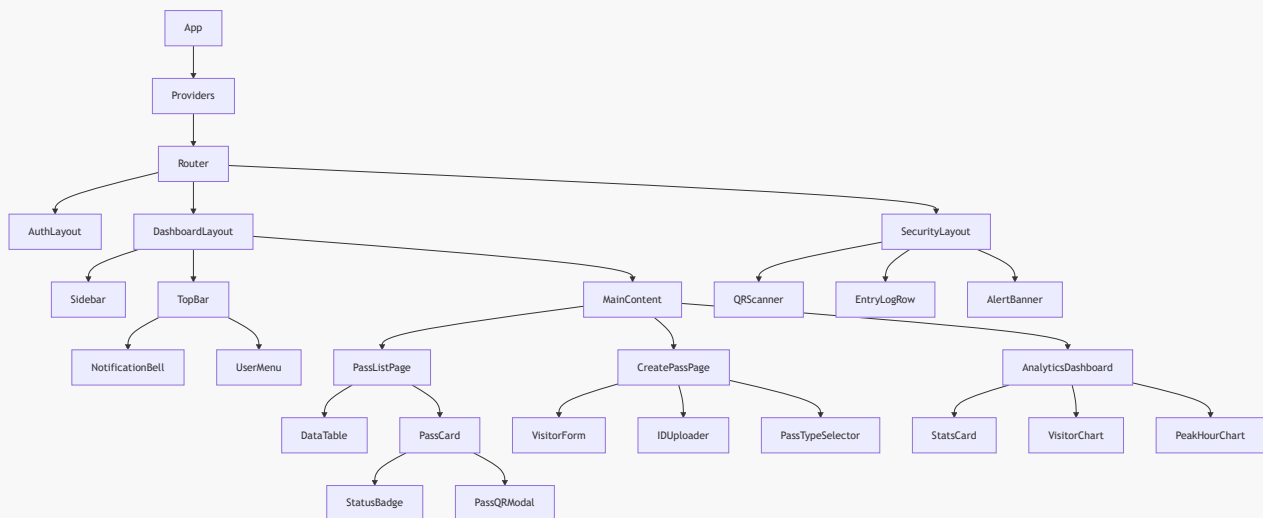
```

}).refine(d => d.validTo > d.validFrom, {
  message: 'End time must be after start time',
  path: ['validTo'],
});

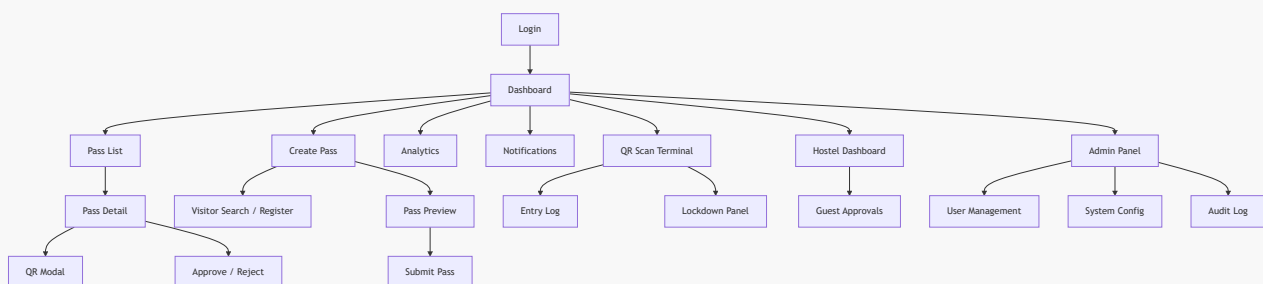
// Component usage
const { register, handleSubmit, formState: { errors } } =
  useForm<CreatePassInput>({ resolver: zodResolver(createPassSchema) });

```

7.7 Component Hierarchy



7.8 Page Navigation Diagram



7.9 Real-Time UI Integration

```

// hooks/useSocket.ts
export const useSocket = () => {
  const { accessToken } = useAuth();
  const dispatch = useDispatch();

```

```
useEffect(() => {
  const socket = io(import.meta.env.VITE_WS_URL, {
    auth: { token: accessToken },
  });

  socket.on('visitor:entered', (data) => {
    dispatch(addNotification({ type: 'info', message: `${data.visitorName}
entered at ${data.gate}` }));
    queryClient.invalidateQueries({ queryKey: ['entry-logs'] });
  });

  socket.on('lockdown:initiated', (data) => {
    dispatch(setLockdown({ active: true, reason: data.reason }));
  });

  socket.on('pass:approved', (data) => {
    queryClient.invalidateQueries({ queryKey: ['passes', data.passId] });
  });

  return () => socket.disconnect();
}, [accessToken]);
};
```

Section 8 — Node.js Backend Architecture

8.1 Architecture Overview



Syntax error in text
mermaid version 11.15.0

8.2 Folder Structure

```
src/
├── config/
│   ├── database.ts      # Prisma client singleton
│   ├── redis.ts         # Redis client singleton
│   ├── minio.ts         # MinIO client config
│   ├── mailer.ts        # Nodemailer transport
│   ├── env.ts           # Validated env vars (zod)
│   └── constants.ts
```

```
├─ controllers/
│   ├── auth.controller.ts
│   ├── user.controller.ts
│   ├── visitor.controller.ts
│   ├── pass.controller.ts
│   ├── approval.controller.ts
│   ├── qr.controller.ts
│   ├── entry.controller.ts
│   ├── hostel.controller.ts
│   ├── vehicle.controller.ts
│   ├── event.controller.ts
│   ├── analytics.controller.ts
│   └── admin.controller.ts
├─ services/
│   ├── auth.service.ts
│   ├── user.service.ts
│   ├── visitor.service.ts
│   ├── pass.service.ts
│   ├── approval.service.ts
│   ├── qr.service.ts           # QR generation + verification
│   ├── entry.service.ts
│   ├── notification.service.ts
│   ├── email.service.ts
│   ├── storage.service.ts     # MinIO file ops
│   ├── analytics.service.ts
│   └── audit.service.ts
├─ repositories/
│   ├── user.repository.ts
│   ├── visitor.repository.ts
│   ├── pass.repository.ts
│   ├── entry.repository.ts
│   ├── audit.repository.ts
│   └── base.repository.ts     # Generic typed Prisma wrapper
├─ middleware/
│   ├── auth.middleware.ts     # JWT verify → attach req.user
│   ├── rbac.middleware.ts     # Role/permission guard factory
│   ├── rateLimit.middleware.ts
│   ├── validate.middleware.ts # Zod schema validation factory
│   ├── upload.middleware.ts   # Multer config
│   ├── audit.middleware.ts    # Auto-log modifying requests
│   └── error.middleware.ts     # Global error handler
├─ routes/
│   ├── index.ts              # Mount all routers
│   ├── auth.routes.ts
│   ├── user.routes.ts
│   ├── visitor.routes.ts
│   ├── pass.routes.ts
│   ├── approval.routes.ts
│   ├── qr.routes.ts
│   └── entry.routes.ts
```

```

├── hostel.routes.ts
├── vehicle.routes.ts
├── event.routes.ts
├── analytics.routes.ts
├── admin.routes.ts
├── validators/
│   ├── auth.validator.ts
│   ├── pass.validator.ts
│   ├── visitor.validator.ts
│   └── entry.validator.ts
├── utils/
│   ├── asyncHandler.ts      # Wrap async controllers
│   ├── ApiError.ts         # Custom error class
│   ├── ApiResponse.ts      # Uniform response envelope
│   ├── pagination.ts
│   ├── tokenUtils.ts       # JWT sign/verify
│   ├── hashUtils.ts        # bcrypt + crypto helpers
│   └── logger.ts           # Winston instance
├── jobs/
│   ├── expirePass.job.ts   # Cron: mark expired passes
│   ├── overstayAlert.job.ts # Cron: detect overstays
│   ├── bulkPassEmail.job.ts # Queue: send bulk event emails
│   └── scheduler.ts        # node-cron setup
├── sockets/
│   ├── index.ts            # Socket.io server setup
│   ├── auth.socket.ts      # Socket JWT auth middleware
│   ├── events.socket.ts    # Event handlers
│   └── rooms.ts            # Room management utilities
├── prisma/
│   ├── schema.prisma
│   ├── seed.ts
│   └── migrations/
├── app.ts                  # Express app setup (no listen)
└── server.ts               # HTTP server + Socket.io attach

```

8.3 Layer Responsibilities

Controller Layer

- Receives validated HTTP request.
- Extracts params, body, query.
- Calls service method.
- Returns `ApiResponse` envelope.
- No business logic.

```
// controllers/pass.controller.ts
export const createPass = asyncHandler(async (req: AuthRequest, res) => {
  const pass = await passService.createPass(req.user.id, req.body);
  res.status(201).json(ApiResponse.success(pass, 'Pass created'));
});
```

Service Layer

- Business logic and orchestration.
- Calls repositories, external services (email, QR, storage).
- Throws `ApiError` for domain violations.
- Transaction management.

```
// services/pass.service.ts
async createPass(requesterId: string, dto: CreatePassDto): Promise<Pass> {
  await this.checkPassLimit(requesterId);
  const visitor = await visitorRepository.findById(dto.visitorId);
  if (visitor.blacklisted) throw new ApiError(403, 'Visitor is blacklisted');
  const pass = await passRepository.create({ ...dto, requesterId, status:
'PENDING' });
  await auditService.log({ action: 'PASS_CREATED', entity: 'Pass', entityId:
pass.id, userId: requesterId });
  await notificationService.notifyApprovers(pass);
  return pass;
}
```

Repository Layer

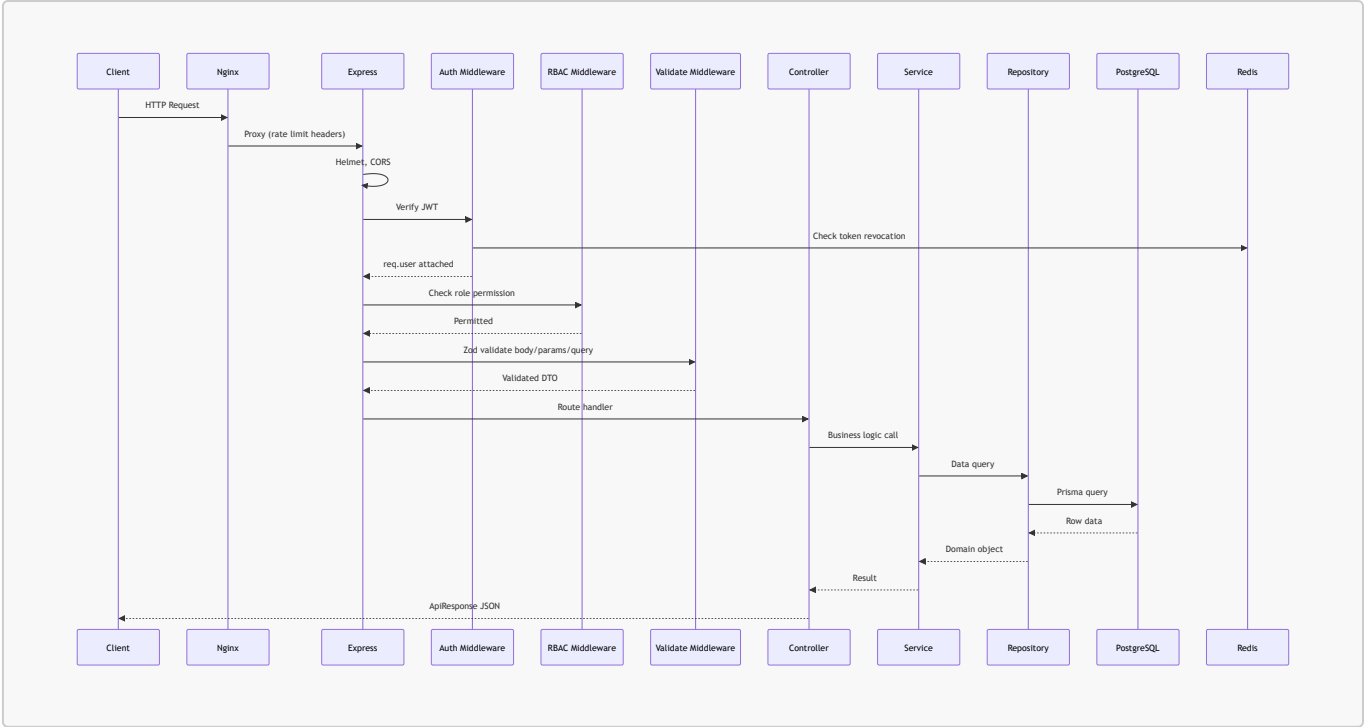
- Wraps Prisma queries.
- Returns typed domain objects.
- No business logic.
- Handles pagination, filtering, ordering.

```
// repositories/pass.repository.ts
async findMany(filters: PassFilters, pagination: Pagination) {
  return prisma.pass.findMany({
    where: buildPassWhereClause(filters),
    include: { visitor: true, requester: true, approvals: true },
    skip: pagination.skip,
    take: pagination.limit,
    orderBy: { createdAt: 'desc' },
  });
}
```

Middleware Layer

Middleware	Responsibility
authMiddleware	Verify JWT, decode user, attach to req.user
rbacMiddleware(roles)	Check req.user.role against required roles
permissionMiddleware(perm)	Fine-grained permission check via DB role-permissions
validateMiddleware(schema)	Run Zod schema; 422 if invalid
uploadMiddleware	Multer memStorage; max 5MB; MIME whitelist
rateLimitMiddleware	Redis sliding window; 100 req/min per IP
auditMiddleware	Log all non-GET requests to AuditLog table

8.4 Request Lifecycle Diagram



8.5 Error Handling Architecture

```
// utils/ApiError.ts
export class ApiError extends Error {
  constructor(
    public statusCode: number,
    public message: string,
    public errors: string[] = [],
    public isOperational = true
  ) { super(message); }
}

// middleware/error.middleware.ts
export const globalErrorHandler: ErrorRequestHandler = (err, req, res, next) => {
```

```
const isDev = process.env.NODE_ENV === 'development';

if (err instanceof ApiError) {
  return res.status(err.statusCode).json({
    success: false,
    message: err.message,
    errors: err.errors,
    ...(isDev && { stack: err.stack }),
  });
}

// Prisma unique constraint
if (err.code === 'P2002') {
  return res.status(409).json({ success: false, message: 'Duplicate entry' });
}

logger.error('Unhandled error', { err, url: req.url });
return res.status(500).json({ success: false, message: 'Internal server error' });
};
```

8.6 Background Jobs

```
// jobs/scheduler.ts
import cron from 'node-cron';

// Every minute: expire passes past validTo
cron.schedule('* * * * *', expirePassJob);

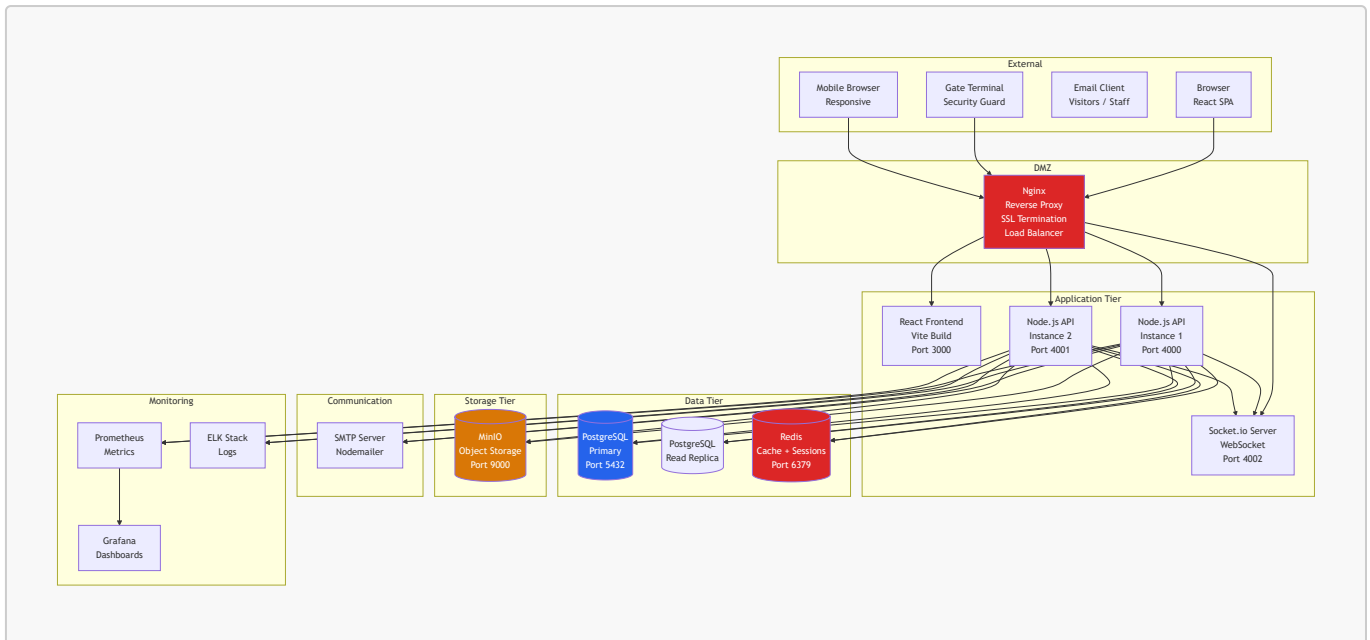
// Every 5 minutes: check for overstays
cron.schedule('*/5 * * * *', overstayAlertJob);

// Midnight: archive old reports
cron.schedule('0 0 * * *', archiveReportsJob);

// Every 30 minutes: send expiry warnings
cron.schedule('*/30 * * * *', passExpiryWarningJob);
```

Section 9 — System Architecture

9.1 High-Level System Architecture



9.2 Communication Flows

9.2.1 HTTPS Request Flow

```
Browser → Nginx (SSL terminate) → React SPA (static files)
Browser → Nginx (SSL terminate) → Node.js API (proxied to /api/v1/*)
```

All traffic between browser and Nginx is HTTPS (TLS 1.2+). Internal container-to-container traffic uses plain HTTP within Docker network (no public exposure).

9.2.2 WebSocket Flow

```
Browser/Terminal —WS (Upgrade)→ Nginx → Socket.io Server
Socket.io Server ← event emit — Node.js API (internal Redis Pub/Sub)
```

When running multiple API instances, Socket.io uses Redis adapter to broadcast events across all instances.

9.2.3 Authentication Flow

1. Client POST /auth/login → API validates credentials → issue JWT (access + httpOnly cookie refresh)
2. Client includes Bearer token in Authorization header for all API calls
3. JWT expires (15 min) → Client POST /auth/refresh → API rotates refresh token → new access token
4. Logout → API revokes refresh token in Redis → access token orphaned (expires naturally)

9.2.4 File Upload Flow

Client → Multipart POST → Multer (mem storage, 5MB limit) → Virus scan check → AES encrypt → MinIO PUT (private bucket) → DB record with MinIO key

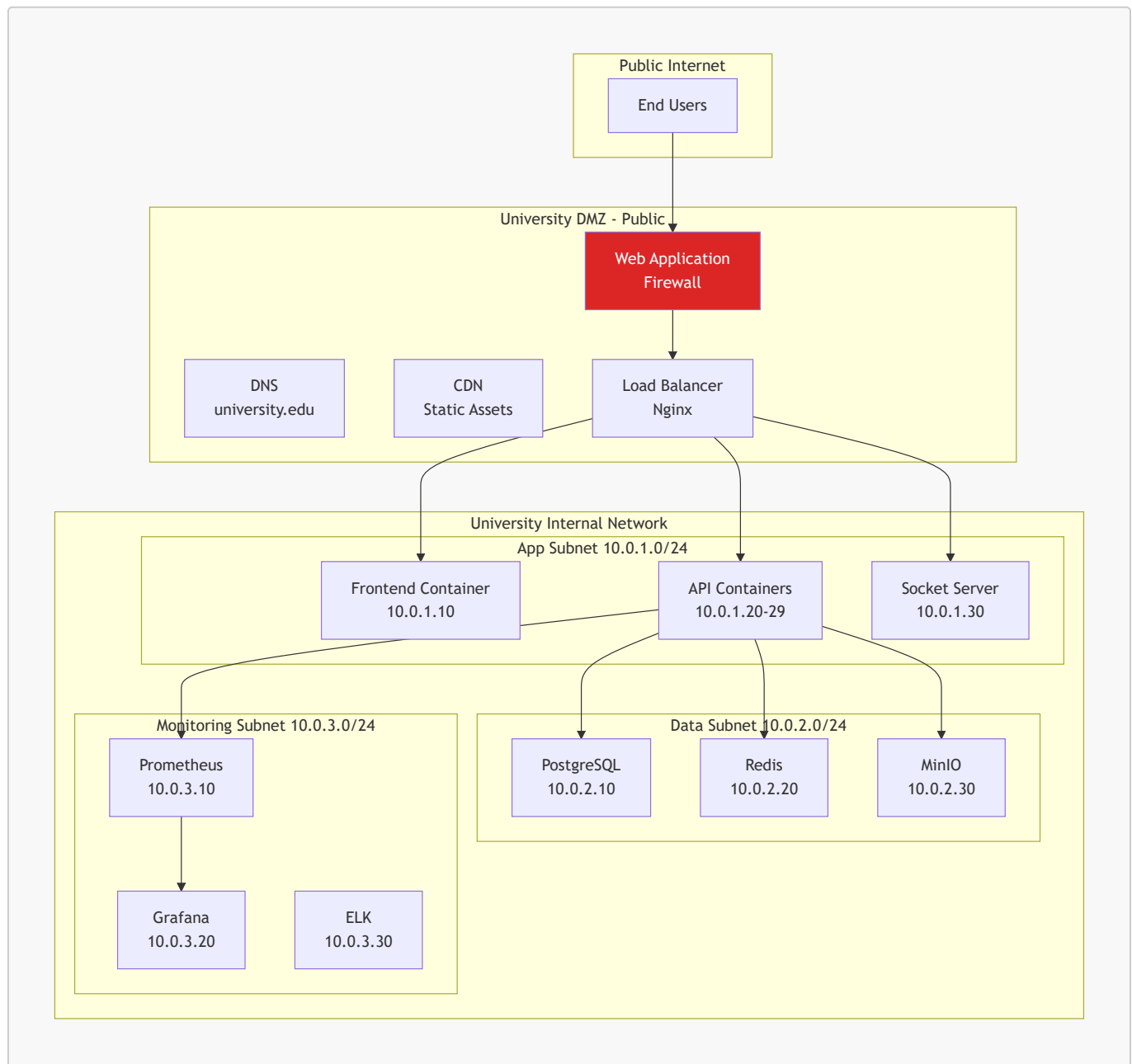
Retrieval: DB lookup → MinIO presigned URL (5 min TTL) → Client streams file directly from MinIO.

9.2.5 QR Verification Flow

Gate Terminal → POST /api/v1/qr/verify → API decrypt+verify → Redis cache lookup (pass status) → EntryLog INSERT → Socket emit → Response

Target latency: < 500 ms end-to-end.

9.3 Network Architecture



9.4 Container Networking

All services run on a Docker internal bridge network `gatepass_net`. Only Nginx is exposed on the host (ports 80/443). All other containers communicate by service name.

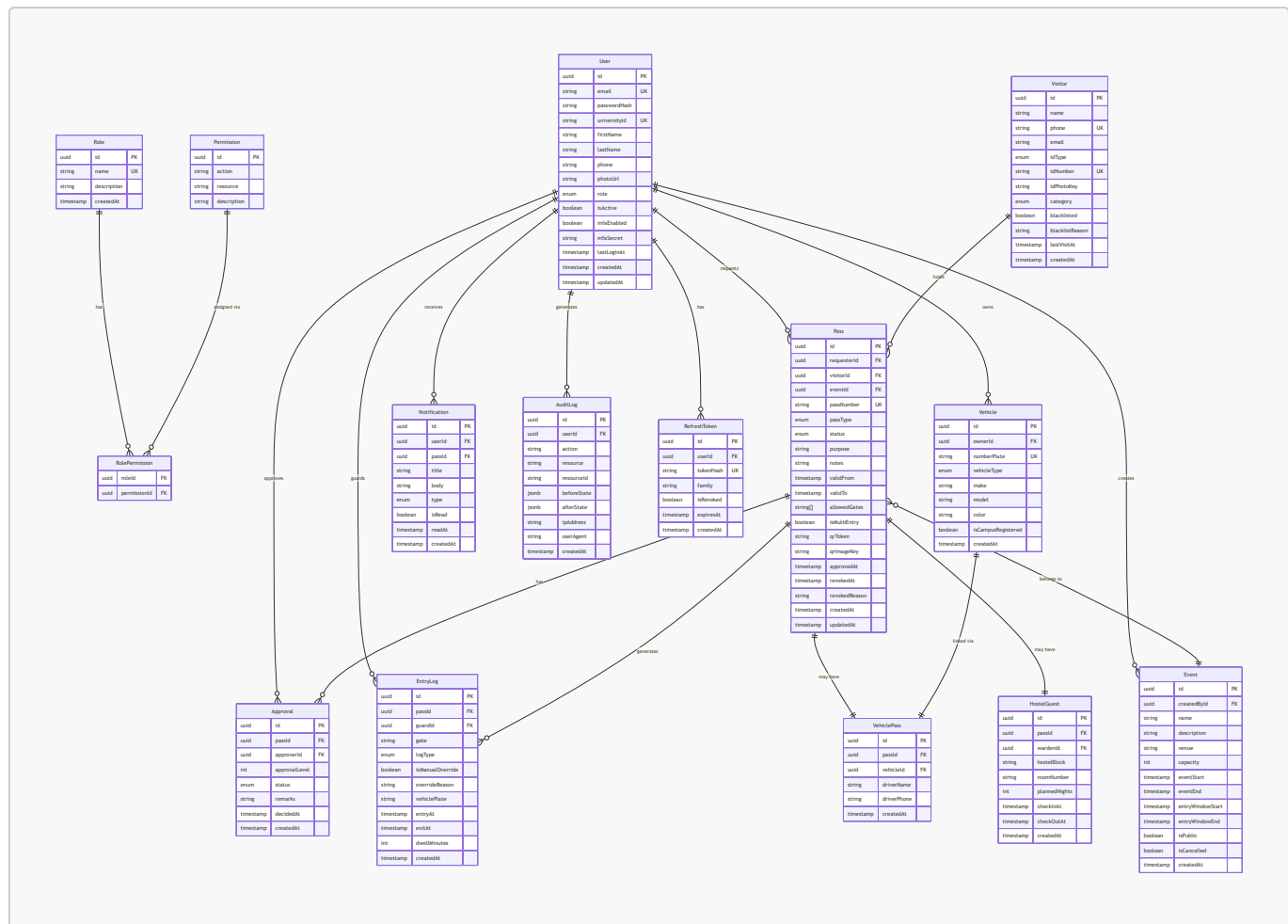
```
Nginx → http://frontend:3000
Nginx → http://api:4000
API → postgresql://db:5432/gatepass
API → redis://redis:6379
API → http://minio:9000
```

9.5 API Gateway Responsibilities (Nginx)

Function	Configuration
SSL Termination	Let's Encrypt or enterprise cert
HTTP → HTTPS Redirect	<code>return 301 https://...</code>
Static Asset Serving	Direct from <code>/dist</code> volume
API Proxy	<code>proxy_pass http://api:4000</code> with upstream round-robin
WebSocket Proxy	<code>proxy_http_version 1.1</code> + <code>Upgrade</code> + <code>Connection</code> headers
Rate Limiting	<code>limit_req_zone</code> by IP; 60 req/min burst 20
Gzip Compression	<code>gzip on</code> for text/html, application/json
Security Headers	Via Nginx <code>add_header</code> (backup to Helmet)
Request Size Limit	<code>client_max_body_size 10M</code>

Section 10 & 11 — Database Design & Prisma Schema

10.1 Entity Relationship Diagram



Users Table

Column	Type	Constraints	Notes
id	UUID	PK, DEFAULT gen_random_uuid()	
email	VARCHAR(255)	NOT NULL, UNIQUE	Indexed
password_hash	VARCHAR(255)	NOT NULL	bcrypt, never returned in API
university_id	VARCHAR(20)	UNIQUE, NULLABLE	For students/faculty
first_name	VARCHAR(100)	NOT NULL	
last_name	VARCHAR(100)	NOT NULL	
phone	VARCHAR(20)	NOT NULL	E.164 format
photo_url	TEXT	NULLABLE	MinIO key
role	user_role ENUM	NOT NULL, DEFAULT 'STUDENT'	
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	
mfa_enabled	BOOLEAN	NOT NULL, DEFAULT FALSE	

Column	Type	Constraints	Notes
mfa_secret	VARCHAR(100)	NULLABLE	Encrypted at rest
last_login_at	TIMESTAMPTZ	NULLABLE	
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	

Indexes: email, university_id, role, is_active

Passes Table

Column	Type	Constraints	Notes
id	UUID	PK	
requester_id	UUID	FK users(id), NOT NULL	Indexed
visitor_id	UUID	FK visitors(id), NOT NULL	Indexed
event_id	UUID	FK events(id), NULLABLE	
pass_number	VARCHAR(30)	NOT NULL, UNIQUE	Human-readable: GP-2025-00001
pass_type	pass_type ENUM	NOT NULL	
status	pass_status ENUM	NOT NULL, DEFAULT 'PENDING'	Indexed
purpose	TEXT	NOT NULL	
valid_from	TIMESTAMPTZ	NOT NULL	Indexed (composite with valid_to)
valid_to	TIMESTAMPTZ	NOT NULL	
allowed_gates	TEXT[]	NOT NULL	Array of gate IDs
is_multi_entry	BOOLEAN	NOT NULL, DEFAULT FALSE	
qr_token	VARCHAR(500)	UNIQUE, NULLABLE	Set on approval
qr_image_key	TEXT	NULLABLE	MinIO key
approved_at	TIMESTAMPTZ	NULLABLE	
revoked_at	TIMESTAMPTZ	NULLABLE	
revoked_reason	TEXT	NULLABLE	
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	

Indexes: `requester_id`, `visitor_id`, `status`, `(valid_from, valid_to)`, `pass_number`, `qr_token`

EntryLogs Table

Column	Type	Constraints	Notes
id	UUID	PK	
pass_id	UUID	FK passes(id), NOT NULL	Indexed
guard_id	UUID	FK users(id), NOT NULL	
gate	VARCHAR(50)	NOT NULL	Gate identifier
log_type	log_type ENUM	NOT NULL	ENTRY/EXIT
is_manual_override	BOOLEAN	NOT NULL, DEFAULT FALSE	
override_reason	TEXT	NULLABLE	
vehicle_plate	VARCHAR(20)	NULLABLE	If vehicle pass
entry_at	TIMESTAMPTZ	NULLABLE	
exit_at	TIMESTAMPTZ	NULLABLE	
dwel_minut	INTEGER	NULLABLE	Computed on exit
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	

Indexes: `pass_id`, `gate`, `entry_at`, `(pass_id, log_type)`

11.1 Complete Prisma Schema

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// — ENUMS —

enum UserRole {
  VISITOR
  STUDENT
  FACULTY
  STAFF
  SECURITY_GUARD
  HOSTEL_WARDEN
```

```
    SECURITY_ADMIN
    UNIVERSITY_ADMIN
    SUPER_ADMIN
}

enum PassType {
    VISITOR
    HOSTEL_GUEST
    VEHICLE
    EVENT
    CONTRACTOR
    PARENT
}

enum PassStatus {
    DRAFT
    SUBMITTED
    PENDING_APPROVAL
    APPROVED
    ACTIVE
    EXPIRED
    REJECTED
    REVOKED
    USED
    SUSPENDED
}

enum ApprovalStatus {
    PENDING
    APPROVED
    REJECTED
    ESCALATED
}

enum VehicleType {
    TWO_WHEELER
    FOUR_WHEELER
    TRUCK
    BUS
    OTHER
}

enum IdType {
    AADHAAR
    PASSPORT
    DRIVING_LICENSE
    VOTER_ID
    PAN_CARD
    OTHER
}

enum VisitorCategory {
    GENERAL
    ACADEMIC
```

```

GOVERNMENT
VIP
CONTRACTOR
VENDOR
PARENT
MEDICAL
}

enum LogType {
    ENTRY
    EXIT
}

enum NotificationType {
    PASS_CREATED
    PASS_APPROVED
    PASS_REJECTED
    PASS_REVOKED
    PASS_EXPIRY_WARNING
    VISITOR_ENTERED
    VISITOR_EXITED
    LOCKDOWN
    SECURITY_ALERT
    SYSTEM
}

// ——— MODELS —————

model User {
    id                String      @id @default(uuid())
    email             String      @unique
    passwordHash      String
    universityId      String?     @unique
    firstName         String
    lastName          String
    phone             String
    photoUrl          String?
    role              UserRole    @default(STUDENT)
    isActive          Boolean     @default(true)
    mfaEnabled        Boolean     @default(false)
    mfaSecret         String?
    lastLoginAt       DateTime?
    createdAt         DateTime    @default(now())
    updatedAt         DateTime    @updatedAt

    passes            Pass[]      @relation("PassRequester")
    approvals         Approval[]
    guardedLogs       EntryLog[]
    notifications     Notification[]
    auditLogs         AuditLog[]
    refreshTokens     RefreshToken[]
    vehicles          Vehicle[]
    createdEvents     Event[]
    hostelApprovals   HostelGuest[] @relation("WardenApprovals")

```



```

    @@index([email])
    @@index([universityId])
    @@index([role])
    @@index([isActive])
    @@map("users")
  }

model RefreshToken {
  id          String    @id @default(uuid())
  userId      String
  tokenHash   String    @unique
  family      String
  isRevoked   Boolean   @default(false)
  expiresAt   DateTime
  createdAt   DateTime @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([tokenHash])
  @@index([family])
  @@map("refresh_tokens")
}

model Visitor {
  id          String    @id @default(uuid())
  name        String
  phone       String    @unique
  email       String?
  idType      IdType
  idNumber    String    @unique
  idPhotoKey  String?
  category    VisitorCategory @default(GENERAL)
  blacklisted Boolean    @default(false)
  blacklistReason String?
  lastVisitAt DateTime?
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt

  passes Pass[]

  @@index([phone])
  @@index([idNumber])
  @@index([blacklisted])
  @@map("visitors")
}

model Pass {
  id          String    @id @default(uuid())
  requesterId String
  visitorId   String
  eventId     String?
  passNumber  String    @unique

```

```

    passType      PassType
    status        PassStatus @default(PENDING_APPROVAL)
    purpose       String
    notes         String?
    validFrom     DateTime
    validTo       DateTime
    allowedGates  String[]
    isMultiEntry  Boolean    @default(false)
    qrToken       String?    @unique
    qrImageKey    String?
    approvedAt    DateTime?
    revokedAt     DateTime?
    revokedReason String?
    createdAt     DateTime   @default(now())
    updatedAt     DateTime   @updatedAt

    requester     User        @relation("PassRequester", fields: [requesterId],
references: [id])
    visitor       Visitor     @relation(fields: [visitorId], references: [id])
    event         Event?      @relation(fields: [eventId], references: [id])
    approvals     Approval[]
    entryLogs     EntryLog[]
    vehiclePass   VehiclePass?
    hostelGuest   HostelGuest?
    notifications Notification[]

    @@index([requesterId])
    @@index([visitorId])
    @@index([status])
    @@index([validFrom, validTo])
    @@index([passNumber])
    @@map("passes")
}

model Approval {
  id          String      @id @default(uuid())
  passId      String
  approverId  String
  approvalLevel Int       @default(1)
  status      ApprovalStatus @default(PENDING)
  remarks     String?
  decidedAt   DateTime?
  createdAt   DateTime     @default(now())

  pass      Pass @relation(fields: [passId], references: [id], onDelete: Cascade)
  approver  User @relation(fields: [approverId], references: [id])

  @@index([passId])
  @@index([approverId])
  @@map("approvals")
}

model EntryLog {
  id          String      @id @default(uuid())

```

```

    passId          String
    guardId         String
    gate            String
    logType         LogType
    isManualOverride Boolean @default(false)
    overrideReason  String?
    vehiclePlate    String?
    entryAt         DateTime?
    exitAt          DateTime?
    dwellMinutes    Int?
    createdAt       DateTime @default(now())

    pass  Pass @relation(fields: [passId], references: [id])
    guard User @relation(fields: [guardId], references: [id])

    @@index([passId])
    @@index([gate])
    @@index([entryAt])
    @@index([logType])
    @@map("entry_logs")
}

model Vehicle {
  id          String @id @default(uuid())
  ownerId     String
  numberPlate String @unique
  vehicleType VehicleType
  make        String?
  model       String?
  color       String?
  isCampusRegistered Boolean @default(false)
  createdAt   DateTime @default(now())

  owner      User @relation(fields: [ownerId], references: [id])
  vehiclePasses VehiclePass[]

  @@index([numberPlate])
  @@map("vehicles")
}

model VehiclePass {
  id          String @id @default(uuid())
  passId      String @unique
  vehicleId   String
  driverName  String
  driverPhone String
  createdAt   DateTime @default(now())

  pass  Pass @relation(fields: [passId], references: [id], onDelete: Cascade)
  vehicle Vehicle @relation(fields: [vehicleId], references: [id])

  @@map("vehicle_passes")
}

```

```

model HostelGuest {
  id          String      @id @default(uuid())
  passId      String      @unique
  wardenId    String
  hostelBlock String
  roomNumber  String
  plannedNights Int
  checkInAt   DateTime?
  checkOutAt  DateTime?
  createdAt   DateTime    @default(now())

  pass  Pass @relation(fields: [passId], references: [id], onDelete: Cascade)
  warden User @relation("WardenApprovals", fields: [wardenId], references: [id])

  @@index([hostelBlock])
  @@map("hostel_guests")
}

model Event {
  id          String      @id @default(uuid())
  createdById String
  name        String
  description  String?
  venue       String
  capacity    Int
  eventStart  DateTime
  eventEnd    DateTime
  entryWindowStart DateTime
  entryWindowEnd DateTime
  isPublic    Boolean    @default(true)
  isCancelled  Boolean    @default(false)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  createdBy User @relation(fields: [createdById], references: [id])
  passes      Pass[]

  @@index([eventStart])
  @@map("events")
}

model Notification {
  id          String      @id @default(uuid())
  userId      String
  passId      String?
  title       String
  body        String
  type        NotificationType
  isRead      Boolean      @default(false)
  readAt      DateTime?
  createdAt   DateTime      @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  pass Pass? @relation(fields: [passId], references: [id], onDelete: SetNull)
}

```

```

    @@index([userId, isRead])
    @@index([createdAt])
    @@map("notifications")
}

model AuditLog {
  id          String    @id @default(uuid())
  userId      String?
  action      String
  resource    String
  resourceId  String?
  beforeState Json?
  afterState  Json?
  ipAddress   String?
  userAgent   String?
  createdAt   DateTime @default(now())

  user User? @relation(fields: [userId], references: [id], onDelete: SetNull)

  @@index([userId])
  @@index([resource, resourceId])
  @@index([createdAt])
  @@map("audit_logs")
}
```

11.2 Database Indexes Strategy

Table	Index	Reason
passes	(status, valid_from, valid_to)	Composite for active pass queries
passes	qr_token	QR verification critical path
entry_logs	(pass_id, log_type)	Find open entry (no exit)
entry_logs	(gate, entry_at)	Gate throughput reports
audit_logs	(resource, resource_id, created_at)	Audit trail queries
notifications	(user_id, is_read)	Unread count queries
visitors	id_number	Duplicate detection on registration

11.3 Database Partitioning Strategy

For high-volume tables, implement PostgreSQL range partitioning by date:

```
-- Partition entry_logs by month
CREATE TABLE entry_logs_2025_01 PARTITION OF entry_logs
FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
```

```
-- Partition audit_logs by month
CREATE TABLE audit_logs_2025_01 PARTITION OF audit_logs
FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
```

Older partitions can be archived to read replicas or cold storage.

Section 12 — REST API Documentation

Base URL: <https://api.university.edu/api/v1>

Auth: Bearer JWT in [Authorization](#) header

Content-Type: [application/json](#) (except file uploads: [multipart/form-data](#))

Response Envelope:

```
{
  "success": true,
  "message": "Human-readable message",
  "data": {},
  "meta": { "page": 1, "limit": 20, "total": 150 }
}
```

12.1 Authentication APIs

POST /auth/register

Description: Self-register as Student or Faculty

Auth: Public

Request Body:

```
{
  "email": "john.doe@university.edu",
  "password": "SecurePass@123",
  "firstName": "John",
  "lastName": "Doe",
  "phone": "+919876543210",
  "universityId": "STU2025001",
  "role": "STUDENT"
}
```

Response 201:

```
{
  "success": true,
```

```
"message": "Registration successful. Please verify your email.",
"data": { "userId": "uuid", "email": "john.doe@university.edu" }
}
```

Errors: 409 email exists | 422 validation failure | 400 domain not allowed

POST /auth/login

Description: Authenticate user; receive access token + set httpOnly refresh cookie

Auth: Public

Request Body:

```
{
  "email": "john.doe@university.edu",
  "password": "SecurePass@123",
  "mfaToken": "123456"
}
```

Response 200:

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": "uuid",
      "email": "john.doe@university.edu",
      "firstName": "John",
      "lastName": "Doe",
      "role": "STUDENT"
    }
  }
}
```

Headers Set: Set-Cookie: refreshToken=...; HttpOnly; Secure; SameSite=Strict;
Path=/auth/refresh

Errors: 401 invalid credentials | 403 account locked | 428 MFA required

POST /auth/refresh

Description: Rotate refresh token; receive new access token

Auth: httpOnly cookie refreshToken

Response 200: New accessToken in body; new refresh token in cookie

Errors: 401 token invalid/revoked | 403 token family compromised (all sessions revoked)

POST /auth/logout

Auth: Bearer

Response 200: Refresh token revoked; cookie cleared

POST /auth/forgot-password

Auth: Public

Body: { "email": "...", "password": "...", "confirmPassword": "...", "otp": "..." }

Response 200: Always returns success (no email enumeration)

POST /auth/reset-password

Auth: Public

Body: { "token": "otp-from-email", "email": "...", "newPassword": "...", "confirmPassword": "..." }

Errors: 400 token expired/invalid | 422 password too weak

12.2 Visitor APIs

GET /visitors

Auth: Guard, Sec Admin, Uni Admin, Super Admin

Query Params: search, category, blacklisted, page, limit

Response 200: Paginated visitor list (ID numbers masked for Guard role)

POST /visitors

Auth: Student, Faculty, Staff, Security Admin

Content-Type: multipart/form-data

Fields:

```
name: "Jane Smith"
phone: "+919876543211"
email: "jane@example.com"
idType: "AADHAAR"
idNumber: "1234-5678-9012"
category: "GENERAL"
idPhoto: [file]
```

Response 201:

```
{
  "success": true,
  "data": {
    "id": "visitor-uuid",
  }
}
```



```
"name": "Jane Smith",
"phone": "+919876543211",
"category": "GENERAL",
"blacklisted": false
}
}
```

Errors: 409 visitor with this ID already exists (returns existing record) | 400 virus scan failed

GET /visitors/:id

Auth: Bearer (own requests + guards + admins)

Response 200: Full visitor profile (with ID masked for lower roles)

PATCH /visitors/:id/blacklist

Auth: Security Admin, Super Admin

Body: { "blacklisted": true, "reason": "Suspicious activity on 2025-01-15" }

Side Effect: All pending/approved passes for this visitor are revoked; notifications sent.

12.3 Pass APIs

GET /passes

Auth: Bearer (filtered by role)

Query: status, passType, requesterId, visitorId, from, to, page, limit

Notes: Students see only their passes; guards see all; admins see all with full PII.

POST /passes

Auth: Student, Faculty, Staff, Sec Admin

Body:

```
{
  "visitorId": "visitor-uuid",
  "passType": "VISITOR",
  "purpose": "Attending research seminar with Dr. Sharma",
  "validFrom": "2025-03-15T09:00:00Z",
  "validTo": "2025-03-15T17:00:00Z",
  "allowedGates": ["MAIN_GATE", "NORTH_GATE"],
  "isMultiEntry": false,
  "notes": "Will arrive by car. Registration number: MH01AB1234"
}
```

Response 201:

```
{
  "success": true,
  "data": {
    "id": "pass-uuid",
    "passNumber": "GP-2025-00347",
    "status": "PENDING_APPROVAL",
    "createdAt": "2025-03-14T10:30:00Z"
  }
}
```

Errors: 403 visitor blacklisted | 422 validation | 429 pass limit reached

GET /passes/:id

Auth: Bearer

Response 200: Full pass details including visitor info, approval history, QR image URL (presigned, 5-min TTL)

PATCH /passes/:id/revoke

Auth: Requester (own), Sec Admin, Uni Admin

Body: { "reason": "Visitor cancelled visit" }

Side Effect: QR invalidated; visitor notified by email.

12.4 Approval APIs

GET /approvals/pending

Auth: Faculty (dept), Hostel Warden (hostel), Sec Admin, Uni Admin

Response 200: List of passes awaiting this approver's decision

POST /approvals/:passId/approve

Auth: Approver roles

Body: { "remarks": "Approved for research visit" }

Response 200: Pass status updated; QR generated if final approval level

Side Effect: Email with QR sent to requester and visitor

POST /approvals/:passId/reject

Auth: Approver roles

Body: { "remarks": "Purpose unclear – please resubmit with more details" }

Response 200: Pass status → REJECTED; requester notified

12.5 QR APIs

POST /qr/verify

Auth: Guard terminal (Bearer)

Body:

```
{
  "qrData": "AES256-ENCRYPTED-PAYLOAD",
  "gateId": "MAIN_GATE",
  "guardId": "guard-uuid"
}
```

Response 200:

```
{
  "success": true,
  "data": {
    "allowed": true,
    "passId": "pass-uuid",
    "passNumber": "GP-2025-00347",
    "visitorName": "Jane Smith",
    "visitorPhoto": "https://minio.../presigned",
    "passType": "VISITOR",
    "purpose": "Research seminar",
    "validTo": "2025-03-15T17:00:00Z"
  }
}
```

Denial Response:

```
{
  "success": true,
  "data": {
    "allowed": false,
    "reason": "PASS_EXPIRED",
    "message": "Pass expired at 17:00 on 15 Mar 2025"
  }
}
```

Denial Reasons: INVALID_QR | SIGNATURE_MISMATCH | PASS_NOT_ACTIVE | PASS_EXPIRED | WRONG_GATE | PASS_ALREADY_USED | LOCKDOWN_ACTIVE

12.6 Entry/Exit APIs

POST /entry/log

Auth: Guard

Body:

```
{
  "passId": "pass-uuid",
  "gateId": "MAIN_GATE",
  "logType": "ENTRY",
  "vehiclePlate": null
}
```

Response 201: Entry log created; real-time notification sent

POST /entry/exit

Auth: Guard

Body: { "passId": "pass-uuid", "gateId": "MAIN_GATE" }

Response 200: Exit logged; dwell time calculated

GET /entry/logs

Auth: Guard, Sec Admin, Uni Admin

Query: gate, passType, from, to, page, limit

Response 200: Paginated entry/exit log

GET /entry/active

Auth: Guard, Sec Admin

Response 200: All visitors currently on campus (entered but not exited)

12.7 Vehicle APIs

POST /vehicles

Auth: Staff, Faculty, Sec Admin

Body: { "numberPlate": "MH01AB1234", "vehicleType": "FOUR_WHEELER", "make": "Toyota",
"model": "Fortuner", "color": "White" }

POST /vehicles/passes

Auth: Staff, Sec Admin

Body: Extends regular pass body with { "vehicleId": "vehicle-uuid", "driverName": "...",
"driverPhone": "..." }

12.8 Event APIs

GET /events

Auth: Public (for public events) / Bearer

Query: `upcoming`, `page`, `limit`

POST /events

Auth: Sec Admin, Uni Admin

Body:

```
{
  "name": "Annual Tech Symposium 2025",
  "description": "...",
  "venue": "Main Auditorium",
  "capacity": 500,
  "eventStart": "2025-04-10T09:00:00Z",
  "eventEnd": "2025-04-10T17:00:00Z",
  "entryWindowStart": "2025-04-10T08:00:00Z",
  "entryWindowEnd": "2025-04-10T10:30:00Z",
  "isPublic": true
}
```

POST /events/:id/passess/bulk

Auth: Sec Admin, Uni Admin

Body: `{ "registrations": [{ "name": "...", "email": "...", "phone": "..." }] }`

Response 202: Background job queued; `jobId` returned

Side Effect: Each attendee receives pass by email

GET /events/:id/checkins

Auth: Sec Admin, Uni Admin

Response 200: Real-time check-in count and attendee list

12.9 Analytics APIs

GET /analytics/summary

Auth: Sec Admin, Uni Admin, Super Admin

Query: `from`, `to`

Response 200:

```
{
  "data": {
    "totalPasses": 1250,
  }
}
```

```
"activeToday": 87,  
"pendingApproval": 12,  
"rejectedThisWeek": 5,  
"avgDwellMinutes": 145,  
"topGate": "MAIN_GATE",  
"peakHour": "09:00-10:00",  
"passTypeBreakdown": {  
  "VISITOR": 650,  
  "HOSTEL_GUEST": 180,  
  "CONTRACTOR": 120,  
  "EVENT": 300  
}  
}  
}
```

GET /analytics/entry-trends

Auth: Sec Admin, Uni Admin

Query: `from, to, groupBy=hour|day|week`

Response 200: Array of { `date, entryCount, exitCount` }

GET /analytics/export

Auth: Sec Admin, Uni Admin

Query: `reportType, from, to, format=csv|pdf`

Response 200: File download stream

Headers: `Content-Disposition: attachment; filename="report-2025-03.csv"`

12.10 Admin APIs

GET /admin/users

Auth: Uni Admin, Super Admin

Query: `role, isActive, search, page, limit`

POST /admin/users

Auth: Uni Admin, Super Admin

Body: Full user creation DTO (used to create guards, wardens, admins)

PATCH /admin/users/:id/deactivate

Auth: Uni Admin, Super Admin

Side Effect: All active sessions revoked; all pending passes suspended

POST /admin/lockdown

Auth: Sec Admin, Uni Admin

Body: { "scope": "ALL_CAMPUS", "reason": "Security threat reported", "gates": [] }

Response 200: Lockdown activated; all gates notified via Socket.io

DELETE /admin/lockdown

Auth: Sec Admin, Uni Admin

Response 200: Lockdown lifted; suspended passes restored

GET /admin/audit-logs

Auth: Sec Admin, Uni Admin, Super Admin

Query: `userId`, `resource`, `action`, `from`, `to`, `page`, `limit`

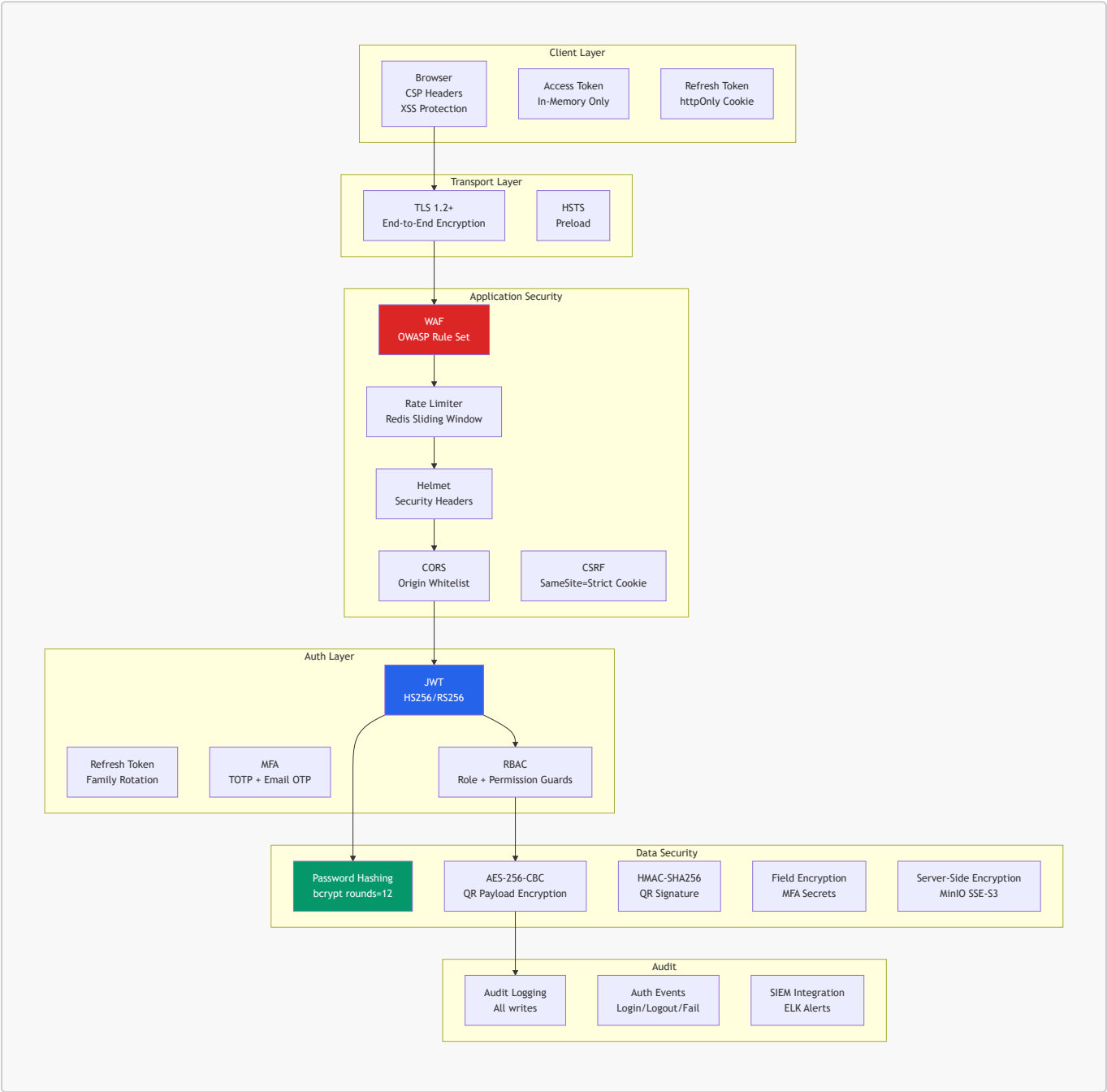
Response 200: Paginated audit trail with before/after state diffs

12.11 Error Codes Reference

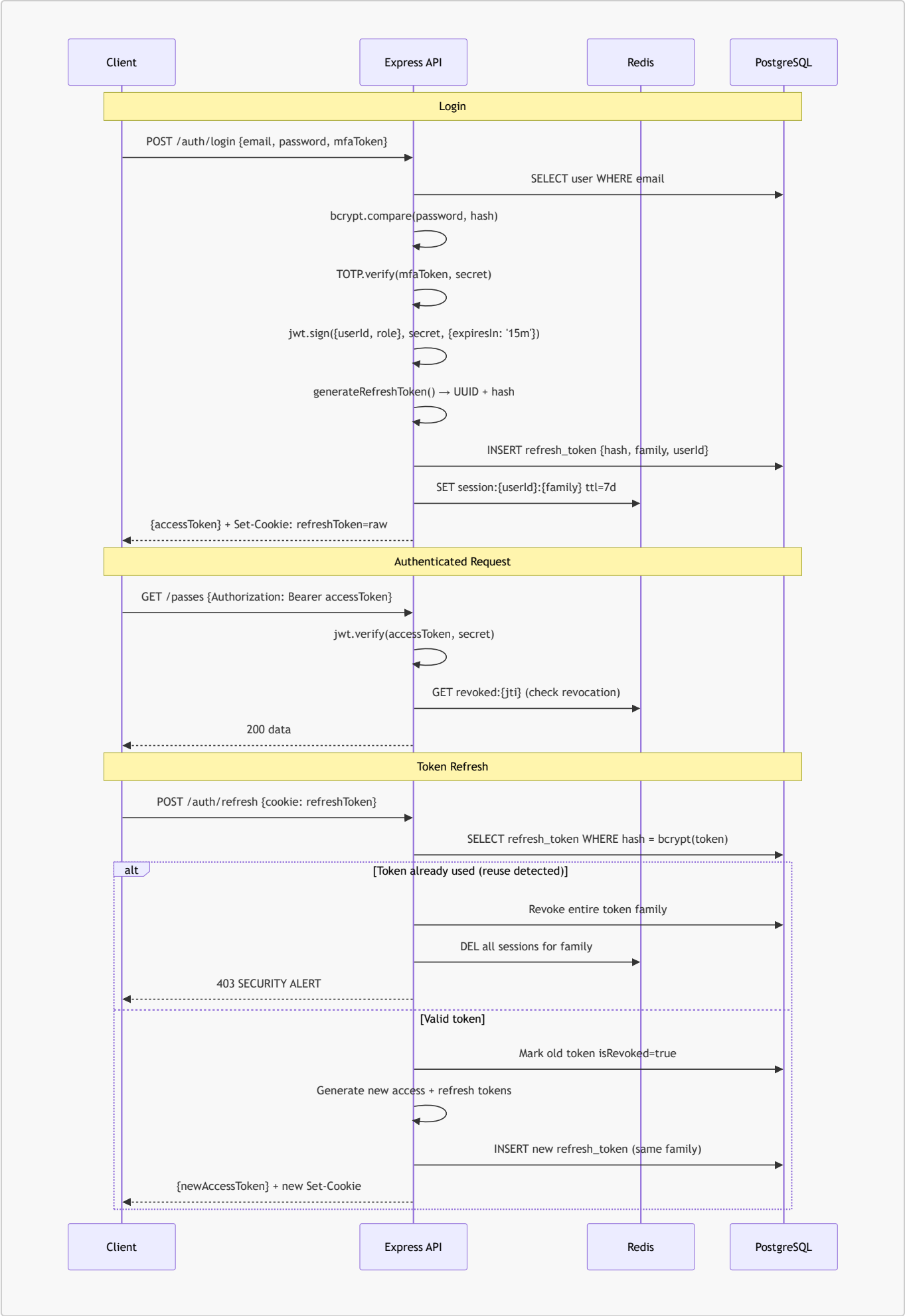
Code	HTTP Status	Meaning
AUTH_001	401	Missing or invalid access token
AUTH_002	401	Access token expired
AUTH_003	403	Insufficient permissions
AUTH_004	403	Account locked
AUTH_005	428	MFA required
PASS_001	422	Pass limit exceeded
PASS_002	403	Visitor blacklisted
PASS_003	409	Duplicate pass for same visitor+date
QR_001	200	Invalid QR (denial — not HTTP error)
QR_002	200	Pass expired
QR_003	200	Wrong gate
FILE_001	400	File too large (>5MB)
FILE_002	400	Unsupported file type
FILE_003	400	Virus scan failed
SYS_001	429	Rate limit exceeded
SYS_002	503	Lockdown active

Section 13 — Security Architecture

13.1 Security Architecture Overview



13.2 JWT Authentication Flow



13.3 RBAC Implementation

```
// middleware/rbac.middleware.ts

// Role-based guard
export const requireRole = (...roles: UserRole[]) =>
  (req: AuthRequest, res: Response, next: NextFunction) => {
    if (!roles.includes(req.user.role)) {
      throw new ApiError(403, 'Insufficient role');
    }
    next();
  };

// Permission-based guard (fine-grained)
export const requirePermission = (action: string, resource: string) =>
  async (req: AuthRequest, res: Response, next: NextFunction) => {
    const hasPermission = await permissionService.userHasPermission(
      req.user.id,
      action,
      resource
    );
    if (!hasPermission) throw new ApiError(403, 'Permission denied');
    next();
  };

// Usage in routes
router.patch(
  '/passes/:id/revoke',
  authMiddleware,
  requireRole('SECURITY_ADMIN', 'UNIVERSITY_ADMIN', 'STUDENT', 'FACULTY',
'STAFF'),
  ownResourceOrAdmin('pass'), // Students/faculty/staff can only revoke own
  passes
  validateMiddleware(revokePassSchema),
  passController.revokePass
);
```

13.4 Rate Limiting Configuration

```
// Redis-backed sliding window rate limiter
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 10, // 10 login attempts per window
  keyGenerator: (req) => req.ip,
  handler: (req, res) => {
    auditService.log({ action: 'RATE_LIMIT_HIT', ipAddress: req.ip });
    res.status(429).json({ message: 'Too many attempts. Try again in 15 minutes.'
});
  },
});
```

```
store: new RedisStore({ client: redisClient })),
});

const apiLimiter = rateLimit({
  windowMs: 60 * 1000,    // 1 minute
  max: 100,
  keyGenerator: (req) => req.user?.id || req.ip,
  store: new RedisStore({ client: redisClient })),
});

const qrVerifyLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 300,    // Gate terminals scan rapidly
  keyGenerator: (req) => req.body.gateId || req.ip,
});
```

13.5 Security Headers (Helmet)

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      scriptSrc: ['self'],
      styleSrc: ['self', 'unsafe-inline'], // Tailwind inline styles
      imgSrc: ['self', 'data:', 'https://minio.university.edu'],
      connectSrc: ['self', 'wss://api.university.edu'],
      fontSrc: ['self'],
      objectSrc: ['none'],
      frameAncestors: ['none'],
    },
  },
  hsts: { maxAge: 31536000, includeSubDomains: true, preload: true },
  noSniff: true,
  xssFilter: true,
  referrerPolicy: { policy: 'strict-origin-when-cross-origin' },
}));
```

13.6 CORS Configuration

```
app.use(cors({
  origin: (origin, callback) => {
    const allowedOrigins = [
      'https://university.edu',
      'https://www.university.edu',
      process.env.NODE_ENV === 'development' && 'http://localhost:5173',
    ].filter(Boolean);
```

```

    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('CORS policy violation'));
    }
  },
  credentials: true, // Required for httpOnly cookie
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));

```

13.7 QR Code Security

```

// utils/qrCrypto.ts
const ALGORITHM = 'aes-256-cbc';
const QR_KEY = Buffer.from(process.env.QR_ENCRYPTION_KEY, 'hex'); // 32 bytes
const HMAC_SECRET = process.env.QR_HMAC_SECRET;

export function encryptQRPayload(payload: QRPayload): string {
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv(ALGORITHM, QR_KEY, iv);
  const json = JSON.stringify(payload);
  const encrypted = Buffer.concat([cipher.update(json, 'utf8'), cipher.final()]);
  const hmac = crypto.createHmac('sha256', HMAC_SECRET)
    .update(encrypted).digest('hex');
  // Format: iv:encrypted:hmac (all hex)
  return `${iv.toString('hex')}:${encrypted.toString('hex')}:${hmac}`;
}

export function decryptAndVerifyQR(qrData: string): QRPayload {
  const [ivHex, encHex, providedHmac] = qrData.split(':');
  const encrypted = Buffer.from(encHex, 'hex');

  // Verify HMAC first (timing-safe compare)
  const expectedHmac = crypto.createHmac('sha256', HMAC_SECRET)
    .update(encrypted).digest('hex');
  if (!crypto.timingSafeEqual(Buffer.from(providedHmac),
    Buffer.from(expectedHmac))) {
    throw new ApiError(400, 'QR signature invalid');
  }

  const iv = Buffer.from(ivHex, 'hex');
  const decipher = crypto.createDecipheriv(ALGORITHM, QR_KEY, iv);
  const decrypted = Buffer.concat([decipher.update(encrypted), decipher.final()]);
  return JSON.parse(decrypted.toString('utf8'));
}

```

13.8 Password Security

```
// utils/hashUtils.ts
const BCRYPT_ROUNDS = 12;

export const hashPassword = (plain: string) =>
  bcrypt.hash(plain, BCRYPT_ROUNDS);

export const verifyPassword = (plain: string, hash: string) =>
  bcrypt.compare(plain, hash);

// Password strength enforced by Zod schema:
const passwordSchema = z.string()
  .min(8)
  .regex(/[A-Z]/, 'Must contain uppercase')
  .regex(/[0-9]/, 'Must contain number')
  .regex(/^[A-Za-z0-9]/, 'Must contain special character');
```

13.9 Secure File Upload

```
// middleware/upload.middleware.ts
const ALLOWED_MIME = ['image/jpeg', 'image/png', 'application/pdf'];
const MAX_SIZE = 5 * 1024 * 1024; // 5MB

export const uploadMiddleware = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: MAX_SIZE },
  fileFilter: (req, file, cb) => {
    if (!ALLOWED_MIME.includes(file.mimetype)) {
      return cb(new ApiError(400, 'Unsupported file type'));
    }
    cb(null, true);
  },
});

// In storage service: scan before store
async function storeIDDocument(file: Express.Multer.File, visitorId: string) {
  await virusScan(file.buffer); // ClamAV integration
  const encrypted = await encryptBuffer(file.buffer);
  const key = `visitors/${visitorId}/id-${Date.now()}.enc`;
  await minioClient.putObject('secure-docs', key, encrypted, {
    'Content-Type': 'application/octet-stream',
    'x-amz-server-side-encryption': 'AES256',
  });
  return key;
}
```

13.10 OWASP Top 10 Protection Matrix

OWASP Risk	Protection Implemented
A01 Broken Access Control	RBAC middleware on every route; ownership checks; no direct object reference without auth
A02 Cryptographic Failures	AES-256 for QR/files; bcrypt for passwords; TLS 1.2+ in transit; HSTS
A03 Injection	Prisma parameterized queries; Zod input validation; no raw SQL
A04 Insecure Design	Threat-modeled flows; pass state machine prevents invalid transitions
A05 Security Misconfiguration	Helmet headers; Docker non-root user; env var validation at startup
A06 Vulnerable Components	<code>npm audit</code> in CI; Dependabot alerts; regular dependency updates
A07 Auth Failures	Refresh token rotation; account lockout; rate limiting on login
A08 Software & Data Integrity	HMAC-signed QR codes; Docker image signing; CI/CD verified builds
A09 Logging & Monitoring	Full audit log; Winston structured logs; Prometheus alerts on anomalies
A10 SSRF	No user-controlled URLs fetched server-side; MinIO internal only

13.11 Audit Logging

All write operations (CREATE, UPDATE, DELETE) and all authentication events are automatically logged:

```
// middleware/audit.middleware.ts
export const auditMiddleware = (action: string, resource: string) =>
  async (req: AuthRequest, res: Response, next: NextFunction) => {
    const originalJson = res.json.bind(res);
    res.json = (data) => {
      if (res.statusCode < 400) {
        auditRepository.create({
          userId: req.user?.id,
          action,
          resource,
          resourceId: req.params.id,
          ipAddress: req.ip,
          userAgent: req.headers['user-agent'],
          afterState: data?.data,
        });
      }
      return originalJson(data);
    };
    next();
  };
};
```

Section 14 — Real-Time System Design

14.1 Socket.io Architecture



Syntax error in text
mermaid version 11.15.0

14.2 Socket.io Server Setup

```
// sockets/index.ts
import { Server } from 'socket.io';
import { createAdapter } from '@socket.io/redis-adapter';
import { createClient } from 'redis';

export function initSocketServer(httpServer: HttpServer) {
  const io = new Server(httpServer, {
    cors: {
      origin: process.env.FRONTEND_URL,
      credentials: true,
    },
    transports: ['websocket', 'polling'],
  });

  // Redis adapter for multi-instance support
  const pubClient = createClient({ url: process.env.REDIS_URL });
  const subClient = pubClient.duplicate();
  io.adapter(createAdapter(pubClient, subClient));

  // Mount namespaces
  const notificationsNS = io.of('/notifications');
  const securityNS = io.of('/security');
  const gatesNS = io.of('/gates');

  // Apply JWT auth middleware to all namespaces
  [notificationsNS, securityNS, gatesNS].forEach(ns => {
    ns.use(socketAuthMiddleware);
  });

  setupNotificationHandlers(notificationsNS);
  setupSecurityHandlers(securityNS);
  setupGateHandlers(gatesNS);
}
```

```
    return io;
  }
```

14.3 Socket Authentication Middleware

```
// sockets/auth.socket.ts
export const socketAuthMiddleware = async (socket: Socket, next: Function) => {
  const token = socket.handshake.auth.token;
  if (!token) return next(new Error('Authentication required'));

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET) as JWPayload;
    const user = await userRepository.findById(decoded.userId);
    if (!user?.isActive) return next(new Error('Account inactive'));

    socket.data.user = user;
    next();
  } catch {
    next(new Error('Invalid token'));
  }
};
```

14.4 Room Management

```
// sockets/rooms.ts
export async function joinUserRooms(socket: Socket) {
  const user = socket.data.user;

  // Personal room for targeted notifications
  socket.join(`user:${user.id}`);

  // Role room for broadcast to all guards, all wardens, etc.
  socket.join(`role:${user.role}`);

  // Gate rooms for security guards
  if (user.role === 'SECURITY_GUARD' && user.assignedGateId) {
    socket.join(`gate:${user.assignedGateId}`);
  }

  // Campus-wide room for security/admin roles
  if (['SECURITY_ADMIN', 'UNIVERSITY_ADMIN', 'SUPER_ADMIN'].includes(user.role)) {
    socket.join('campus:all');
  }
}
```


14.5 Event Definitions

Notification Namespace Events

Event Name	Direction	Payload	Recipients
<code>pass:approved</code>	Server → Client	<code>{passId, passNumber, message}</code>	<code>user:</code> <code>{requesterId}</code>
<code>pass:rejected</code>	Server → Client	<code>{passId, reason}</code>	<code>user:</code> <code>{requesterId}</code>
<code>pass:expiry_warning</code>	Server → Client	<code>{passId, expiresIn}</code>	<code>user:</code> <code>{requesterId}</code>
<code>visitor:entered</code>	Server → Client	<code>{passId, visitorName, gate, time}</code>	<code>user:</code> <code>{requesterId}</code>
<code>visitor:exited</code>	Server → Client	<code>{passId, visitorName, dwellMinutes}</code>	<code>user:</code> <code>{requesterId}</code>
<code>approval:pending</code>	Server → Client	<code>{passId, requesterName, passType}</code>	<code>user:</code> <code>{approverId}</code>
<code>notification:read</code>	Client → Server	<code>{notificationId}</code>	Echo to other sessions

Security Namespace Events

Event Name	Direction	Payload	Recipients
<code>lockdown:initiated</code>	Server → Client	<code>{reason, scope, initiatedBy, time}</code>	<code>campus:all</code>
<code>lockdown:lifted</code>	Server → Client	<code>{liftedBy, time}</code>	<code>campus:all</code>
<code>alert:suspicious_activity</code>	Server → Client	<code>{gate, description, severity}</code>	<code>role:SECURITY_ADMIN</code>
<code>alert:overstay</code>	Server → Client	<code>{passId, visitorName, minutesOverstayed}</code>	<code>role:SECURITY_ADMIN</code>
<code>pass:revoked</code>	Server → Client	<code>{passId, gate}</code>	<code>campus:all, gate:</code> <code>{gateId}</code>

Gates Namespace Events

Event Name	Direction	Payload	Recipients
<code>entry:logged</code>	Server → Client	<code>{passId, visitorName, photo}</code>	<code>gate:{gateId} room</code>
<code>gate:status</code>	Server → Client	<code>{gateId, status, queueCount}</code>	<code>campus:all</code>

Event Name	Direction	Payload	Recipients
qr:invalid	Server → Client	{reason, attemptCount}	gate:{gateId}

14.6 Emitting Events from API Services

```
// services/notification.service.ts
export class NotificationService {
  constructor(private io: Server) {}

  async notifyPassApproved(pass: Pass) {
    // Persist notification to DB
    await notificationRepository.create({
      userId: pass.requesterId,
      passId: pass.id,
      title: 'Pass Approved',
      body: `Your pass ${pass.passNumber} has been approved.`,
      type: 'PASS_APPROVED',
    });

    // Real-time push to requester's personal room
    this.io
      .of('/notifications')
      .to(`user:${pass.requesterId}`)
      .emit('pass:approved', {
        passId: pass.id,
        passNumber: pass.passNumber,
        message: 'Your visitor pass has been approved. QR code sent to email.',
      });
  }

  async broadcastLockdown(lockdown: LockdownEvent) {
    this.io
      .of('/security')
      .to('campus:all')
      .emit('lockdown:initiated', {
        reason: lockdown.reason,
        scope: lockdown.scope,
        initiatedBy: lockdown.initiatedByName,
        time: new Date().toISOString(),
      });

    // Also push to gates namespace so terminals show lockdown screen
    this.io
      .of('/gates')
      .emit('lockdown:initiated', { reason: lockdown.reason });
  }
}
```

14.7 Client-Side Socket Usage

```
// hooks/useSocket.ts
export const useNotificationSocket = () => {
  const { accessToken, user } = useAuth();
  const dispatch = useDispatch();

  useEffect(() => {
    const socket = io(`${import.meta.env.VITE_WS_URL}/notifications`, {
      auth: { token: accessToken },
      reconnectionAttempts: 5,
      reconnectionDelay: 2000,
    });

    socket.on('connect', () => console.log('Socket connected'));

    socket.on('pass:approved', (data) => {
      dispatch(addNotification({
        type: 'success',
        title: 'Pass Approved!',
        body: data.message,
      }));
      queryClient.invalidateQueries({ queryKey: ['passes'] });
    });

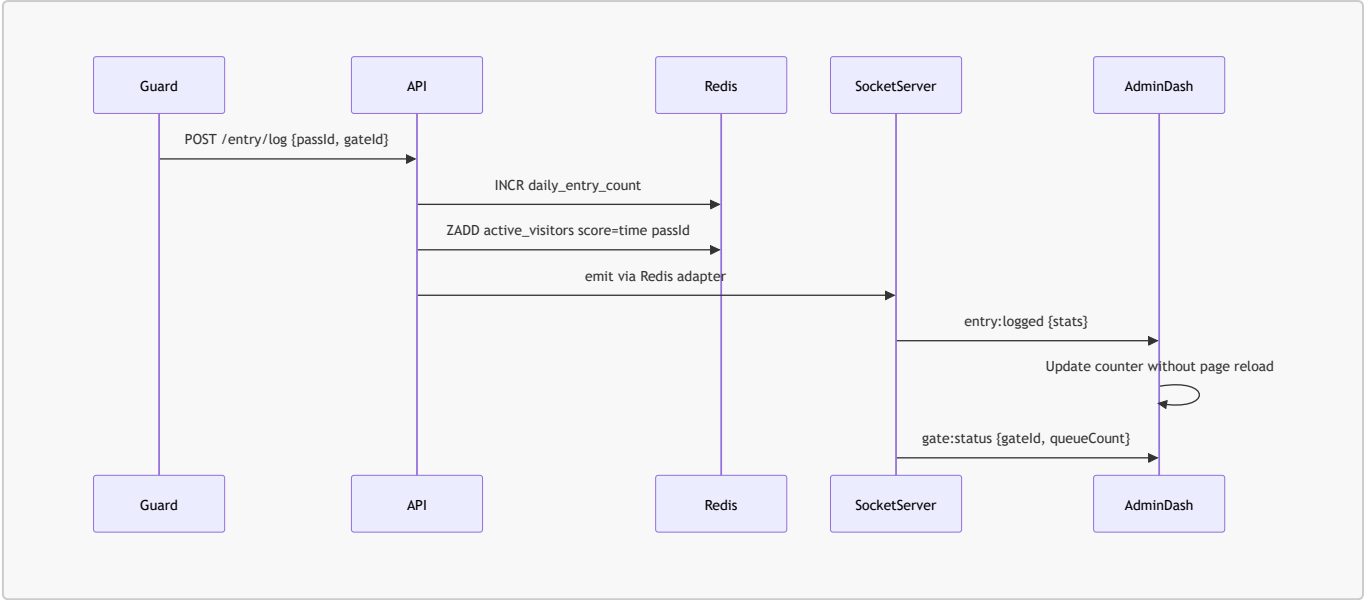
    socket.on('visitor:entered', (data) => {
      dispatch(addNotification({
        type: 'info',
        title: `${data.visitorName} has entered`,
        body: `Gate: ${data.gate} at ${formatTime(data.time)}`,
      }));
    });

    socket.on('lockdown:initiated', (data) => {
      dispatch(setLockdown({ active: true, reason: data.reason }));
      // Show full-screen lockdown banner
    });

    socket.on('disconnect', () => {
      // Queue notifications to fetch on reconnect
    });

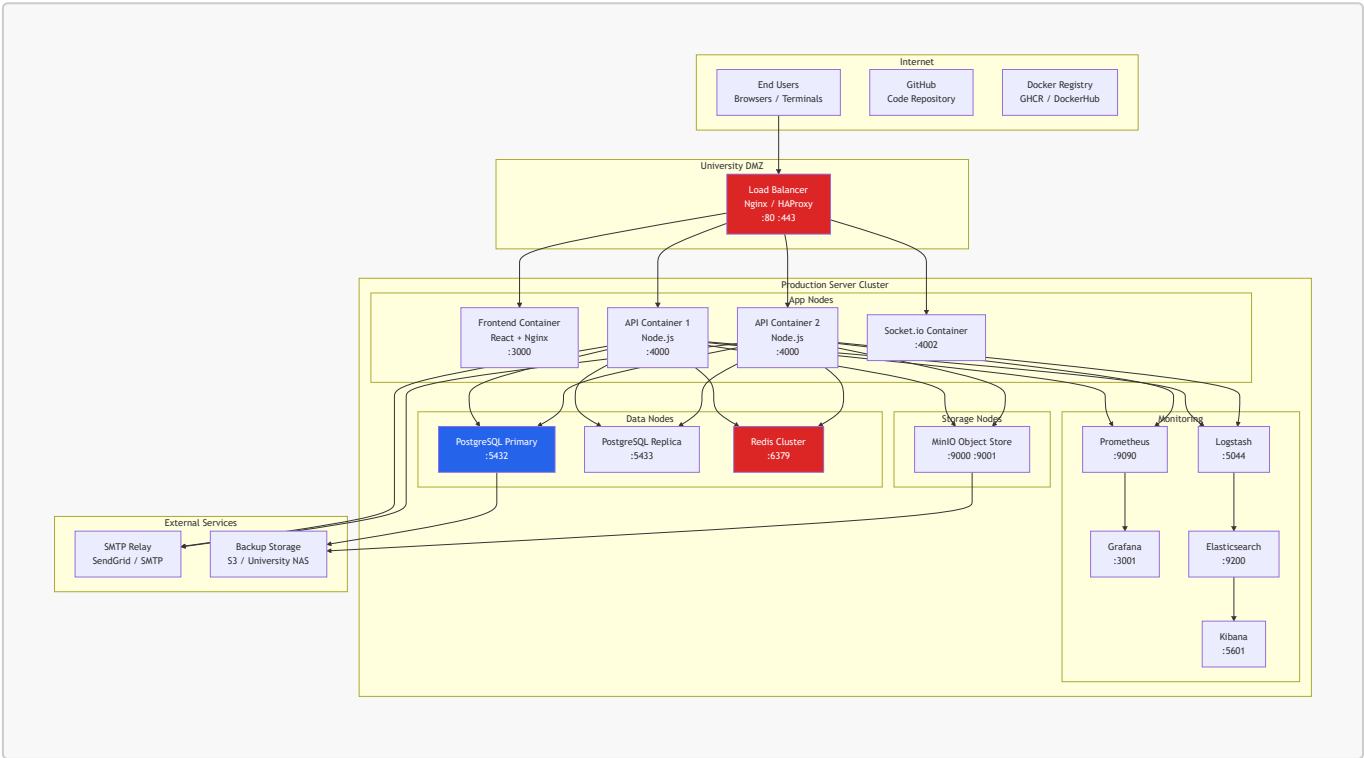
    return () => socket.disconnect();
  }, [accessToken]);
};
```

14.8 Real-Time Dashboard Updates



Section 15, 16 & 17 — Deployment, Docker & CI/CD

15.1 Deployment Architecture Diagram



16.1 Docker Compose Configuration

```
# docker-compose.yml
version: '3.9'

networks:
```

```
gatepass_net:
  driver: bridge

volumes:
  postgres_data:
  redis_data:
  minio_data:
  prometheus_data:
  grafana_data:
  elasticsearch_data:

services:
# — Frontend —————
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  args:
    - VITE_API_URL=${VITE_API_URL}
    - VITE_WS_URL=${VITE_WS_URL}
  container_name: gatepass_frontend
  restart: unless-stopped
  networks:
    - gatepass_net
  expose:
    - "3000"

# — Backend API —————
api:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: gatepass_api
  restart: unless-stopped
  env_file: .env.production
  environment:
    DATABASE_URL: postgresql://gatepass:${DB_PASSWORD}@db:5432/gatepass
    REDIS_URL: redis://redis:6379
    MINIO_ENDPOINT: minio
    MINIO_PORT: 9000
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_healthy
  networks:
    - gatepass_net
  expose:
    - "4000"
  deploy:
    replicas: 2
    resources:
      limits:
        memory: 512M
```

```
    cpus: '0.5'
```

```
# — Socket.io —
socketio:
  build:
    context: ./backend
    dockerfile: Dockerfile.socket
  container_name: gatepass_socket
  restart: unless-stopped
  env_file: .env.production
  environment:
    REDIS_URL: redis://redis:6379
  depends_on:
    - redis
  networks:
    - gatepass_net
  expose:
    - "4002"
```

```
# — PostgreSQL —
db:
  image: postgres:16-alpine
  container_name: gatepass_db
  restart: unless-stopped
  environment:
    POSTGRES_DB: gatepass
    POSTGRES_USER: gatepass
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./scripts/init-db.sql:/docker-entrypoint-initdb.d/init.sql
  networks:
    - gatepass_net
  expose:
    - "5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U gatepass -d gatepass"]
    interval: 10s
    timeout: 5s
    retries: 5
  shm_size: 256m
```

```
# — Redis —
redis:
  image: redis:7-alpine
  container_name: gatepass_redis
  restart: unless-stopped
  command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD}
  volumes:
    - redis_data:/data
  networks:
    - gatepass_net
  expose:
    - "6379"
```

```
healthcheck:
  test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
  interval: 10s
  retries: 3
```

— MinIO

```
minio:
  image: minio/minio:latest
  container_name: gatepass_minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_ACCESS_KEY}
    MINIO_ROOT_PASSWORD: ${MINIO_SECRET_KEY}
  volumes:
    - minio_data:/data
  networks:
    - gatepass_net
  expose:
    - "9000"
    - "9001"
```

— Nginx

```
nginx:
  image: nginx:1.25-alpine
  container_name: gatepass_nginx
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    - ./nginx/conf.d:/etc/nginx/conf.d:ro
    - ./certs:/etc/nginx/certs:ro
  depends_on:
    - frontend
    - api
    - socketio
  networks:
    - gatepass_net
```

— Prometheus

```
prometheus:
  image: prom/prometheus:latest
  container_name: gatepass_prometheus
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prometheus_data:/prometheus
  networks:
    - gatepass_net
  expose:
    - "9090"
```

— Grafana

```

grafana:
  image: grafana/grafana:latest
  container_name: gatepass_grafana
  environment:
    GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD}
  volumes:
    - grafana_data:/var/lib/grafana
    - ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards:ro
  networks:
    - gatepass_net
  expose:
    - "3001"

# — Elasticsearch —————
elasticsearch:
  image: elasticsearch:8.11.0
  container_name: gatepass_elasticsearch
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
  volumes:
    - elasticsearch_data:/usr/share/elasticsearch/data
  networks:
    - gatepass_net
  expose:
    - "9200"

```

16.2 Dockerfiles

Frontend Dockerfile

```

# frontend/Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --frozen-lockfile
COPY . .
ARG VITE_API_URL
ARG VITE_WS_URL
ENV VITE_API_URL=$VITE_API_URL
ENV VITE_WS_URL=$VITE_WS_URL
RUN npm run build

FROM nginx:1.25-alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 3000
CMD ["nginx", "-g", "daemon off;"]

```


Backend Dockerfile

```
# backend/Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --frozen-lockfile
COPY . .
RUN npx prisma generate
RUN npm run build

FROM node:20-alpine AS runner
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/prisma ./prisma
COPY package*.json ./
USER appuser
EXPOSE 4000
CMD ["node", "dist/server.js"]
```

16.3 Nginx Configuration

```
# nginx/conf.d/gatepass.conf

upstream api_backend {
    least_conn;
    server api:4000;
    keepalive 32;
}

upstream socket_backend {
    server socketio:4002;
}

server {
    listen 80;
    server_name university.edu www.university.edu;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name university.edu www.university.edu;

    ssl_certificate      /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key  /etc/nginx/certs/privkey.pem;
    ssl_protocols        TLSv1.2 TLSv1.3;
```

```
ssl_ciphers          HIGH:!aNULL:!MD5;
ssl_session_cache    shared:SSL:10m;

add_header Strict-Transport-Security "max-age=31536000; includeSubDomains;
preload";
add_header X-Frame-Options DENY;
add_header X-Content-Type-Options nosniff;

client_max_body_size 10M;

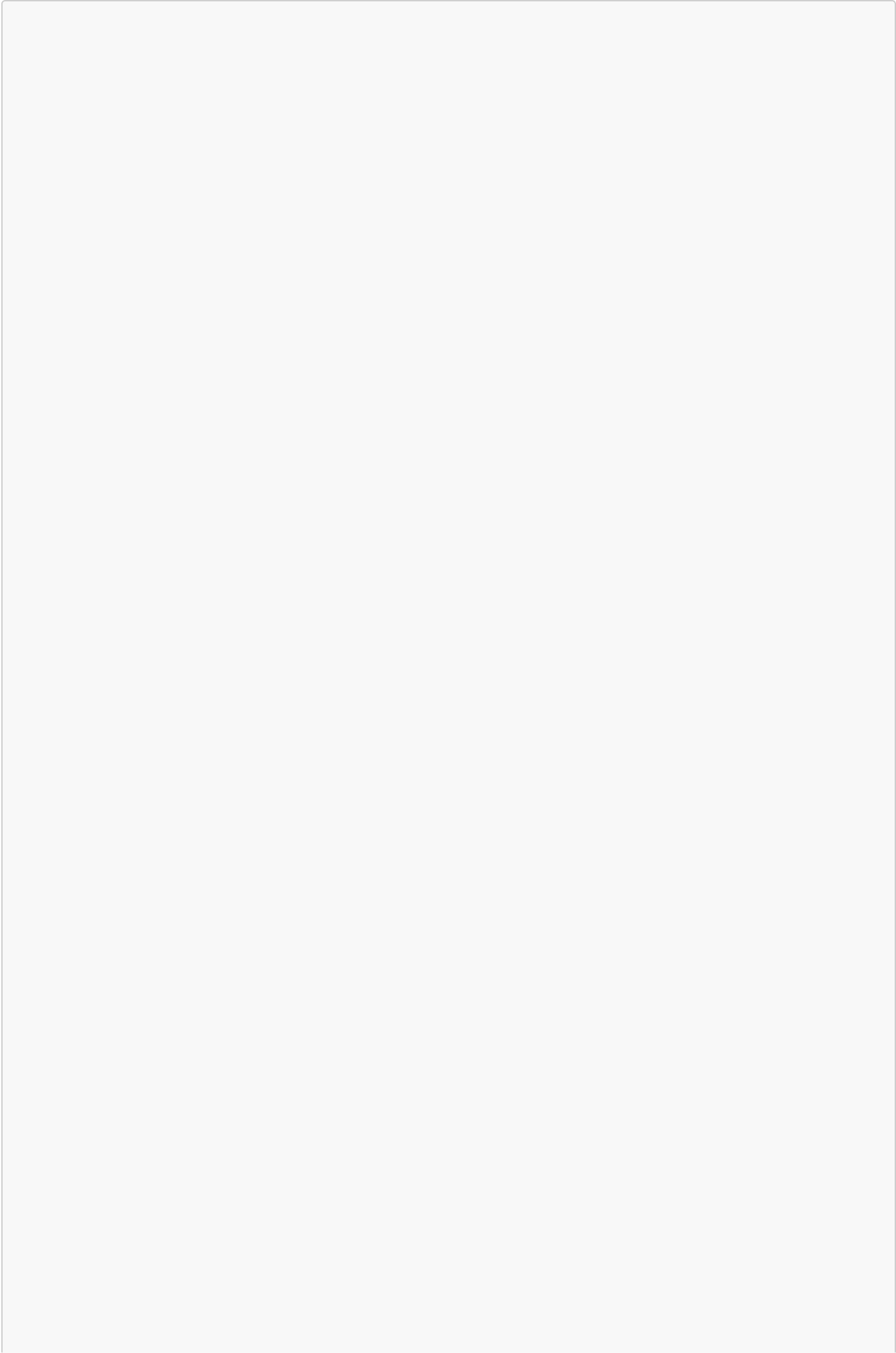
# Frontend SPA
location / {
    proxy_pass http://frontend:3000;
    proxy_cache_use_stale error timeout;
    gzip on;
    gzip_types text/plain application/javascript text/css application/json;
}

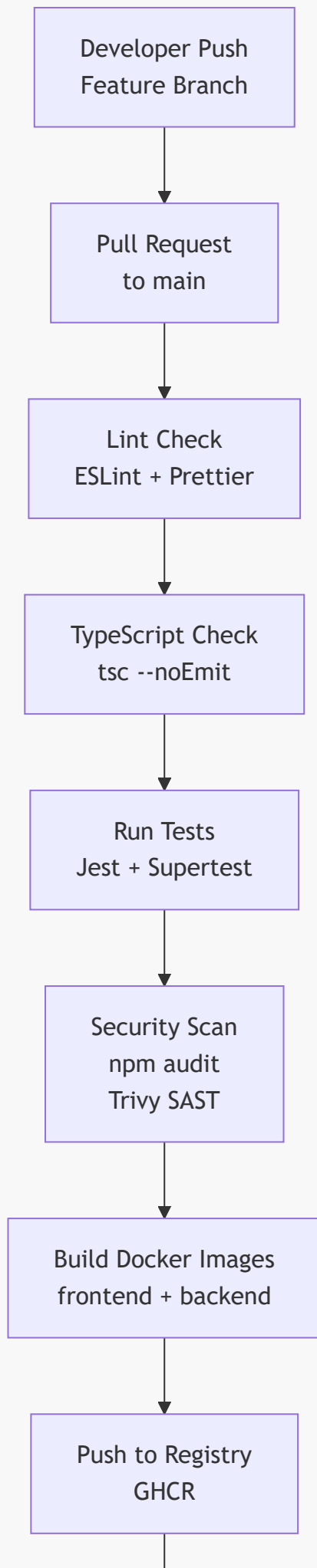
# API
location /api/ {
    proxy_pass http://api_backend;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

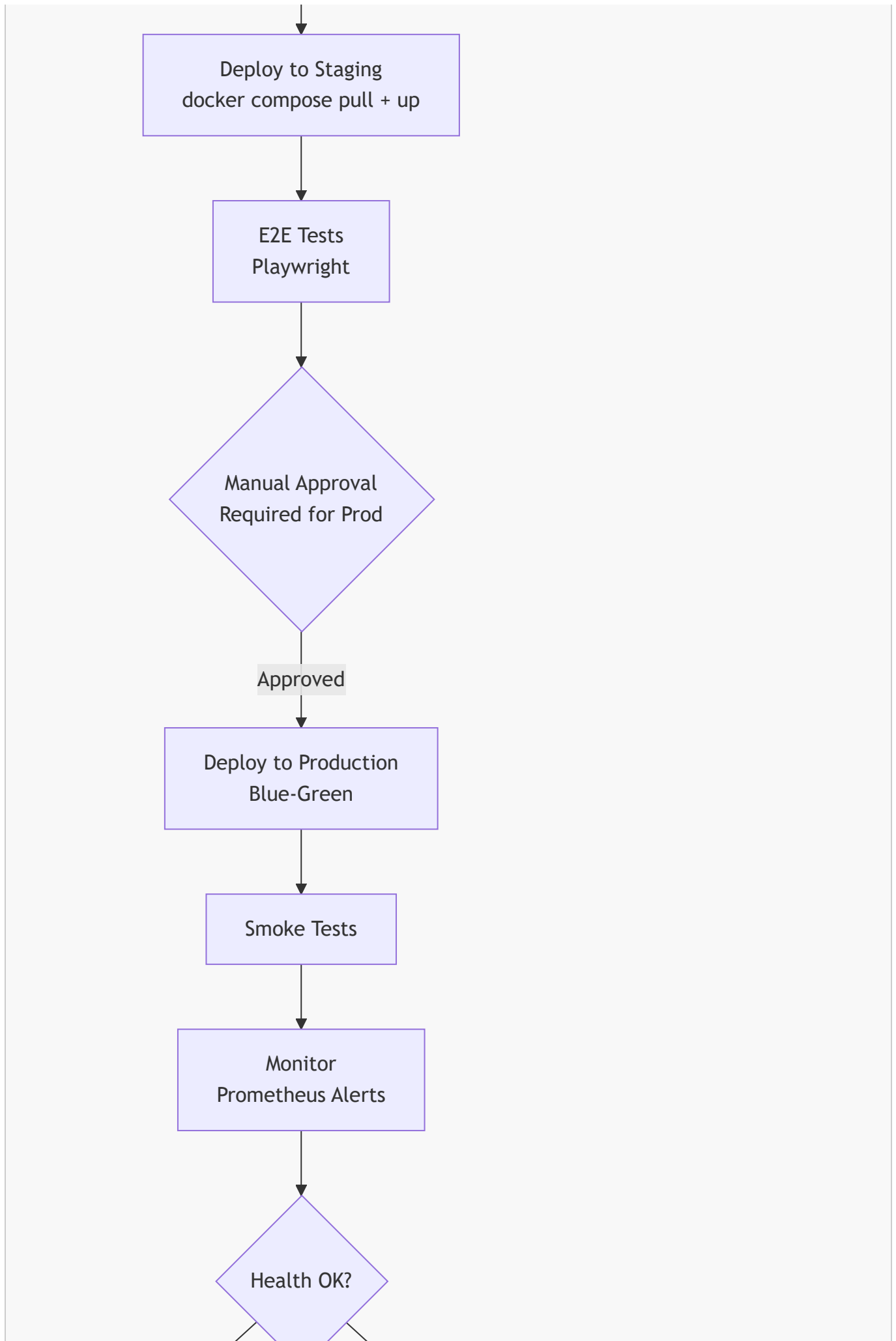
    limit_req zone=api_limit burst=20 nodelay;
}

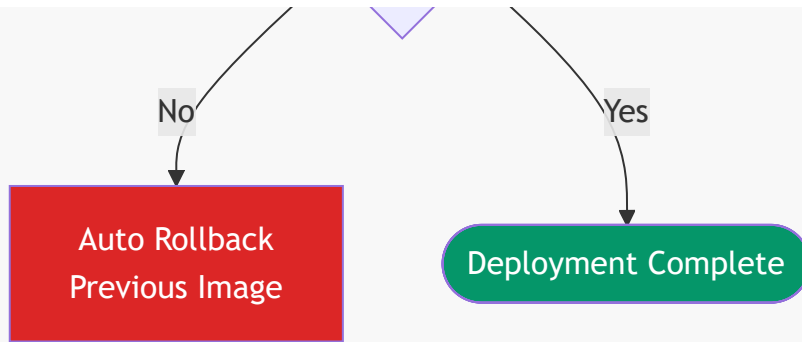
# WebSocket
location /socket.io/ {
    proxy_pass http://socket_backend;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_read_timeout 86400;
}
}
```

17.1 CI/CD Pipeline









17.2 GitHub Actions Workflow

```
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME_API: ${ github.repository }}/api
  IMAGE_NAME_FE: ${ github.repository }}/frontend

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: testpass
          POSTGRES_DB: gatepass_test
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
      redis:
        image: redis:7
        options: --health-cmd "redis-cli ping" --health-interval 10s

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
```

```
node-version: '20'
cache: 'npm'
cache-dependency-path: |
  backend/package-lock.json
  frontend/package-lock.json

- name: Install backend dependencies
  run: npm ci
  working-directory: ./backend

- name: Install frontend dependencies
  run: npm ci
  working-directory: ./frontend

- name: Lint backend
  run: npm run lint
  working-directory: ./backend

- name: TypeScript check backend
  run: npm run type-check
  working-directory: ./backend

- name: Lint frontend
  run: npm run lint
  working-directory: ./frontend

- name: TypeScript check frontend
  run: npm run type-check
  working-directory: ./frontend

- name: Run backend tests
  run: npm run test:ci
  working-directory: ./backend
  env:
    DATABASE_URL:
      postgresql://postgres:testpass@localhost:5432/gatepass_test
    REDIS_URL: redis://localhost:6379
    JWT_SECRET: test-secret-key-32-chars-minimum
    QR_ENCRYPTION_KEY: ${ secrets.QR_TEST_KEY }

- name: Security audit
  run: npm audit --audit-level=moderate
  working-directory: ./backend

build-and-push:
  needs: lint-and-test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  permissions:
    contents: read
    packages: write

  steps:
    - uses: actions/checkout@v4
```

```

- name: Log in to registry
  uses: docker/login-action@v3
  with:
    registry: ${ env.REGISTRY }
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- name: Build and push API image
  uses: docker/build-push-action@v5
  with:
    context: ./backend
    push: true
    tags: ${ env.REGISTRY }/${ env.IMAGE_NAME_API }:${ github.sha
}},${ env.REGISTRY }/${ env.IMAGE_NAME_API }:latest

- name: Build and push Frontend image
  uses: docker/build-push-action@v5
  with:
    context: ./frontend
    push: true
    build-args: |
      VITE_API_URL=${ vars.VITE_API_URL }
      VITE_WS_URL=${ vars.VITE_WS_URL }
    tags: ${ env.REGISTRY }/${ env.IMAGE_NAME_FE }:${ github.sha }},${
env.REGISTRY }/${ env.IMAGE_NAME_FE }:latest

- name: Run Trivy vulnerability scan
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ env.REGISTRY }/${ env.IMAGE_NAME_API }:${ github.sha
}}

    format: sarif
    output: trivy-results.sarif
    severity: HIGH,CRITICAL

deploy-staging:
  needs: build-and-push
  runs-on: ubuntu-latest
  environment: staging

  steps:
    - name: Deploy to staging
      uses: appleboy/ssh-action@master
      with:
        host: ${ secrets.STAGING_HOST }
        username: deploy
        key: ${ secrets.STAGING_SSH_KEY }
        script: |
          cd /opt/gatepass
          docker compose pull
          docker compose up -d --remove-orphans
          docker compose exec api npx prisma migrate deploy

```



```

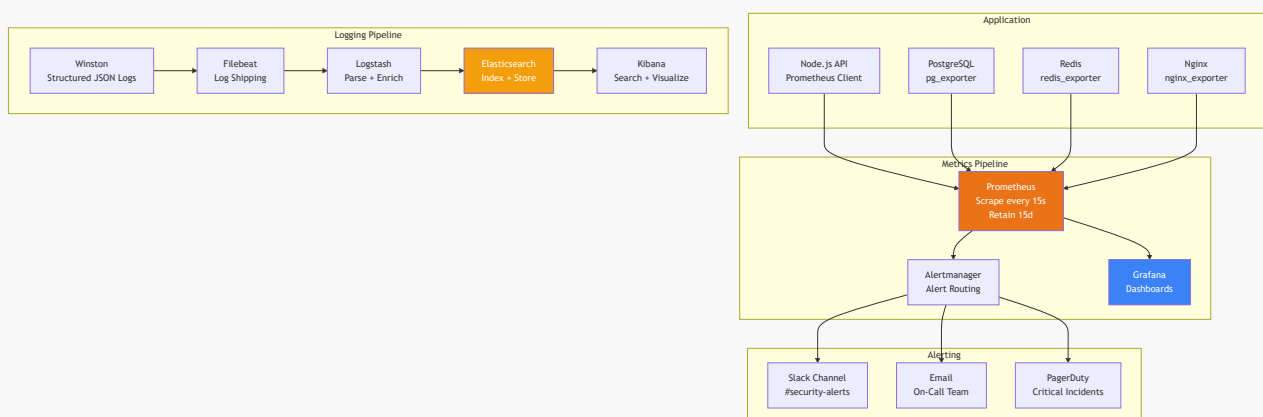
deploy-production:
  needs: deploy-staging
  runs-on: ubuntu-latest
  environment: production # Requires manual approval in GitHub Environments

  steps:
    - name: Deploy to production (Blue-Green)
      uses: appleboy/ssh-action@master
      with:
        host: ${ secrets.PROD_HOST }
        username: deploy
        key: ${ secrets.PROD_SSH_KEY }
        script: |
          cd /opt/gatepass
          # Pull new images
          docker compose pull
          # Run DB migrations
          docker compose run --rm api npx prisma migrate deploy
          # Swap: bring up new, then remove old
          docker compose up -d --remove-orphans --scale api=4
          sleep 15
          # Health check
          curl -f https://university.edu/api/v1/health || (docker compose
rollback && exit 1)
          docker compose up -d --scale api=2

```

Section 18, 19 & 20 — Monitoring, Scalability & Disaster Recovery

18.1 Monitoring Architecture



18.2 Prometheus Metrics

Custom Application Metrics

```
// utils/metrics.ts
import { Counter, Histogram, Gauge, register } from 'prom-client';

export const httpRequestDuration = new Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP request duration in seconds',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.05, 0.1, 0.3, 0.5, 1, 2, 5],
});

export const passesCreated = new Counter({
  name: 'gatepass_passes_created_total',
  help: 'Total passes created',
  labelNames: ['pass_type'],
});

export const qrVerifications = new Counter({
  name: 'gatepass_qr_verifications_total',
  help: 'Total QR verifications',
  labelNames: ['result', 'gate'],
});

export const activeVisitors = new Gauge({
  name: 'gatepass_active_visitors',
  help: 'Visitors currently on campus',
});

export const pendingApprovals = new Gauge({
  name: 'gatepass_pending_approvals',
  help: 'Passes awaiting approval',
  labelNames: ['pass_type'],
});

export const qrVerificationDuration = new Histogram({
  name: 'gatepass_qr_verification_duration_ms',
  help: 'QR verification time in milliseconds',
  buckets: [50, 100, 200, 300, 500, 1000],
});
```

18.3 Prometheus Alert Rules

```
# monitoring/prometheus-rules.yml
groups:
- name: gatepass_alerts
  rules:
  - alert: HighQRVerificationLatency
    expr: histogram_quantile(0.95, gatepass_qr_verification_duration_ms) > 500
```

```
for: 2m
labels:
  severity: warning
annotations:
  summary: "QR verification p95 latency > 500ms"

- alert: APIErrorRateHigh
  expr: rate(http_request_duration_seconds_count{status_code=~"5.."}[5m]) > 0.05
  for: 1m
  labels:
    severity: critical
  annotations:
    summary: "API error rate above 5%"

- alert: DatabaseConnectionsHigh
  expr: pg_stat_database_numbackends / pg_settings_max_connections > 0.8
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "PostgreSQL connections above 80% of max"

- alert: RedisMemoryHigh
  expr: redis_memory_used_bytes / redis_memory_max_bytes > 0.85
  for: 5m
  labels:
    severity: warning

- alert: LockdownActive
  expr: gatepass_lockdown_active == 1
  for: 0m
  labels:
    severity: critical
  annotations:
    summary: "CAMPUS LOCKDOWN IS ACTIVE"
```

18.4 Grafana Dashboards

Dashboard	Panels
Operations Overview	Active visitors, passes today, pending approvals, error rate, system uptime
Gate Activity	Per-gate entry/exit rate, queue depth, scan latency heatmap
Security Dashboard	Failed QR scans, override rate, lockdown history, alert timeline
Performance	API latency p50/p95/p99, DB query time, Redis hit rate, memory/CPU
Business Analytics	Pass type distribution, peak hours, dwell time distribution, department breakdown

18.5 Structured Logging

```
// utils/logger.ts
import winston from 'winston';

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: {
    service: 'gatepass-api',
    environment: process.env.NODE_ENV,
  },
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({
      filename: '/var/log/gatepass/error.log',
      level: 'error',
    }),
    new winston.transports.File({
      filename: '/var/log/gatepass/combined.log',
    }),
  ],
});
```

Log fields for all API requests:

```
{
  "timestamp": "2025-03-15T09:23:11.445Z",
  "level": "info",
  "service": "gatepass-api",
  "method": "POST",
  "url": "/api/v1/qr/verify",
  "userId": "uuid",
  "role": "SECURITY_GUARD",
  "gate": "MAIN_GATE",
  "responseCode": 200,
  "durationMs": 45,
  "passId": "uuid",
  "allowed": true,
  "requestId": "uuid"
}
```

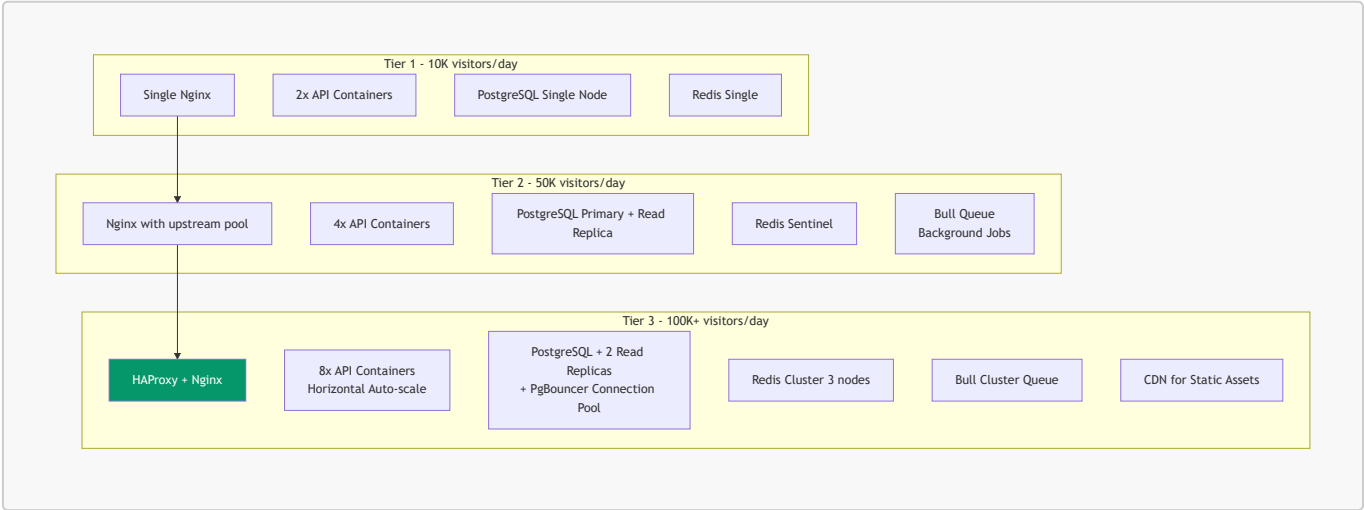
18.6 Health Check Endpoint

```
// GET /api/v1/health
router.get('/health', async (req, res) => {
  const checks = await Promise.allSettled([
    prisma.$queryRaw`SELECT 1`,
    redisClient.ping(),
    minioClient.listBuckets(),
  ]);

  const status = {
    status: checks.every(c => c.status === 'fulfilled') ? 'healthy' : 'degraded',
    timestamp: new Date().toISOString(),
    services: {
      database: checks[0].status === 'fulfilled' ? 'up' : 'down',
      redis: checks[1].status === 'fulfilled' ? 'up' : 'down',
      storage: checks[2].status === 'fulfilled' ? 'up' : 'down',
    },
    uptime: process.uptime(),
    version: process.env.APP_VERSION,
  };

  res.status(status.status === 'healthy' ? 200 : 503).json(status);
});
```

19.1 Scalability Strategy



19.2 Scaling Strategies by Layer

Layer	Strategy	Detail
Frontend	CDN distribution	Serve static React build from CDN (Cloudflare / university edge)
API	Horizontal scale	Add API container replicas; Socket.io uses Redis adapter for event fanout
Database reads	Read replicas	Route SELECT queries to replica; writes go to primary

Layer	Strategy	Detail
Database writes	Connection pooling	PgBouncer in transaction mode; pool size = CPU cores × 2
Cache	Redis Cluster	3-node cluster with hash slots; cache pass data, session data
Background jobs	Bull Queue	Distribute email sending, QR generation across worker processes
File storage	MinIO distributed	4-node MinIO for erasure coding; or migrate to S3

19.3 Caching Strategy

Data	Cache Key	TTL	Invalidation
Pass by ID	<code>pass:{id}</code>	5 min	On status change
User profile	<code>user:{id}</code>	15 min	On profile update
Active visitor count	<code>stats:active_visitors</code>	30 sec	On entry/exit
QR token → pass	<code>qr:{token}</code>	Pass validity	On revoke
Campus lockdown status	<code>campus:lockdown</code>	No TTL	On lift
Pending approvals count	<code>pending:{approverId}</code>	1 min	On approval action

19.4 Database Performance

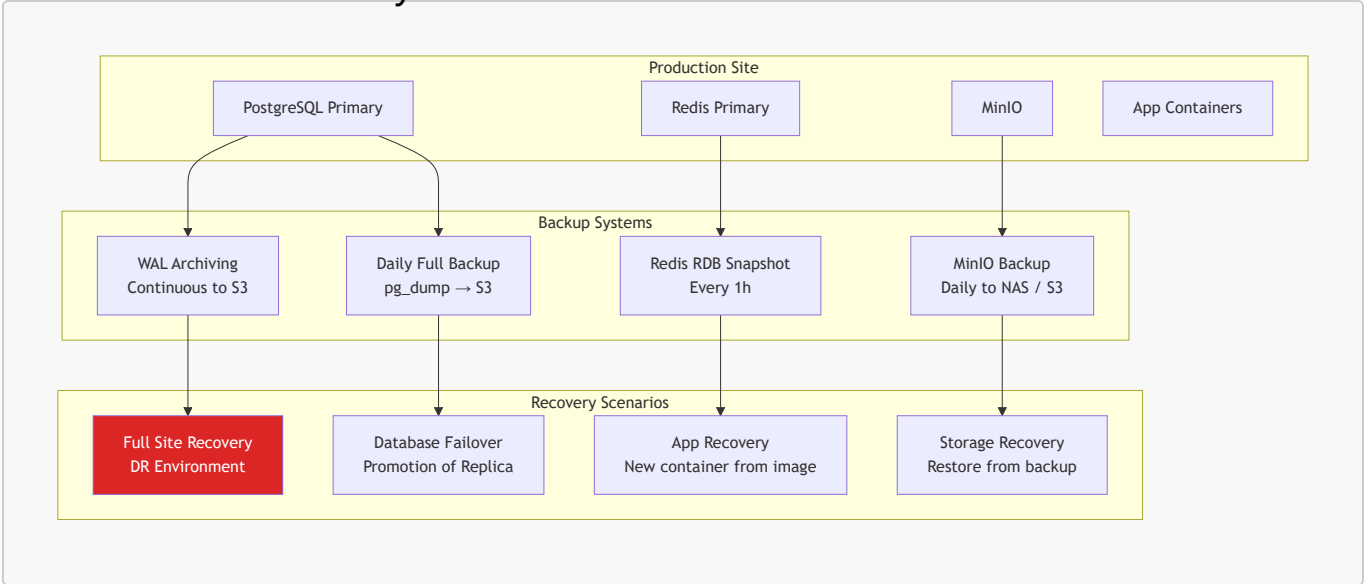
```
-- Composite indexes for hot queries
CREATE INDEX CONCURRENTLY idx_passes_requester_status
  ON passes(requester_id, status) WHERE status IN ('PENDING_APPROVAL', 'ACTIVE');

CREATE INDEX CONCURRENTLY idx_entry_logs_gate_date
  ON entry_logs(gate, entry_at) WHERE exit_at IS NOT NULL;

-- Partial index for active passes only (heavily queried)
CREATE INDEX CONCURRENTLY idx_passes_active
  ON passes(valid_to, allowed_gates)
  WHERE status = 'ACTIVE';

-- PgBouncer config
[gatepass]
host=db
port=5432
dbname=gatepass
pool_mode=transaction
max_client_conn=1000
default_pool_size=50
```

20.1 Disaster Recovery Architecture



20.2 Backup Configuration

```
#!/bin/bash
# scripts/backup.sh - runs daily via cron

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
S3_BUCKET="s3://university-gatepass-backups"

# PostgreSQL full backup
docker exec gatepass_db pg_dump -U gatepass -F c gatepass \
  | gzip | aws s3 cp - "${S3_BUCKET}/postgres/gatepass_${TIMESTAMP}.dump.gz"

# MinIO sync
mc mirror minio/secure-docs "${S3_BUCKET}/minio/${TIMESTAMP}/"

# Redis RDB snapshot
docker exec gatepass_redis redis-cli -a "${REDIS_PASSWORD}" BGSAVE
docker cp gatepass_redis:/data/dump.rdb /tmp/redis_${TIMESTAMP}.rdb
aws s3 cp "/tmp/redis_${TIMESTAMP}.rdb" "${S3_BUCKET}/redis/"

# Verify backup exists
aws s3 ls "${S3_BUCKET}/postgres/gatepass_${TIMESTAMP}.dump.gz" || \
  curl -X POST "${SLACK_WEBHOOK}" -d '{"text":"BACKUP FAILED"}'

echo "Backup completed: ${TIMESTAMP}"
```

20.3 RTO and RPO Targets

Scenario	RPO (Data Loss)	RTO (Recovery Time)	Strategy
API container crash	0	< 30 seconds	Docker restart policy: <code>unless-stopped</code>

Scenario	RPO (Data Loss)	RTO (Recovery Time)	Strategy
API node failure	0	< 2 minutes	Load balancer removes unhealthy node; other replicas take traffic
Redis failure	< 1 hour	< 10 minutes	Redis persistence (AOF) + restore from RDB snapshot
PostgreSQL primary failure	< 1 minute	< 5 minutes	Streaming replication; promote replica; update DNS
MinIO failure	< 24 hours	< 30 minutes	Restore from daily S3 backup
Full site failure	< 24 hours	< 2 hours	Deploy to DR environment from images + latest DB backup
Ransomware / data loss	< 24 hours	< 4 hours	Restore from immutable S3 backups (Object Lock enabled)

20.4 Database Failover Runbook

```
# Step 1: Verify primary is down
pg_isready -h primary-db.internal -U gatepass # should fail

# Step 2: Promote replica to primary
docker exec gatepass_db_replica pg_ctl promote -D /var/lib/postgresql/data

# Step 3: Update application DATABASE_URL to point to replica
docker exec gatepass_api sh -c "export DATABASE_URL=postgresql://gatepass:${PASS}@replica-db:5432/gatepass"

# Step 4: Restart API containers to pick up new connection
docker compose restart api

# Step 5: Notify team and begin root cause analysis
curl -X POST "${SLACK_WEBHOOK}" -d '{"text":"DB FAILOVER COMPLETED – please investigate primary"}'

# Step 6: When primary is restored, reconfigure as new replica
```

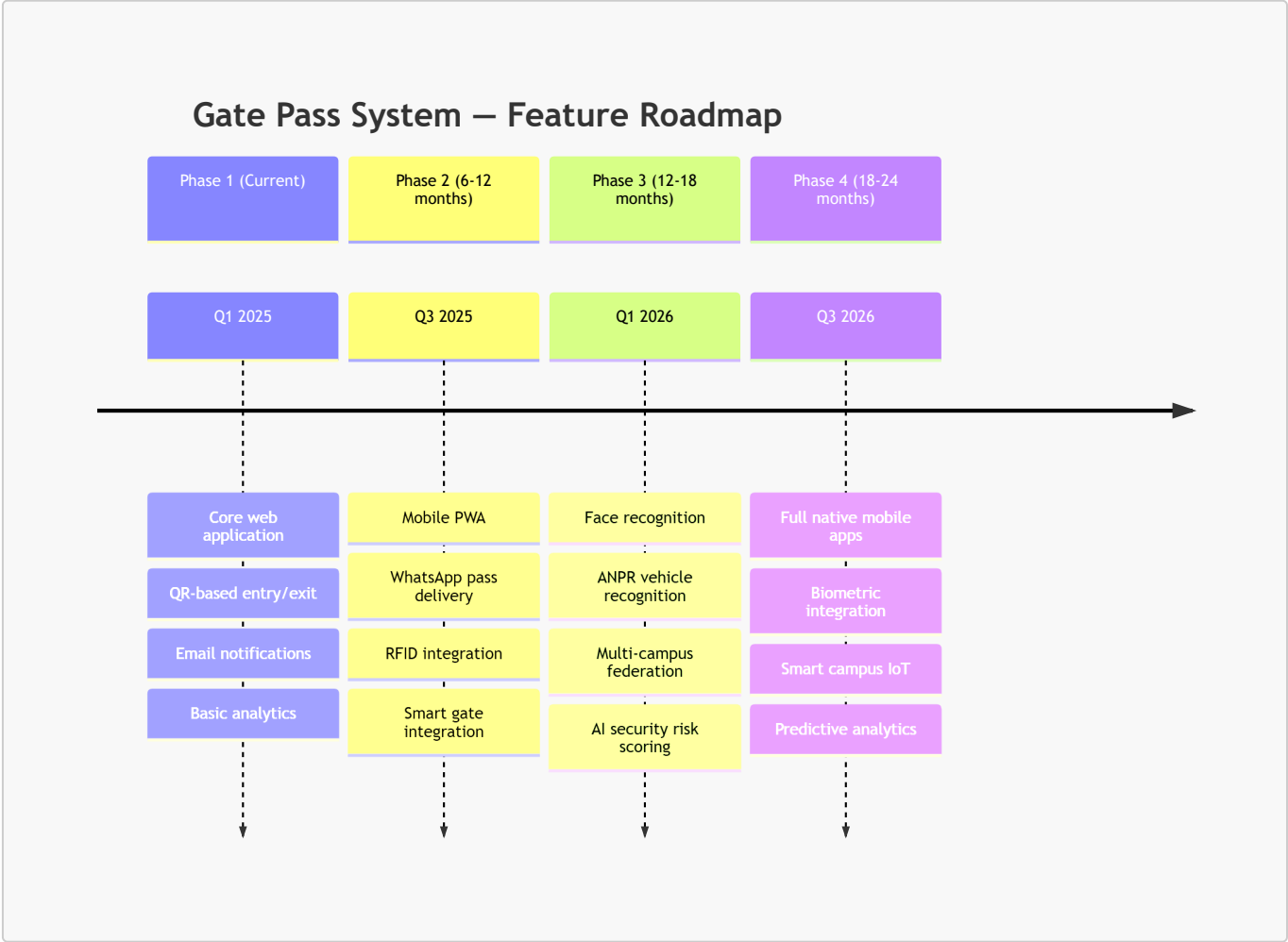
20.5 Recovery Testing Schedule

Test	Frequency	Owner	Procedure
Backup restore test	Monthly	IT Admin	Restore latest backup to staging; verify data integrity
DB failover drill	Quarterly	DBA	Simulate primary failure; measure RTO
Full DR test	Annually	IT + Security	Deploy entire system from scratch to DR environment

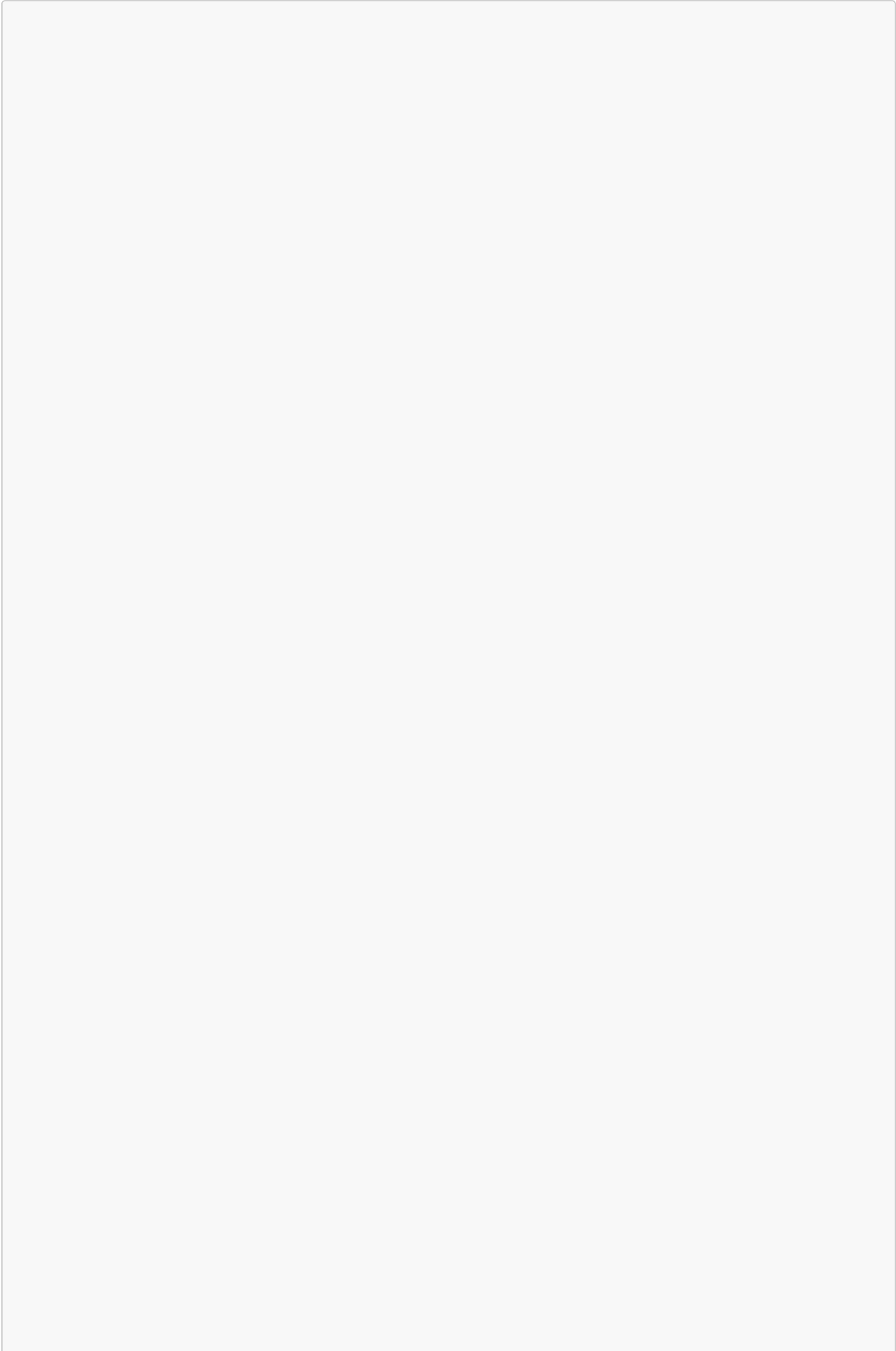
Test	Frequency	Owner	Procedure
Redis recovery test	Monthly	IT Admin	Restore from RDB; verify session invalidation

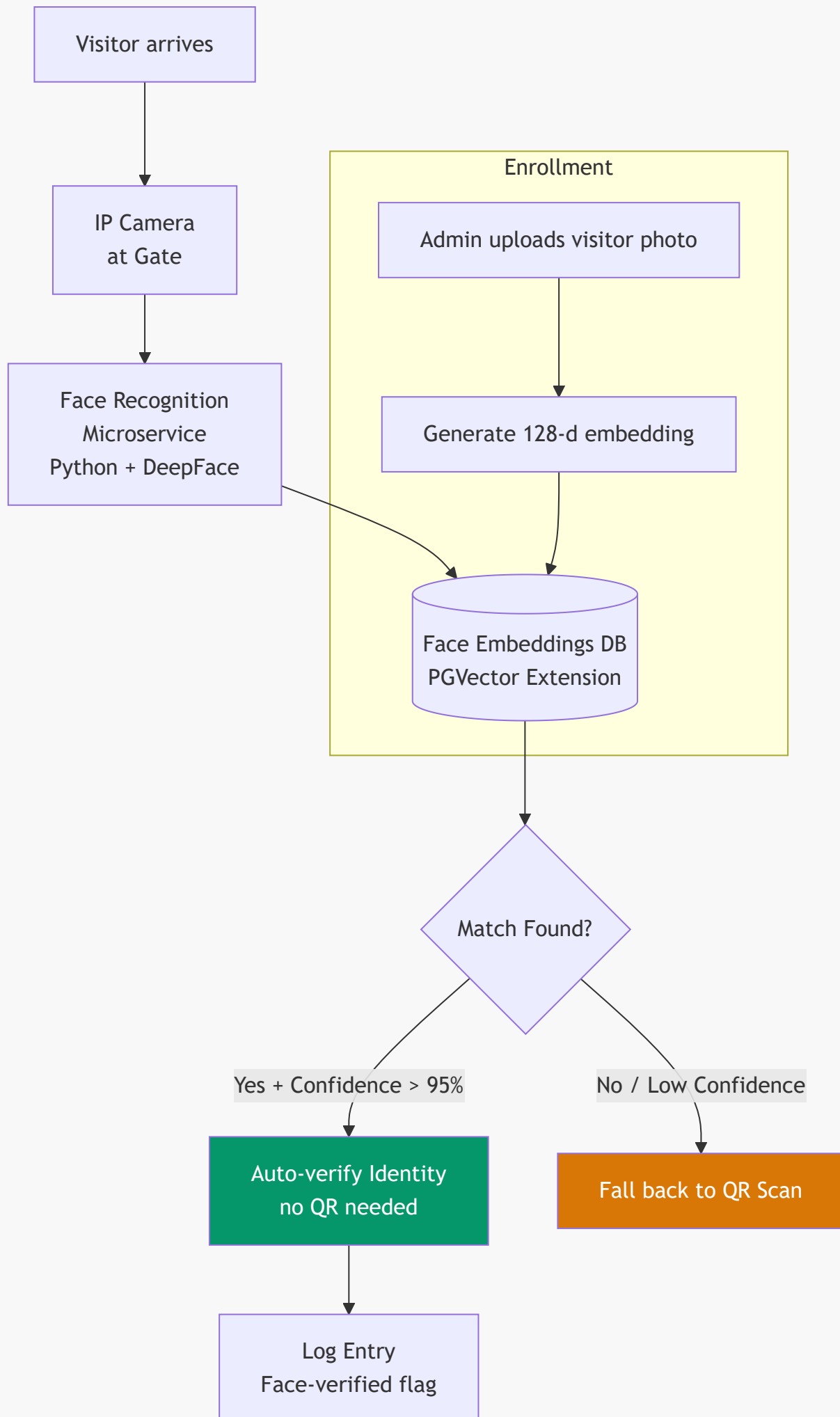
Section 21 — Future Enhancements

21.1 Enhancement Roadmap



21.2 Face Recognition Integration

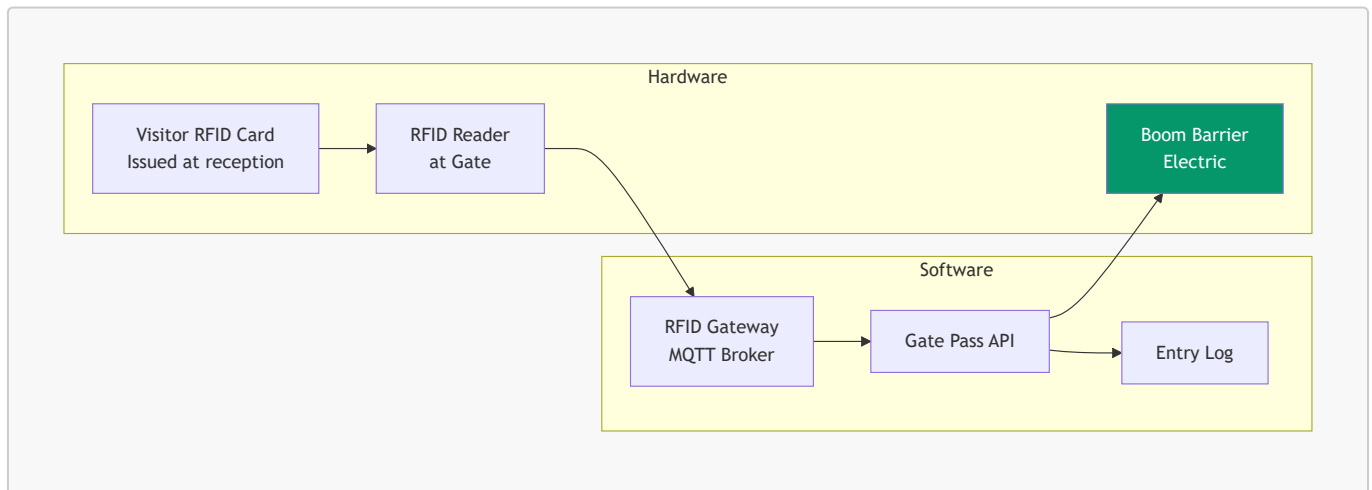




Technology Stack Addition:

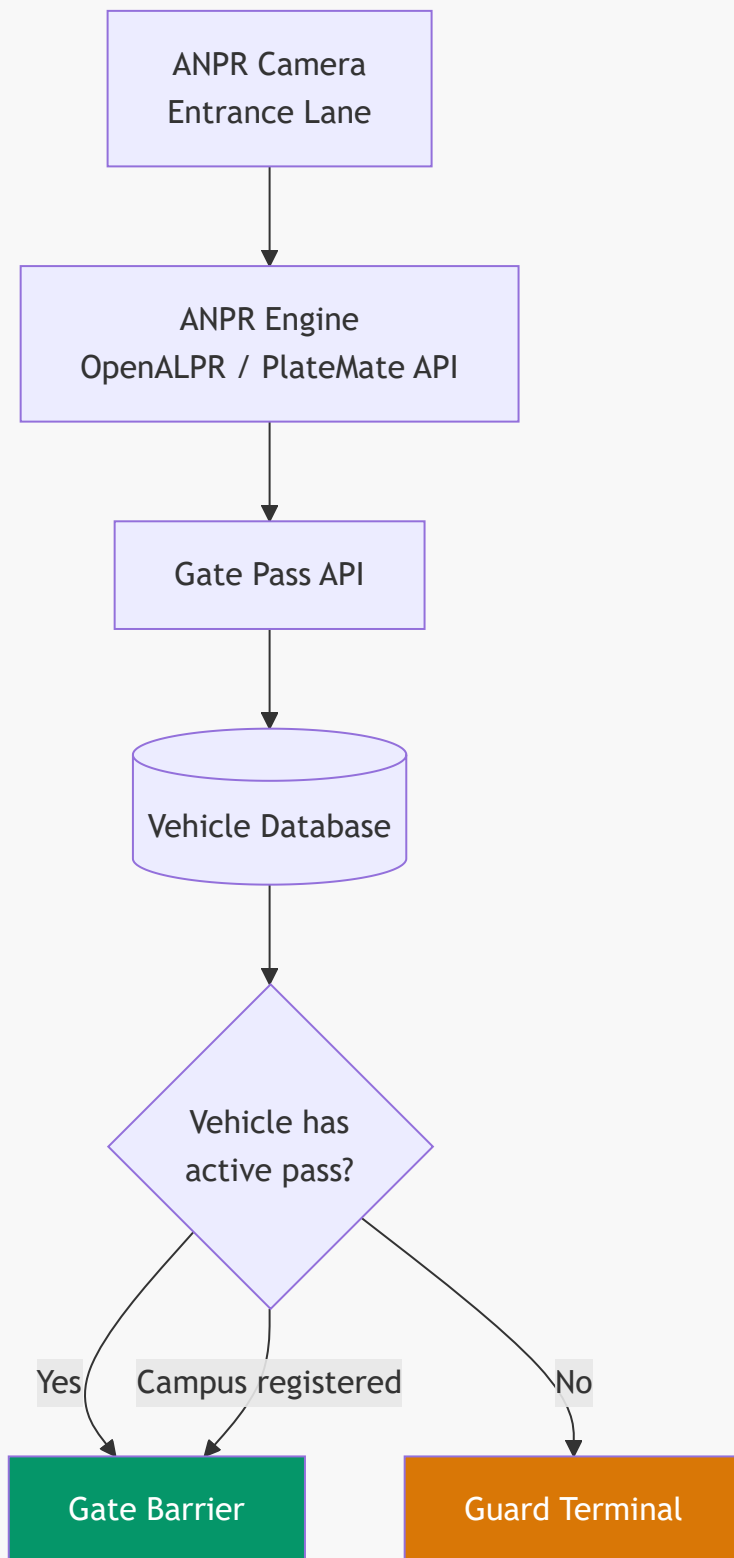
- Python microservice: FastAPI + DeepFace / InsightFace
- pgvector PostgreSQL extension for similarity search
- Liveness detection to prevent photo spoofing
- GDPR/compliance: face data stored separately with explicit consent

21.3 RFID Integration



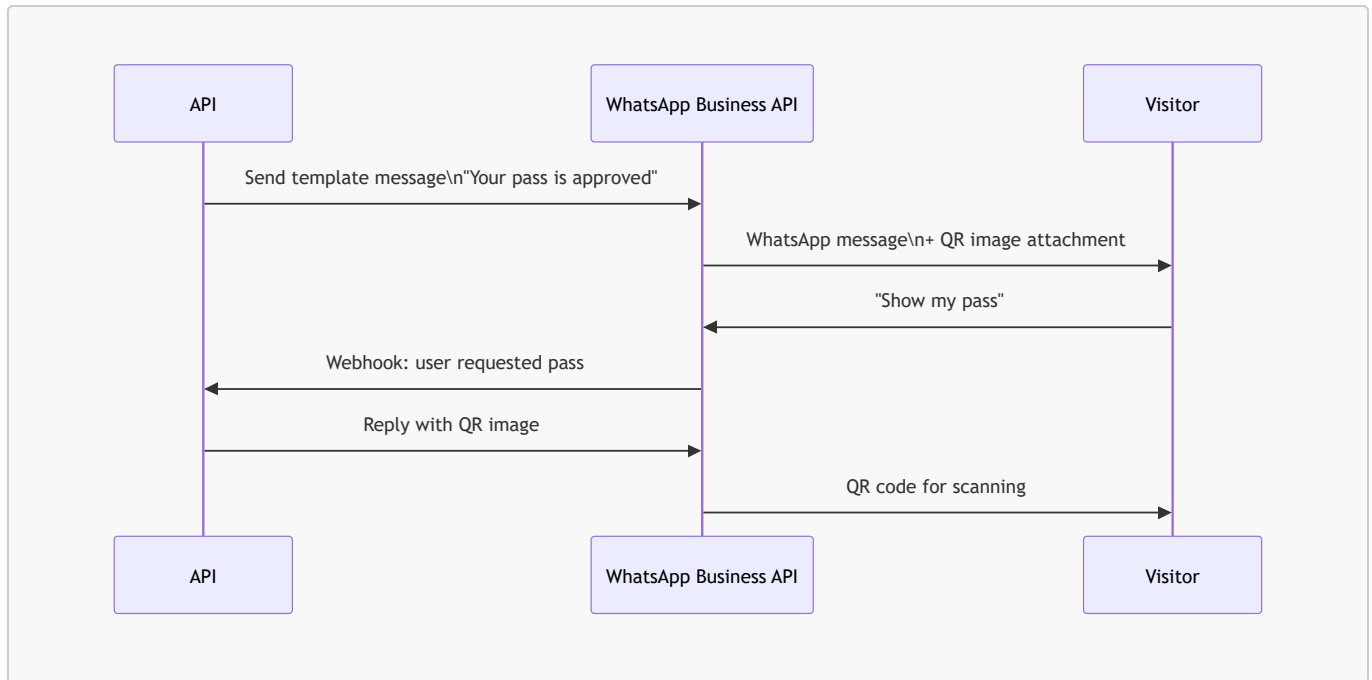
- Visitor receives RFID card on first visit; linked to visitor record
- Card is reprogrammed on each visit with pass-specific token
- Boom barrier opens automatically on valid RFID + active pass
- MQTT protocol between hardware and API gateway

21.4 ANPR (Automatic Number Plate Recognition)



- Camera captures number plate image at entrance
- ANPR API extracts plate text (OpenALPR / commercial API)
- System looks up active vehicle pass for the plate
- Registered campus vehicles auto-open barrier
- Unknown vehicles alert guard terminal

21.5 WhatsApp Pass Delivery



- Twilio / Meta WhatsApp Business Cloud API
- Pre-approved WhatsApp message templates for pass notifications
- Interactive buttons: "Show QR", "Cancel Visit"
- Fallback to email if WhatsApp delivery fails

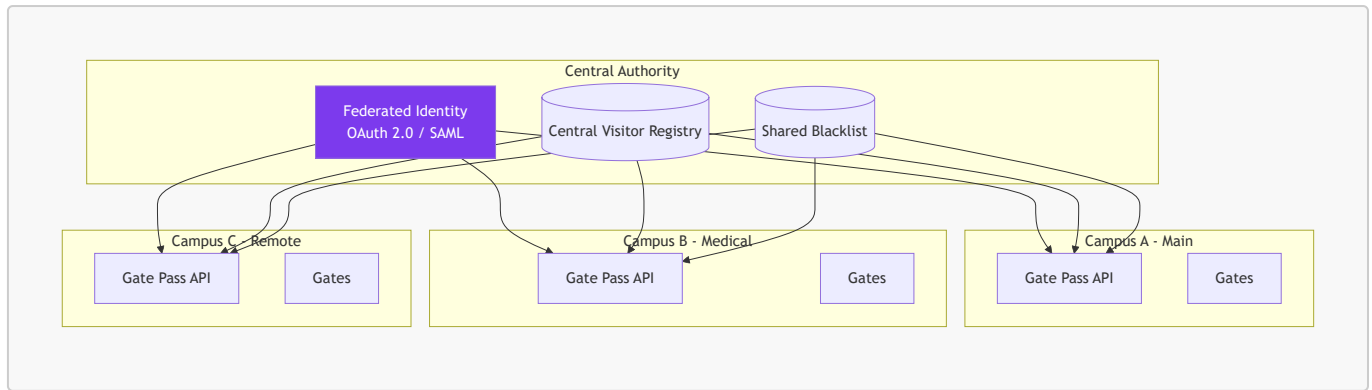
21.6 Mobile App Architecture



Syntax error in text
mermaid version 11.15.0

- React Native (shared codebase iOS + Android)
- Offline QR display: last approved pass cached locally
- Biometric authentication (Face ID / Fingerprint)
- Push notifications via FCM/APNs replacing email for mobile users
- Guard scanner app: optimized camera feed for rapid scanning

21.7 Multi-Campus Federation



- Single visitor registration valid across all campuses
- Shared blacklist propagated via event bus (Kafka / Redis Pub-Sub)
- Each campus retains independent pass approval workflows
- Central analytics aggregating cross-campus data
- SAML / OpenID Connect federation for single sign-on

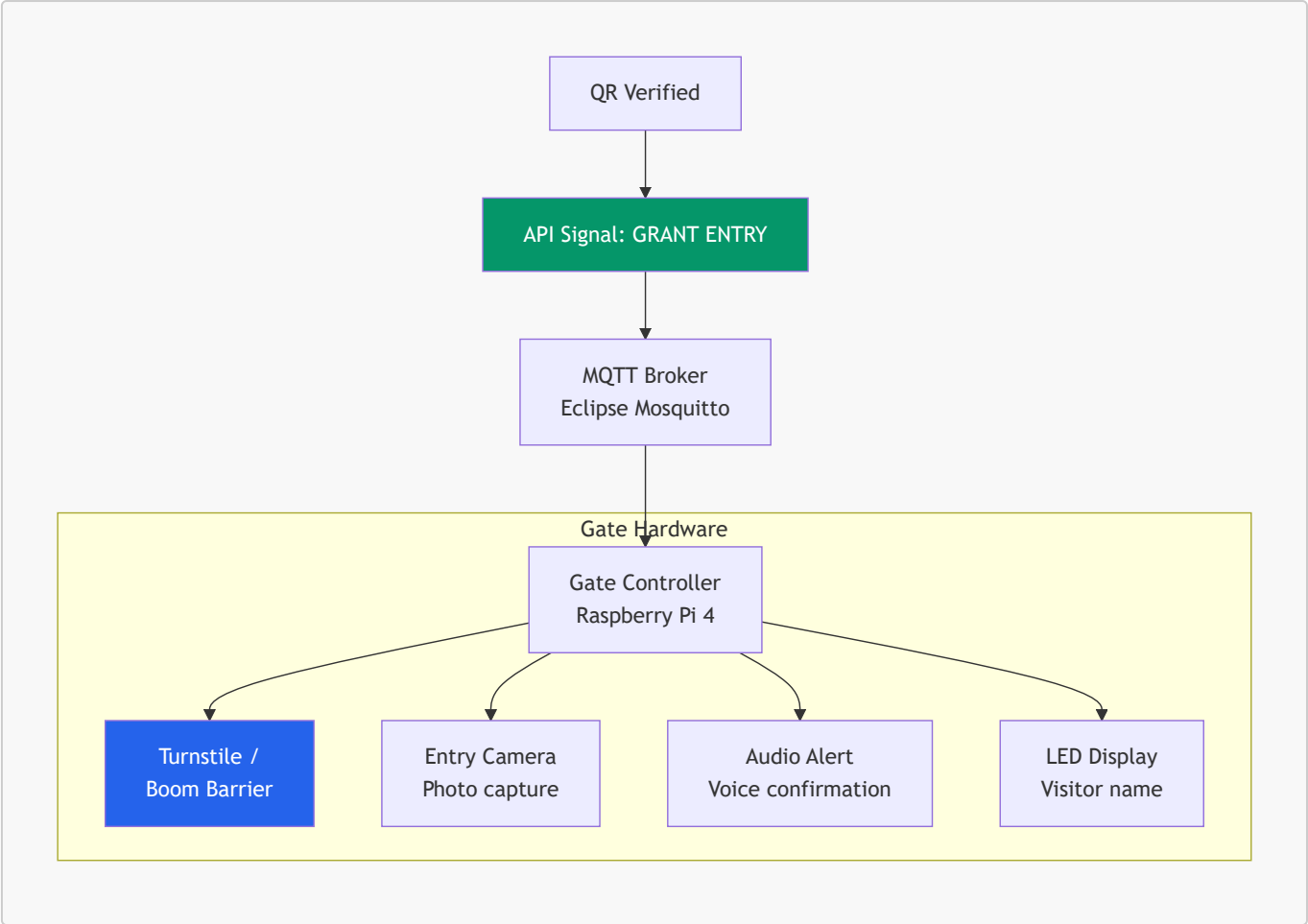
21.8 AI Security Risk Scoring



Syntax error in text
mermaid version 11.15.0

- Python ML microservice: scikit-learn / XGBoost model
- Features: visitor history, time of day, day of week, visit frequency, blacklist proximity, campus event calendar
- Risk score appended to each pass request; approvers see score
- Retrained weekly on ground-truth data from security incidents
- Explainability: top 3 contributing factors shown to approver

21.9 Smart Gate Integration (IoT)



- Gate hardware communicates via MQTT (lightweight IoT protocol)
- API publishes `gate/{gateId}/command OPEN` on valid QR
- Controller actuates turnstile/boom for 5 seconds
- Camera captures entry photo, stored against entry log
- Display shows visitor name; speaker announces "Welcome, [Name]"
- Hardware failsafe: power outage → barrier defaults open (fail-safe) or closed (fail-secure) based on security policy

21.10 Enhancement Impact Matrix

Enhancement	Security Impact	UX Impact	Cost	Effort	Priority
Mobile App	Medium	Very High	Medium	High	High
WhatsApp Delivery	Low	High	Low	Low	High
RFID Cards	High	High	Medium	Medium	Medium
ANPR	High	Very High	High	High	Medium
Face Recognition	Very High	Very High	High	Very High	Low
AI Risk Scoring	High	Medium	Medium	High	Medium
Multi-Campus	Medium	Medium	High	Very High	Low
Smart Gates	High	Very High	Very High	Very High	Low

